

CONSOLIDACIÓN ARQUITECTÓNICA · ETAPA II

Documento de Consolidación Arquitectónica — ERP Datafast

ERP Datafast ISP

EtapII · Fecha: 2026-08-06 · Rama `main`, commit base `f8d52b00`

De la auditoría al conocimiento arquitectónico estructurado

Propósito

Transformar los hallazgos de la [Auditoría Arquitectónica \(EtapI\)](#) en conocimiento arquitectónico estructurado. Este documento identifica **qué debe preservarse, qué debe corregirse y qué principios deben gobernar la evolución** del ERP.

Fecha de levantamiento: 2026-08-06 · Rama: main · Commit base: f8d52b00

Alcance: consolidar la arquitectura existente. NO implica reconstruir el ERP.

Principio rector: Documento: Consolidación completa — capítulos 0 a 13

No se está reconstruyendo el ERP. Se está consolidando su arquitectura.

Ninguna recomendación de este documento implica reescribir el sistema. Todas son evoluciones incrementales sobre la plataforma existente, y varias consisten explícitamente en **extender un patrón que el propio ERP ya inventó y aplicó bien en uno o dos sitios** al resto del sistema.

Capítulos

#	Capítulo	Archivo
0	Correcciones a la Etapa I	este archivo
1	Arquitectura General	01-arquitectura-backend.md
2	Arquitectura del Backend	01-arquitectura-backend.md
3	Arquitectura del Frontend	02-frontend-datos.md
4	Arquitectura de Datos	02-frontend-datos.md
5	Modelo de Negocio	03-negocio-integraciones-flujos.md
6	Integraciones	03-negocio-integraciones-flujos.md
7	Flujos del Negocio	03-negocio-integraciones-flujos.md
8	Dependencias	04-dependencias-riesgos-fortalezas.md
9	Reutilización	04-dependencias-riesgos-fortalezas.md
10	Riesgos Arquitectónicos	04-dependencias-riesgos-fortalezas.md
11	Fortalezas	04-dependencias-riesgos-fortalezas.md
12	Inventario Arquitectónico	05-inventario-arquitectonico.md
13	Recomendaciones Estratégicas para la Consolidación Arquitectónica	06-recomendaciones-estrategicas.md

CAPÍTULO 0 — Correcciones a la Auditoría de Etapa I

Al profundizar para esta etapa se encontraron **estructuras arquitectónicas que la Etapa I no detectó** porque la extracción inicial buscó por convención de nombre de archivo (`*.service.ts` , `*.controller.ts` , `*.entity.ts`) y estas viven en subdirectorios con otra convención. Se corrigen aquí porque las dos primeras eran afirmaciones **de carga**: sobre ellas se habrían apoyado recomendaciones equivocadas.

0.1 Corrección 1 — La capa de repositorio Sí existe

Lo que decía la Etapa I: "No existe capa de repositorio. Los servicios inyectan `Repository<T>` y `DataSource` y ejecutan SQL directamente."

Lo real (medido): existe en **6 módulos**, 1.614 LOC:

Repositorio	LOC
<code>facturacion/repositories/factura.repository.ts</code>	409
<code>pagos/repositories/pago.repository.ts</code>	358
<code>contratos/repositories/contrato.repository.ts</code>	333
<code>tickets/repositories/ticket.repository.ts</code>	212
<code>smartolt/repositories/onu.repository.ts</code>	153
<code>clientes/repositories/cliente.repository.ts</code>	149

Por qué importa: no hay que *introducir* una capa de repositorio — hay que **completar** una que ya existe, y que además está exactamente en los módulos correctos (los del núcleo comercial y financiero). El patrón ya está probado en producción y su forma ya está decidida (`ContratoRepository` inyecta `DataSource` , obtiene los `Repository<T>` en el constructor y expone métodos de dominio con `empresaId` obligatorio y soft-delete respetado).

Esto cambia la naturaleza de la recomendación correspondiente: de "adoptar un patrón nuevo" a "extender el patrón vigente a los 38 módulos restantes".

0.2 Corrección 2 — Existen puertos y adaptadores formales en TypeScript

Lo que decía la Etapa I: los drivers Python eran "el único patrón adapter formal del sistema".

Lo real (medido): hay **cuatro puertos declarados en el backend TypeScript**:

Puerto (interfaz)	Adaptadores	LOC
<code>olt-nativo/interfaces/olt-provider.interface.ts</code> (<code>IOltProvider</code>)	<code>nativo-ssh.provider.ts</code> (285), <code>smartolt.provider.ts</code> (385), <code>adminolt.provider.ts</code> (304)	1.143
<code>pagos/adaptadores/adaptador-cobro.interface.ts</code>	ninguno — deliberado	140 + spec
<code>aprovisionamiento/interfaces/provisionamiento-provider.interface.ts</code>	<code>mock-provisionamiento.provider.ts</code>	18

Puerto (interfaz)	Adaptadores	LOC
<code>olt-nativo/ztp/</code> — driver ACS	<code>genieacs.driver.ts</code> (390) + <code>registry.ts</code> + <code>resolver.ts</code> + <code>device-profiles/</code> + <code>parameter-maps/</code>	2.579

`IOltProvider` no es una interfaz nominal: **impone reglas de implementación por contrato escrito**, y son las correctas —

1. Nunca propagar una excepción al llamador; todo error se devuelve como `OltOperacionResult { exitoso: false, mensaje, latenciaMs }`.
2. Medir latencia incluyendo el tiempo de conexión SSH/HTTP.
3. **No modificar el estado de la BD desde dentro de un proveedor** — los proveedores son adaptadores puros de protocolo; el estado lo actualizan el Router y el CircuitBreaker.

Eso es una arquitectura hexagonal correcta en el dominio más crítico del sistema. La Etapa I la reportó como ausente.

0.3 Corrección 3 — Hay más subsistemas con diseño explícito de los reportados

Subsistema	Ubicación	Contenido
Catálogo de capacidades	<code>olt-nativo/capabilidad/</code>	<code>capability.engine.ts</code> (motor genérico de reglas puras), <code>olt-capability-catalog.ts</code> , <code>olt-model-catalog.ts</code> , <code>cpe-provisioning-catalog.ts</code> , <code>olt-baseline-standard.ts</code> (+ 2 specs)
Reglas de compliance	<code>olt-nativo/compliance/</code>	<code>olt-compliance-rules.ts</code> (+ spec)
Subsistema ZTP	<code>olt-nativo/ztp/</code>	2.579 LOC: driver GenieACS, registry, resolver, contratos, perfiles de dispositivo, mapas de parámetros, auth CWMP, preset de ONU (+ 7 specs)
Dominio de planta externa	<code>planta-externa/domain/</code>	<code>planta-externa-maquina-estados.ts</code> y <code>presupuesto-optico.ts</code> (+ specs)
Orquestador de velocidad	<code>mikrotik/servicios/velocidad/</code>	<code>velocidad-orquestador.service.ts</code> , <code>velocidad.service.ts</code> , <code>mangle.service.ts</code> , <code>queue-tree-cliente.service.ts</code> — con estrategia de queue seleccionable (<code>simple_queue</code> / <code>queue_tree</code> / <code>pcq</code> / <code>sin_limite</code>)

Consecuencia: hay **dos máquinas de estados declarativas**, no una (`ftth-maquina-estados` y `planta-externa-maquina-estados`). El patrón ya se replicó por decisión propia del equipo, lo que demuestra que es adoptable y no un caso aislado.

0.4 Lo que las correcciones cambian en el diagnóstico

Diagnóstico Etapa I	Diagnóstico corregido
"Faltan patrones arquitectónicos"	Los patrones existen y son buenos. Lo que falta es que sean obligatorios y uniformes.
"Hay que introducir repositorios"	Hay que extender los repositorios existentes
"Solo Python tiene adaptadores"	Hay 4 puertos en TS; falta aplicarlos donde aún no
"Una máquina de estados declarativa"	Dos, replicada voluntariamente

Este es el hallazgo central de la Etapa II y condiciona todo el documento: el problema del ERP Datafast **no es carencia de arquitectura, es cobertura desigual de una arquitectura que ya demostró funcionar**. Los patrones nacieron reactivamente —cada uno resolviendo un incidente concreto— y se aplicaron solo donde dolía. Nunca se declararon obligatorios, así que un módulo nuevo no los hereda.

0.5 Corrección 4 — Dominios que el enunciado da por existentes

El enunciado de Etapa II lista dominios que el ERP **no tiene implementados**. Se registran aquí para que no entren en el modelo de negocio como si existieran:

Dominio del enunciado	Estado real medido
SUNAT / Facturación Electrónica	No implementado. Existe la página <code>configuracion/facturacion-electronica</code> en el frontend y <code>pdf.service.ts</code> genera comprobantes, pero no hay cliente SUNAT, ni OSE, ni firma XML, ni envío/recepción de CDR.
Inventario	No implementado como módulo de negocio. Existe la página <code>(dashboard)/inventario</code> y <code>olt_onu_inventario</code> (inventario <i>observado</i> de ONUs en la OLT), pero no hay control de stock, almacenes ni descuento de materiales.
GIS	Existe como capacidad , no como módulo: coordenadas en <code>contratos</code> , CTE <code>PUNTOS_SERVICIO</code> , mapa MapLibre/OSM, geocoding por cola Google. No hay módulo GIS.
ChatBot	No existe. <code>crm-nativo</code> es bandeja de conversación humana.
SMS	No existe proveedor en el gateway de mensajería.

0.6 Nota de método

Todo dato numérico de este documento proviene de extracción mecánica sobre el árbol de código en el commit indicado. Las valoraciones (criticidad, probabilidad, impacto, prioridad) están marcadas como tales y son juicio arquitectónico, no medición.

Las limitaciones declaradas en la Etapa I siguen vigentes y acotan este documento: **no hay APM, ni métricas de request, ni volúmenes de tabla de producción, ni pruebas de carga**. Ninguna recomendación de este documento depende de un dato que no se pueda verificar en el repositorio.

Capítulos 1–2 — Arquitectura General y Arquitectura del Backend

CAPÍTULO 1 — Arquitectura General

1.1 Qué es realmente el ERP Datafast

No es un monolito tradicional ni una arquitectura de microservicios. Es un patrón concreto y poco común que conviene nombrar con precisión, porque casi todas las decisiones futuras dependen de entenderlo bien:

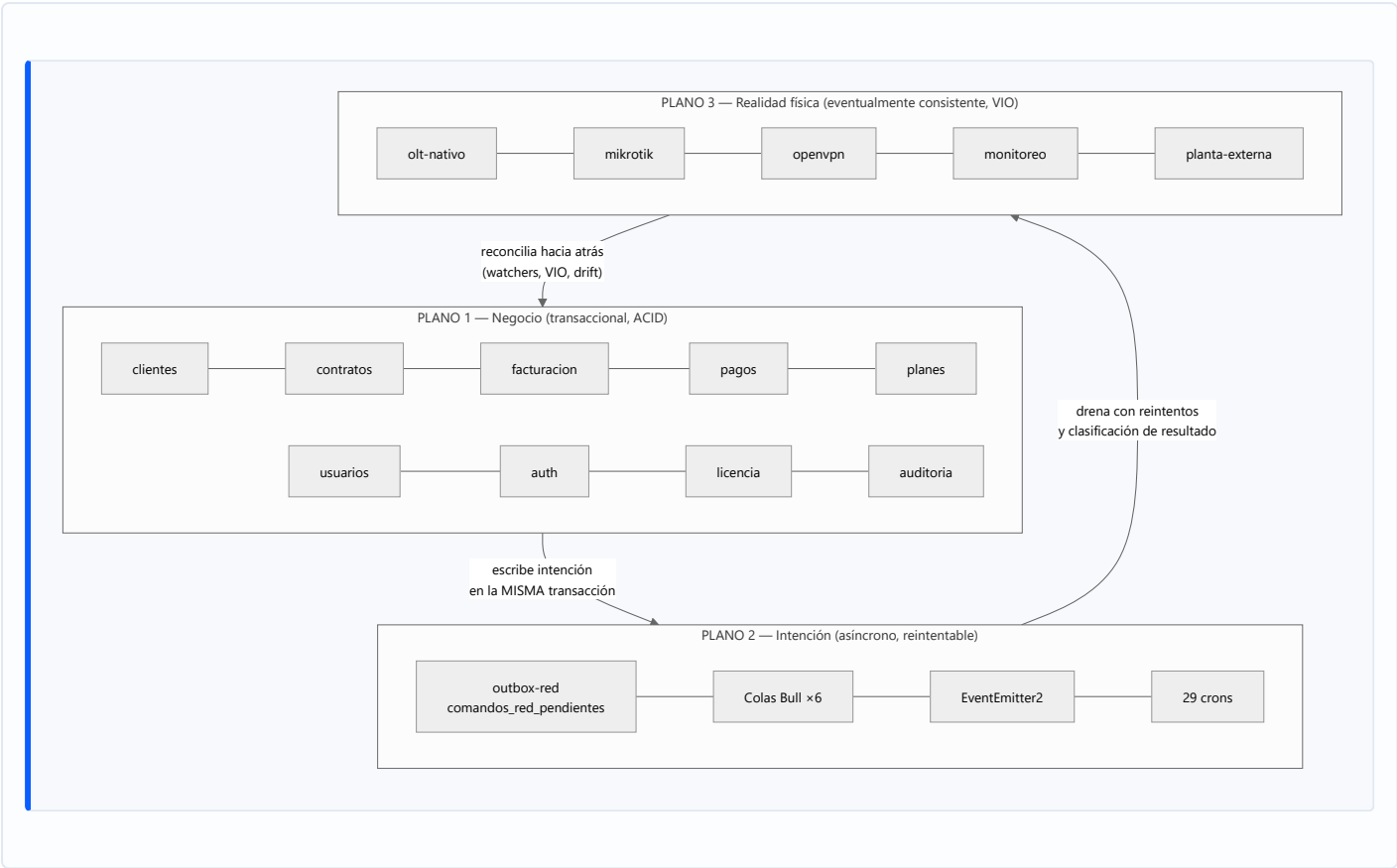
Monolito modular de despliegue segmentado por rol, con un satélite de protocolo y un plano de red gobernado por outbox transaccional.

Descompuesto:

Componente del enunciado	Qué significa aquí
Monolito modular	Un solo <code>dist/main.js</code> , 44 módulos NestJS con fronteras declaradas por <code>@Module</code>
Despliegue segmentado por rol	El mismo binario corre como 3 procesos PM2 distintos, diferenciados solo por variables de entorno (<code>RUN_CRONS</code> , <code>RUN_MIGRATIONS</code> , <code>WA_ENABLED</code>)
Satélite de protocolo	<code>olt-automation-service</code> (FastAPI) existe por una razón de ecosistema de librerías (netmiko/paramiko son Python), no por separación de dominio
Outbox transaccional	Las mutaciones de red no se ejecutan en línea: se persisten en <code>comandos_red_pendientes</code> dentro de la transacción del negocio y las drena un cron

1.2 Los tres planos del sistema

La organización lógica real del ERP no sigue las carpetas. Sigue **tres planos con garantías distintas**, y esa es la lectura que un arquitecto necesita:

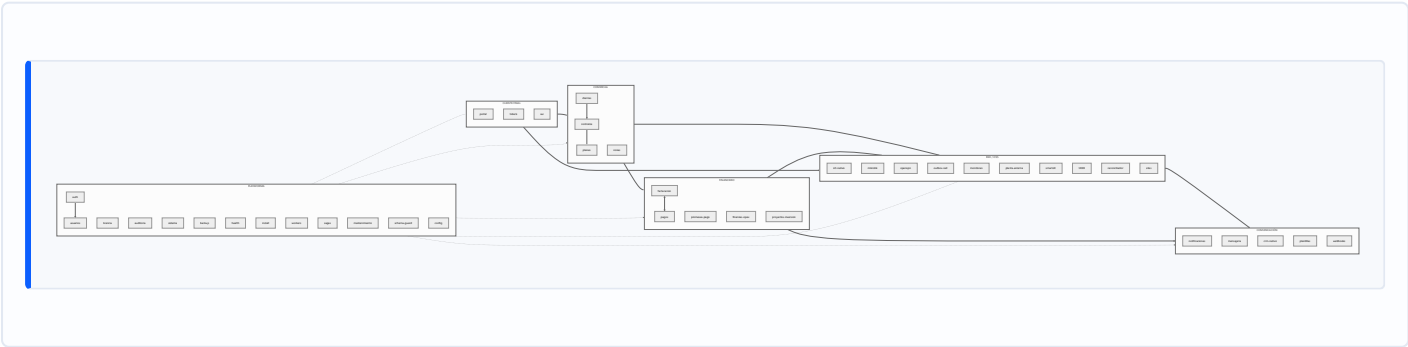


Plano	Garantía	Verdad	Fallo típico
1 — Negocio	ACID en PostgreSQL	La BD es la verdad	Violación de invariante contable
2 — Intención	Entrega eventual con reintentos acotados	La verdad es "esto debe pasar"	Comando agotado / claim huérfano
3 — Realidad física	Consistencia eventual verificada (VIO)	El hardware es la verdad; la BD es una creencia	Discordancia físico↔lógico (ONU huérfana)

La regla que ordena todo el sistema: el plano 1 nunca llama al plano 3 directamente en una operación de negocio. Escribe en el plano 2. Cuando esa regla se salta —y se salta en las operaciones interactivas del operador— aparecen los problemas de latencia y timeout documentados.

1.3 Organización lógica por dominios

Los 44 módulos se agrupan en 6 dominios funcionales reales:



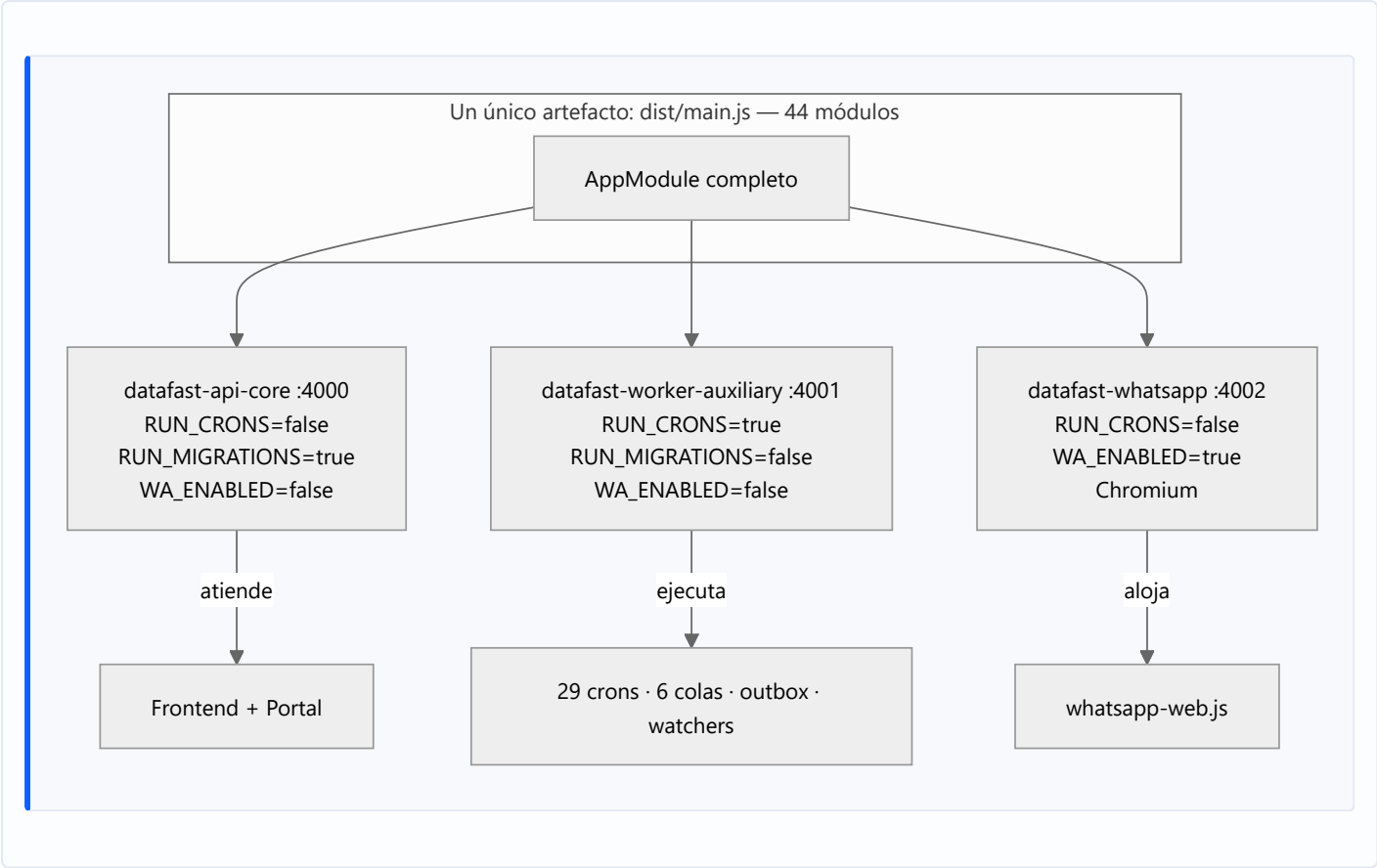
1.4 Los cuatro mecanismos de comunicación y cuándo se usa cada uno

Esta es la tabla que más falta hace al equipo: **hoy la elección del mecanismo no está normada**, y de ahí sale buena parte de los problemas.

#	Mecanismo	Implementación	Cruza proceso	Sobrevive reinicio	Uso actual real
1	Llamada directa (DI)	Inyección NestJS	No	—	La mayoría de interacciones entre módulos
2	EventEmitter2	@nestjs/event-emitter	No — in-process	No	25 listeners, casi todos solo para encolar
3	Colas Bull	Redis db=2	Sí	Sí	6 colas: cobranza, facturación, notificaciones, campañas, google-sync, velocidad
4	Outbox	Tabla PostgreSQL	Sí	Sí, transaccionalmente	Toda mutación de red disparada por el negocio
5	HTTP	axios	Sí	No	Backend → olt-automation-service , GenieACS, terceros

Hecho crítico poco evidente: el bus de eventos es in-process. Un evento emitido en datafast-api-core no llega a datafast-worker-auxiliary . Funciona porque los listeners no hacen trabajo: encolan en Bull, que sí cruza procesos vía Redis. **El bus de eventos es, en la práctica, un adaptador hacia las colas.** Si algún día un listener hiciera trabajo real, fallaría silenciosamente según el proceso en que se emitiera el evento.

1.5 Segmentación de procesos — el mapa de responsabilidad



Proceso	Responsabilidad exclusiva	Si muere...
api-core	Atender peticiones; ejecutar migraciones al arrancar	El ERP deja de responder, pero la cobranza y el outbox siguen
worker-auxiliary	Todo lo automático	El ERP responde normalmente mientras la red deja de aplicarse, nadie se corta, nadie se reactiva y ningún watcher repara nada — sin ninguna señal en la interfaz
whatsapp	Chromium	Se pierde la bandeja de WhatsApp; nada más
olt-automation-service	Todo el acceso a OLT	Ninguna operación FTTH funciona
frontend	Render	Nadie entra

El aislamiento es correcto y deliberado (Chromium fuera del worker tras dejar el VPS con 87 MB libres; migraciones en un solo proceso tras la colisión del 21/07). La debilidad no está en la segmentación sino en que **la diferenciación es por variable de entorno leída dentro de cada servicio**: un servicio nuevo que olvide comprobar `RUN_CRONS` ejecutará su cron en los tres procesos a la vez.

1.6 Estilo arquitectónico por dominio (no es uniforme, y no debería serlo)

Dominio	Estilo real	¿Adecuado?
Comercial / Financiero	Transaction Script sobre TypeORM + repositorio parcial	Sí — dominio CRUD-intensivo con invariantes contables
Red / OSS	Ports & Adapters + máquina de estados + saga + outbox	Sí — es el estilo correcto para consistencia eventual contra hardware
Comunicación	Event-driven + Strategy	Sí
Cliente final (portal)	Fachada sobre otros dominios, con auth propia	Sí
Plataforma	Aspectos transversales (guards/interceptors globales)	Sí

Conclusión del capítulo: la heterogeneidad de estilos **no es un defecto**: cada dominio usa el estilo que su naturaleza exige. El defecto es que **la elección no está escrita en ninguna parte**, así que un módulo nuevo elige por imitación del vecino más cercano en el árbol de carpetas, no por naturaleza del problema.

CAPÍTULO 2 — Arquitectura del Backend

2.1 Composición

Capa	Elementos	Estado
Entrada HTTP	46 controladores, ~560 endpoints, prefijo <code>/api/v1</code>	Uniforme
Aspectos transversales	4 guards + 5 interceptors + 1 pipe + 1 filtro, todos globales	Uniforme y correcto
Servicios de aplicación	~160 <code>*.service.ts</code>	Heterogéneo en tamaño
Repositorios	6 módulos (1.614 LOC)	Parcial — 14 % de cobertura
Puertos y adaptadores	4 puertos, 5 adaptadores	Parcial, pero en los sitios correctos
Dominio explícito	2 máquinas de estados, <code>ResultadoOperacion</code> , catálogo de capacidades, presupuesto óptico	Parcial — solo en red
Entidades	81 TypeORM sobre ~120 tablas	Parcial — 39 tablas sin entidad
Persistencia directa	445 llamadas <code>.query()</code>	Muy extendido
Procesos automáticos	29 crons, 6 colas, 6 processors, 25 listeners	Concentrados en un proceso

2.2 Los componentes más críticos

Criticidad = (impacto de su fallo) × (irreversibilidad del daño). **Valoración.**

#	Componente	Por qué es crítico	Daño si falla mal
1	<code>outbox-red.service.ts</code>	Es la única frontera transaccional entre el negocio y el hardware	Un comando duplicado corta a un cliente pagado; uno perdido deja a un moroso navegando
2	<code>provision-ftth.service.ts</code>	Escribe en el plano físico	ONU huérfana: existe en la OLT y no en el ERP (o al revés)
3	<code>politica-facturacion.service.ts</code>	Fórmula única del ciclo de cobro	Corte antes del vencimiento a todo el parque (ya ocurrió: 05/08)
4	<code>aplicador-factura.service.ts</code>	Único escritor del saldo	Dinero aplicado a facturas anuladas (ya ocurrió en la copia de <code>adelantos</code>)
5	<code>cobranza.worker.ts</code>	Decide quién se queda sin servicio	Corte masivo indebido
6	<code>LicenciaGuard</code>	Primer guard global	Bloquea el ERP completo, incluido <code>auth</code>
7	<code>connection-pool.service.ts</code> (MikroTik)	Único acceso a routers desde NestJS	Pérdida de gestión de toda la planta WISP
8	<code>openvpn</code> (CCD + certs)	Único canal hacia los MikroTik	Sin VPN no hay red gestionable

#	Componente	Por qué es crítico	Daño si falla mal
9	<code>olt-operation-router.service.ts</code> + <code>IoltProvider</code>	Enruta y clasifica todo lo que va a la OLT	Clasificación errónea → 1.788 reintentos (ya ocurrió)
10	<code>ftth-maquina-estados.ts</code>	Define qué transiciones son legales	Un origen faltante deja ONUs huérfanas (ya ocurrió con <code>suspendido</code>)

2.3 Componentes que concentran demasiadas responsabilidades

Componente	Métrica	Responsabilidades mezcladas
<code>olt-nativo</code> (módulo)	25.659 LOC · 41 servicios · 24 entidades · 11 crons	Inventario · configuración declarativa · 3 pools de recursos · provisión FTTH · TR-069/ZTP · saga de wizard · locking/idempotencia · salud/firmware · multi-proveedor
<code>olt-nativo.controller.ts</code>	1.845 LOC · ~150 endpoints	17 grupos funcionales distintos en un archivo
<code>provisioning.py</code>	4.792 LOC	Todos los modelos de OLT × todas las operaciones × verificación × rollback
<code>ClienteDetalle.tsx</code> (frontend)	3.776 LOC	Ficha, contratos, facturación, pagos, ONU, router, historial, tickets
<code>contratos.service.ts</code>	3.894 LOC (módulo) · 8 dependencias	Contrato + pool IPv4 + activación + prórroga + disparo de aprovisionamiento
<code>pagos.service.ts</code>	5.836 LOC (módulo)	Registro + canales + conciliación + Mercado Pago + arqueo + adelantos + extorno
<code>sistema</code>	13 endpoints	Watchers + eventos + update + reinicio + crontab + logs de notificación + gateway

Matiz importante sobre `olt-nativo` : su tamaño es en parte **irreducible** — el dominio FTTH es genuinamente el más complejo del negocio. Pero contiene al menos **cinco subdominios separables con fronteras ya visibles en el propio árbol** (`capability/` , `compliance/` , `domain/` , `ztp/` , `providers/`). El módulo ya se está dividiendo solo; lo que falta es formalizarlo.

2.4 Componentes con mayor riesgo de mantenimiento

Riesgo = probabilidad de que un cambio rompa algo no relacionado. **Valoración.**

Riesgo	Componente	Causa medible
Muy alto	<code>olt-nativo.controller.ts</code>	150 endpoints en un archivo: cualquier merge lo toca
Muy alto	Las 39 tablas sin entidad	Un cambio de esquema no rompe la compilación — incluye outbox, lock FTTH y saga
Alto	<code>provisioning.py</code>	Sin tipado fuerte, sin tests, 4.792 LOC, toca hardware real
Alto	<code>contratos</code>	8 dependencias salientes; es el nodo del que más cosas cuelgan
Alto	Los 4 ciclos de dependencia	Un cambio de firma puede requerir tocar ambos lados a la vez
Alto	<code>workers.constants.ts</code>	Importado por 6 módulos solo por constantes: acopla módulos que no ejecutan workers
Medio-alto	Los 29 crons en un proceso	Un cron nuevo compite con la cobranza y el outbox

Riesgo	Componente	Causa medible
Medio-alto	Las 445 llamadas <code>.query()</code>	El SQL crudo no se refactoriza con el compilador
Medio	<code>ClienteDetalle.tsx</code>	3.776 LOC: el archivo que todo el equipo toca
Medio	Triggers de negocio en PostgreSQL	3 reglas financieras/de red fuera del alcance de los tests

2.5 Análisis por tipo de componente

2.5.1 Controladores

Fortaleza: convención uniforme, versionado URI, validación por DTO, respuesta normalizada.

Debilidades medidas:

- Distribución desbalanceada: 1 controlador con 150 endpoints, 8 con menos de 5.
- `@RequirePermission` aplicado en **4 de 44 módulos** (`contratos`, `planes`, `zonas`, `promesas-pago`). Los demás dependen solo de rol.
- `@SetMetadata('skipAudit', true)` aplicado sistemáticamente solo en `contratos`: el resto audita lecturas de alto volumen.
- El aislamiento multi-tenant depende de que cada handler pase `empresaId` — no hay guard que lo garantice.

2.5.2 Servicios

Fortaleza: los servicios del plano de red devuelven `ResultadoOperacion` (vocabulario de dominio), no excepciones HTTP. Es una decisión avanzada y correcta.

Debilidades:

- Cobertura del vocabulario limitada al plano de red; el plano financiero sigue lanzando excepciones HTTP a consumidores que a veces son máquinas.
- Servicios con acceso directo a `DataSource` conviviendo con servicios que usan repositorio, **dentro del mismo módulo**.
- No hay separación explícita entre servicio de aplicación (orquesta) y servicio de dominio (decide).

2.5.3 Repositorios

Existen en 6 módulos. Ver Cap. 0.1. **La forma ya está decidida y es correcta**; el problema es la cobertura (14 %) y que su existencia no impide que el mismo módulo consulte por fuera.

2.5.4 Entidades

- 81 entidades para ~120 tablas → **39 tablas sin tipar**, entre ellas las más críticas del plano de intención (`comandos_red_pendientes`, `ftth_operacion_lock`, `operacion_wizard`, `operacion_wizard_paso`) y del financiero (`pago_extorno`, `cierre_caja`, `cuentas_bancarias`).
- Convención de ubicación inconsistente (`entities/` vs raíz del módulo).
- `smartolt/entities/onu.entity.ts` mapea la tabla `olts`.
- Riesgo conocido y documentado: columnas `string | null` sin `type:` explícito crashean el backend en frío bajo SWC.

2.5.5 Cron jobs

29 tareas, todas en `worker-auxiliary`. Análisis en Cap. 10 (riesgos). Puntos estructurales:

- La protección es `RUN_CRONS` comprobado **dentro de cada servicio** — es una convención, no un mecanismo.
- Cinco barridos pesados en 100 minutos de madrugada (03:00 → 04:40).
- Frecuencias que van de **30 segundos** (XUI) a diaria, sin criterio declarado.
- Ningún cron declara su presupuesto de tiempo ni su cap de trabajo. `reconciliar()` itera sin límite ni lock.

2.5.6 Workers y colas

Fortaleza notable: matriz de prioridades y política de reintentos por tipo de job (`JOB_OPTIONS`), con goteo y jitter para campañas y concurrencia 1 para no saturar el gateway. Esto es diseño operativo maduro.

Debilidad: `workers.constants.ts` mezcla tres cosas —nombres de cola, tipos de payload y política de reintentos— y por eso lo importan 6 módulos, creando acoplamiento donde solo hacía falta una constante.

2.5.7 Eventos

- 15 eventos de notificación + 8 de dominio + 3 de WebSocket.
- **El bus es in-process** (§1.4): es funcionalmente un adaptador hacia Bull.
- No hay catálogo versionado de eventos ni contrato de payload; el payload se define en el emisor.

2.5.8 Guards, interceptors y middleware

Los mejor resueltos del sistema: 4 guards globales en orden correcto, 5 interceptors, filtro único. **Observación estructural:** `TimeoutInterceptor(30 s)` es global y las operaciones de hardware legítimas duran 90–150 s. El sistema lo resuelve haciéndolas asíncronas, que es correcto — pero significa que **cualquier operación síncrona nueva contra hardware nace rota**, y nada lo impide en tiempo de compilación.

2.5.9 Configuración

Fortaleza destacable: la regla de portabilidad multi-VPS está implementada, no solo escrita — sin IPs ni dominios en el repositorio, lazy getters para constantes de módulo, `.env.example` como contrato, `ecosystem.config.js` sin secretos. Es de las prácticas más maduras del proyecto.

Debilidad: la configuración *de negocio* está dispersa en 6 tablas sin servicio unificado (`empresas` , `configuracion_facturacion` , `portal_config` , `olt_onu_preset` , `openvpn_config` , `olt_proveedor_config`), y hay campos que se guardan y no se usan (`mora` , `reconexión` , `esquemaImpuesto` , `impuesto1` , `avisoPantalla`).

Capítulos 3–4 — Arquitectura del Frontend y Arquitectura de Datos

CAPÍTULO 3 — Arquitectura del Frontend

3.1 Composición medida

Aspecto	Valor
Framework	Next.js 14 — App Router
Páginas (<code>page.tsx</code>)	92 — 3 auth, 68 dashboard, 10 portal, install, raíz
Componentes <code>.tsx</code>	57.435 LOC en <code>components/</code> + <code>app/</code>
Directorios de componentes	23
Estado global	4 stores Zustand
Hooks propios	5
Cientes de API	27 archivos en <code>lib/api/</code> + <code>lib/api.ts</code>
Contextos	1 (<code>undo-redo.context.tsx</code>)
Tests	2 (<code>como-llegar.test.ts</code> , <code>coordenadas.test.ts</code>)

3.2 Organización — tres convenciones simultáneas

```
components/  
├─ atoms/ (4) · molecules/ (0) · organisms/ (1)    ← Atomic Design, abandonado  
├─ ui/ (8) · shared/ (5) · layout/ (4)             ← Por función  
└─ clientes/ · red/ · olt/ · monitoreo/ · portal/ · ... ← Por dominio (el real)
```

Hallazgo: `molecules/` está **vacío** y `organisms/` tiene **1 archivo**. Atomic Design se intentó y se abandonó, pero los directorios siguen ahí. La organización real y viva es **por dominio**, y coincide razonablemente con los módulos del backend.

Riesgo concreto: un desarrollador nuevo no sabe dónde poner un componente. Existen tres respuestas defendibles y ninguna es la correcta por convención escrita.

3.3 Distribución del peso

Directorio	Archivos	LOC	Observación
<code>clientes</code>	11	8.932	El más pesado con diferencia; 3.776 en un solo archivo
<code>red</code>	17	8.616	Routers, VPN, ONUs, redes IPv4
<code>configuracion</code>	16	5.084	22 páginas de configuración
<code>monitoreo</code>	16	4.154	
<code>portal</code>	13	3.340	
<code>olt</code>	13	3.205	
<code>planta-externa</code>	11	3.107	

Directorio	Archivos	LOC	Observación
finanzas	7	2.115	
pagos	3	1.465	
contratos	4	1.266	
integraciones	1	1.110	Un único archivo
ui + shared + atoms	17	1.016	Todo lo reutilizable del sistema

El dato que define el capítulo: el sistema tiene **57.435 LOC de frontend** y solo **1.016 LOC (1,8 %)** en componentes genuinamente reutilizables (`ui` + `shared` + `atoms`). El 98 % es componente de dominio, mayormente no reutilizado.

3.4 Componentes desproporcionados

Archivo	LOC	Responsabilidades mezcladas
components/clientes/ClienteDetalle.tsx	3.776	Ficha, contratos, facturación, pagos, ONU, router, historial, tickets, acciones
components/clientes/ClienteWizard.tsx	1.452	Alta guiada completa
app/(dashboard)/mensajeria/whatsapp/page.tsx	1.289	Bandeja completa en la página
components/red/RoutersContent.tsx	1.262	Listado + detalle + acciones + estado en vivo
components/integraciones/GoogleIntegrationDashboard.tsx	1.110	Todo Google en un archivo
app/(dashboard)/finanzas/registro/page.tsx	1.059	Lógica de negocio en la página
components/clientes/ModalProvisionFtth.tsx	998	Wizard FTTH completo
components/red/AgregarRouterWizard.tsx	874	Wizard VPN completo

Patrón detectado: los componentes más grandes son **fichas maestras y wizards**. No es casualidad: son las pantallas donde convergen varios dominios del backend, y el frontend no tiene ninguna capa que componga esa convergencia — la compone el componente.

3.5 Capa de acceso a datos

Fortaleza: existe una capa dedicada — `lib/api/` con **27 archivos**, uno por dominio del backend (`clientes.ts` , `contratos.ts` , `olt-nativo.ts` , `mikrotik.ts` , `portal.ts` ...). El alineamiento con los módulos del backend es alto y facilita razonar sobre el sistema completo.

Debilidades:

- Los tipos de request/response se **duplican** respecto a los DTO del backend (`frontend/src/types` vs `backend/src/**/dto`), sin paquete compartido ni generación desde OpenAPI — pese a que el backend expone Swagger.
- No hay librería de data-fetching con cache (TanStack Query/SWR): cada pantalla gestiona su propio `loading` / `error` / `refetch` .
- Existe `portal.test.ts` como único test de cliente API.

3.6 Estado

Store	Alcance
<code>auth.store.ts</code>	Sesión del operador
<code>empresa.store.ts</code>	Empresa activa (multi-tenant)
<code>portal.store.ts</code>	Sesión del abonado
<code>theme-customizer.store.ts</code>	Preferencias visuales

Evaluación: el uso de Zustand es **correcto y contenido** — solo 4 stores, todos de estado verdaderamente global. No hay el antipatrón habitual de meter datos de servidor en el store global.

Consecuencia del acierto: al no cachear datos de servidor en el store, cada pantalla los pide de nuevo. Sin librería de cache HTTP, eso se traduce en peticiones repetidas — visible en las pantallas que consultan hardware en vivo.

3.7 Hooks

5 hooks propios, todos justificados: `useDebounce`, `useInactivityLogout`, `useMonitoreo` (WS), `useOltSocket` (WS), `useProcedimientoWizard` (heartbeat del wizard FTTH).

`useProcedimientoWizard` **merece mención:** implementa el lado cliente del contrato de wizard (heartbeat contra el servidor). Es la prueba de que la regla de "wizard que se cierra anula lo no confirmado" está cableada de extremo a extremo, no solo en el backend.

3.8 Navegación y protección de rutas

- Grupos de rutas correctos: `(auth)`, `(dashboard)`, `portal/(privado)`.
- `middleware.ts` protege en el edge.
- **Separación fuerte ERP↔Portal:** distinto grupo de rutas, distinto store, distinta sesión, distinto vhost de Nginx con API acotada por regex.
- Detalle: la ruta del instalador es `/installl` (con typo). Funciona, pero queda en la URL pública.

3.9 Formularios

No hay librería de formularios unificada (React Hook Form / Formik) ni esquema de validación compartido. La validación vive en el componente y **se duplica respecto a los `class-validator` del backend**. En wizards de 800–1.400 LOC esto es una fuente estructural de divergencia: el frontend puede aceptar lo que el backend rechaza, y al revés.

3.10 Fortalezas del frontend

1. Organización por dominio alineada con el backend.
2. Capa `lib/api/` dedicada y completa (27 dominios).
3. Estado global mínimo y bien delimitado.
4. Separación ERP↔Portal fuerte y verificada.
5. Realtime resuelto con hooks dedicados por dominio.

6. Utilidades de dominio extraídas y testeadas (`senal-ftth.ts` , `coordenadas.ts`).
7. `mock-data/` separado del código productivo (aunque esté en el árbol).

3.11 Oportunidades de mejora del frontend

1. Convención única de organización de componentes.
 2. Descomposición de las fichas maestras y wizards (>800 LOC).
 3. Tipos generados desde el Swagger del backend en vez de duplicados a mano.
 4. Capa de cache/estado de servidor.
 5. Librería y esquema único de formularios.
 6. Cobertura de tests (hoy: 2 archivos para 57.435 LOC).
 7. Ampliar `ui` / `shared` — hoy es el 1,8 % del código.
-

CAPÍTULO 4 — Arquitectura de Datos

4.1 Perfil general

Aspecto	Valor
Motor	PostgreSQL 16, TZ <code>America/Lima</code>
Tablas	~120
Entidades TypeORM	81 (39 tablas sin entidad)
Migraciones	215 archivos, dos juegos (<code>core/</code> , <code>auxiliary/</code>)
Índices	376 sentencias <code>CREATE INDEX</code>
Triggers	27
Funciones	13
Vistas	4
Estrategia	<code>synchronize: false</code> — solo migraciones, modo <code>each</code>
Ejecutor de migraciones	Solo <code>api-core</code>
<code>max_connections</code>	100 · pool 15/proceso
Logging SQL	Desactivado

4.2 Clasificación del modelo

Tablas maestras (baja escritura, alta lectura)

`empresas` · `usuarios` · `roles` · `permisos` · `planes` · `zonas` · `sites` · `routers` · `olt_dispositivos` · `olt_boards` · `canal_pago` · `cuentas_bancarias` · `bancos_isp` · `formas_pago_isp` · `comprobantes_config` · `plantillas_mensajes` · `plantillas_abonados` · `umbrales_alerta` · `olt_onu_preset` · `olt_baselines` · `xui_servidores` · `portal_config`

Tablas transaccionales (alta escritura)

`facturas` · `pagos` · `pago_aplicaciones` · `pago_extorno` · `cargos_pendientes` · `egresos_ingresos` · `promesas_pago` · `cierre_caja` · `tickets` · `ordenes_trabajo` · `crm_mensajes` · `notificaciones_logs`

Tablas de estado de infraestructura (espejo del mundo físico)

`ftth_onu_registro` · `contrato_onu_config` · `olt_onu_inventario` · `tr069_device` · `ips_asignadas` · `segmentos_ipv4` · `vpn_clientes` · `pe_*` (9 tablas)

Tablas de coordinación (el plano de intención)

`comandos_red_pendientes` · `ftth_operacion_lock` · `operacion_wizard` · `operacion_wizard_paso` · `saga_log` · `olt_service_port_pool` · `olt_mgmt_ip_pool` · `olt_onu_id_pool`

Tablas de serie temporal (crecimiento sin techo)

Tabla	Ritmo de escritura
<code>metricas_monitoreo</code>	Cada minuto × n° de dispositivos
<code>nodos_mediciones</code>	Ídem
<code>metricas_onu_optical</code>	Por lectura óptica
<code>olt_health_snapshots</code>	Por poll de salud
<code>consumo_datos</code> / <code>consumo_snapshot</code>	Cada 15 min × n° de contratos

Tablas de auditoría y bitácora

`auditoria_logs` · `entity_versions` · `eventos_sistema` · `olt_operacion_log` · `ftth_rollback_log` · `reconciliation_log` · `google_sync_logs` · `notificaciones_logs` · `contratos_historial` · `clientes_historial_estados`

4.3 Multi-tenancy

Modelo: **discriminador por columna** (`empresa_id`) con índices únicos compuestos por empresa y parciales por estado:

<code>uq_usuarios_empresa_email</code>	<code>uq_clientes_empresa_documento</code>
<code>uq_contratos_empresa_numero</code>	<code>uq_contratos_empresa_onu</code>
<code>uq_planes_empresa_nombre</code>	<code>uq_rols_empresa_nombre</code>
<code>uq_segmentos_empresa_red_cidr</code>	<code>uq_routers_empresa_ip_gestion</code> (WHERE activo)
<code>uq_onus_empresa_serial</code>	<code>uq_onus_olt_pon_id</code>
<code>uq_tickets_empresa_numero</code>	<code>uq_ordenes_empresa_numero</code>
<code>uq_plantillas_empresa_tipo_codigo</code>	<code>uq_vpn_clientes_nombre_cert</code>

Fortaleza: el modelo es correcto y los índices parciales (`WHERE deleted_at IS NULL`, `WHERE activo`) permiten soft-delete y reactivación sin colisionar. El índice `uq_routers_empresa_ip_gestion WHERE activo` fue la corrección definitiva del incidente de routers/OLTs duplicados.

Riesgo estructural: el aislamiento en **lectura** depende de que cada consulta incluya `empresa_id`. Con 445 consultas crudas y sin Row-Level Security en PostgreSQL, **una omisión no produce error: produce una fuga silenciosa entre empresas**. La base garantiza que no colisionen; no garantiza que no se lean entre sí.

4.4 Lógica de negocio residente en la base de datos

Objeto	Tipo	Regla que implementa
<code>fn_calcular_deuda_contrato</code>	Función	Cuánto debe un contrato
<code>fn_next_available_ip</code>	Función	Asignación de IP libre de un segmento
<code>fn_sync_factura_saldo</code> + <code>trg_factura_saldo</code>	Trigger	Saldo de la factura
<code>fn_update_ips_usadas</code> + <code>trg_update_ips_usadas</code>	Trigger	Contador de IPs usadas del segmento

Objeto	Tipo	Regla que implementa
<code>pe_recalcular_contadores_nap</code> + <code>trg_pe_contadores_nap</code>	Trigger	Ocupación de una NAP
<code>recalc_tipo_servicio_cliente</code>	Función	Derivación de tipo de servicio
<code>fn_generar_numero_contrato</code> / <code>_ticket</code>	Función	Correlativos
<code>sn_onu_normalizado</code>	Función	Normalización de SN de ONU entre OLT/GenieACS/SmartOLT
<code>fn_cleanup_old_data</code>	Función	Retención

Evaluación equilibrada: los correlativos y la normalización de SN **están bien en la base** — son invariantes que deben cumplirse escriba quien escriba. Los tres triggers de negocio (saldo, IPs usadas, ocupación de NAP) son **decisiones defendibles** (garantizan el invariante aunque el escritor sea un script manual) con un **coste real**: no son visibles desde el código de aplicación, no aparecen en ningún test de NestJS, y un desarrollador que lea `aplicador-factura.service.ts` no ve que el saldo también lo toca la base.

Precedente relevante que apoya mantenerlos: el 28/07 un **UPDATE** directo se saltó las cascadas de aplicación y dejó un cert VPN huérfano reservando una IP. La lección registrada fue que *"el invariante que solo vive en la doc no es invariante"*. Un trigger es la forma más fuerte de que un invariante no dependa de la disciplina del llamador.

4.5 Vistas

Vista	Consumo
<code>v_contratos_completos</code>	Listados con joins precalculados
<code>v_resumen_clientes</code>	Resúmenes y dashboard
<code>v_resumen_financiero</code>	Reportes y dashboard
<code>v_estado_dispositivos</code>	Monitoreo en tiempo real

Observación: las vistas conviven con consultas de aplicación que calculan lo mismo (Cap. 9). Existir no las convierte en la fuente única.

4.6 Riesgos de escalabilidad del modelo de datos

#	Riesgo	Evidencia	Horizonte
1	Series temporales sin particionado ni retención declarada	<code>metricas_monitoreo</code> escribe cada minuto por dispositivo; existe <code>fn_cleanup_old_data</code> pero no hay política visible por tabla	Meses
2	39 tablas sin entidad	Un ALTER no rompe la compilación; incluye outbox, lock FTTH y saga	Inmediato
3	Aislamiento multi-tenant por convención	445 consultas crudas, sin RLS	Al segundo tenant grande

#	Riesgo	Evidencia	Horizonte
4	376 índices sin verificación de uso	No hay lectura de <code>pg_stat_user_indexes</code> ; cada índice penaliza escritura	Al crecer la escritura
5	<code>max_connections=100</code> con 3 procesos × 15	45 de aplicación + migraciones + Evolution + herramientas	Al añadir procesos
6	Consultas sin paginación	<code>GET /clientes/mapa</code> devuelve el parque completo; exportaciones en memoria	Al llegar a miles de abonados
7	Sin logging SQL ni <code>pg_stat_statements</code> declarado	<code>logging: false</code> ; no hay forma de saber qué consulta duele	Ya
8	Dos juegos de migraciones (<code>core</code> + <code>auxiliary</code>) con runners distintos	<code>datasource.ts</code> , <code>datasource.auxiliary.ts</code> , <code>datasource.install.ts</code>	Al instalar un VPS nuevo
9	Correlativos por función de BD sin secuencia declarada por tenant	<code>fn_generar_numero_contrato</code>	Bajo concurrencia alta
10	Ausencia de archivado	Ninguna tabla transaccional tiene política de archivo histórico	Años

4.7 Consistencia del modelo

Fortalezas:

- `synchronize: false` sin excepciones — el esquema solo cambia por migración versionada.
- Migraciones transaccionales (`each`) y un solo proceso migrando.
- `schema-guard` verifica el esquema al arrancar.
- Soft-delete consistente con índices parciales.
- `idx_notif_logs_idempotency_key` UNIQUE: la idempotencia de notificaciones está garantizada por la base, no por la aplicación.

Debilidades:

- Convención de nombres mixta: `pe_*` (prefijo), `olt_*` (prefijo), `ftth_*` (prefijo), y el resto sin prefijo. `olts` / `onus` (SmartOLT) conviven con `olt_dispositivos` / `olt_onu_inventario` (nativo) — **dos modelos del mismo objeto físico**.
- `contrato_onu_config` guarda `revision` / `last_applied_revision` como mecanismo de drift, pero `ftth_onu_registro` no: dos formas distintas de rastrear divergencia en el mismo dominio.
- No hay diccionario de datos ni descripción de columnas.

4.8 El riesgo de datos más grave del sistema

Merece su propia sección porque **no es un riesgo de diseño: es una bomba con temporizador condicional**, y está documentado.

Mecanismo: una ONU con `contrato_onu_config.provisioning_enabled = true` y `last_applied_revision IS NULL` figura como *drift*. `ZtpReconcileCron.reconciliarDiario` (03:30, `America/Lima`) le **reescribe SSID, clave WiFi y credenciales de acceso web** con el preset de la OLT.

Por qué hoy no explota: solo las ONUs que el ERP aprovisionó tienen fila en `contrato_onu_config` . `adoptarOnusHuerfanas` inserta únicamente en `ftth_onu_registro` y **no crea config**, dejando fuera del reconcile a las

ONUs adoptadas. De las 205 ONUs del NODO MALVINAS, la inmensa mayoría está fuera.

Qué lo detonaría: cualquier migración masiva (SmartOLT, MikroWISP, adopción en bloque) que cree `contrato_onu_config` para ONUs preexistentes. El efecto sería sobre **todo el parque a la vez, de madrugada, sin que nadie lo pida**: clientes con años de configuración propia — muchos con su propia clave WiFi — se quedan sin internet en sus dispositivos a la mañana siguiente.

Consulta de verificación obligatoria antes de cualquier migración:

```
SELECT COUNT(*) FROM contrato_onu_config
WHERE provisioning_enabled
      AND (last_applied_revision IS NULL OR last_applied_revision < revision);
```

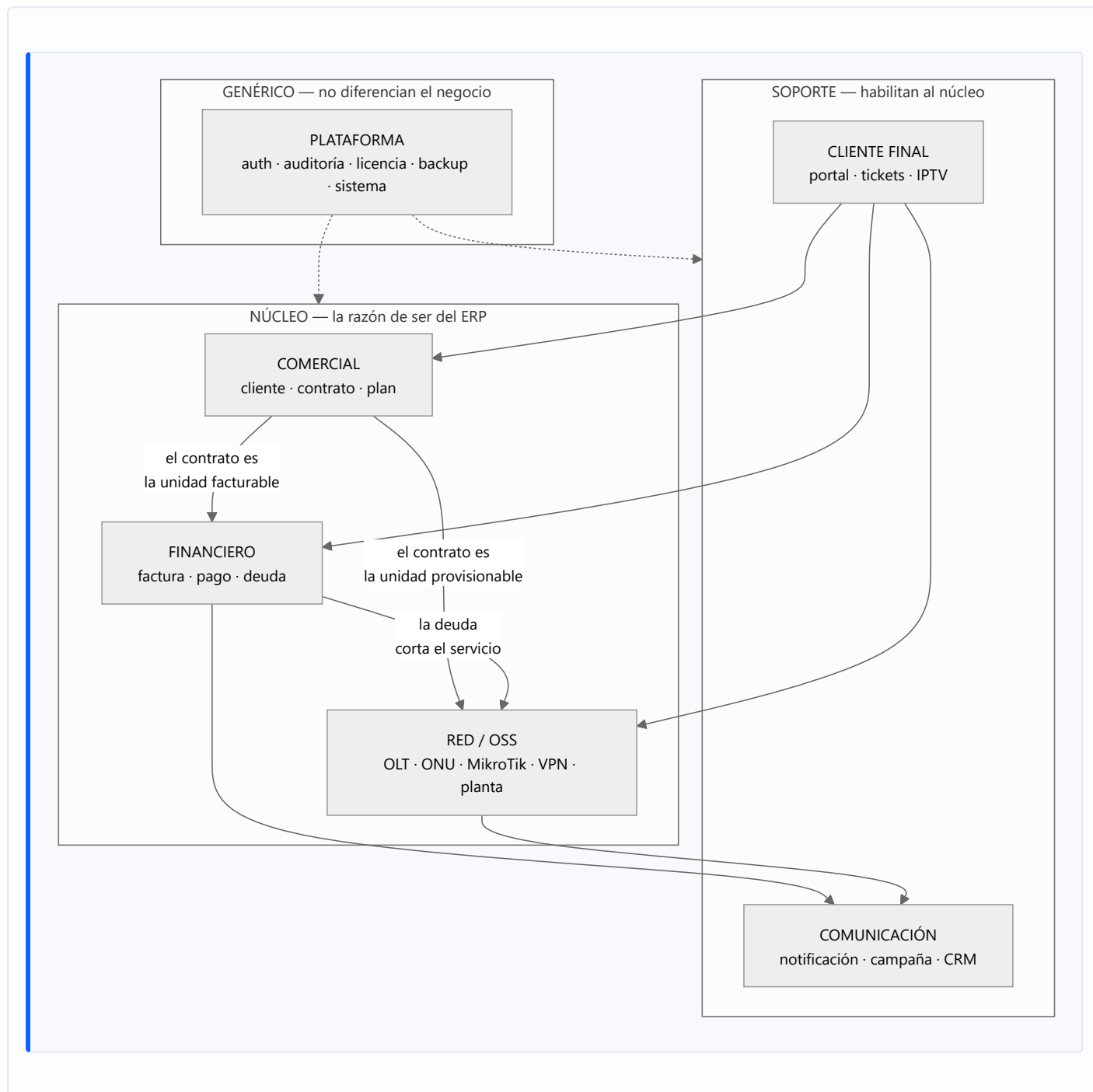
Calificación: el sistema está protegido **por construcción, no por precaución**. La garantía es un efecto lateral de cómo está escrito `adoptarOnusHuerfanas`, no una regla explícita que alguien deba respetar. Nada impide que el próximo desarrollador cree la config "para completar el registro" y detone la mina sin saber que existe.

Capítulos 5–7 — Modelo de Negocio, Integraciones y Flujos

CAPÍTULO 5 — Modelo de Negocio

Se documentan los dominios **existentes**. No se proponen dominios nuevos.

5.1 Mapa de dominios



5.2 El objeto central: el contrato

contrato es la unidad operativa del ERP. No lo es el cliente. Un cliente sin contrato no factura, no consume red y no aparece en el mapa. Todo el sistema pivota sobre él:

Dominio	Qué le cuelga al contrato
Comercial	cliente, plan, zona, coordenadas de instalación
Financiero	facturas, cargos pendientes, promesas de pago, deuda
Red	IP asignada, usuario PPPoE, router, <code>ftth_onu_registro</code> , <code>contrato_onu_config</code> , acometida, puerto NAP
Cliente final	sesión de portal, tickets, consumo, línea IPTV
Coordinación	<code>ftth_operacion_lock</code> , <code>operacion_wizard</code> , comandos de red pendientes

Esto explica —y en buena medida **justifica**— que **contratos** sea el módulo con más dependencias salientes del sistema (8). No es acoplamiento accidental: es el reflejo de que el contrato es el agregado raíz del negocio.

5.3 Ficha de cada dominio

5.3.1 COMERCIAL

Aspecto	Detalle
Responsabilidad	Quién es el abonado, qué servicio contrató, dónde está instalado
Módulos	<code>clientes</code> , <code>contratos</code> , <code>planes</code> , <code>zonas</code> , <code>sites</code>
Entidades raíz	<code>cliente</code> , <code>contrato</code> , <code>plan</code>
Depende de	<code>auth</code> , <code>mikrotik</code> (validación de CIDR y pools), <code>outbox-red</code> , <code>promesas-pago</code> , <code>sagas</code> , <code>smartolt</code> , <code>xui</code>
Es consumido por	Financiero, Red, Cliente final, Comunicación
Invariantes	Un contrato pertenece a una empresa · numeración correlativa por empresa · una ONU por contrato (<code>uq_contratos_empresa_onu</code>) · la IP asignada sale de un segmento con contador mantenido por trigger
Criticidad	Máxima — Core Indestructible

5.3.2 FINANCIERO

Aspecto	Detalle
Responsabilidad	Cuánto debe el abonado, cuánto pagó, en qué estado está su cuenta
Módulos	<code>facturacion</code> , <code>pagos</code> , <code>promesas-pago</code> , <code>finanzas-opex</code> , <code>proyectos-inversion</code>
Entidades raíz	<code>factura</code> , <code>pago</code>
Depende de	<code>contratos</code> , <code>config</code> , <code>auth</code> , <code>workers</code>

Aspecto	Detalle
Invariantes protegidos por test	Un solo escritor del saldo (<code>frontera-dinero.spec.ts</code>) · el extorno es la única reversión (<code>extorno.spec.ts</code>) · una sola fórmula del ciclo de cobro (<code>politica-facturacion.service.spec.ts</code>)
Frontera declarada	<code>pagos/adaptadores/README.md</code> fija el contrato de cobro y prohíbe implementar pasarelas hasta pasar una puerta de estabilidad de 30 días
Criticidad	Máxima

Nota de madurez: este dominio es el que tiene la relación más sana entre reglas e implementación. Cada invariante tiene un test que **nombra el incidente que lo motivó**, y hay un README que explica por escrito por qué un trabajo pendiente está pendiente a propósito.

5.3.3 RED / OSS

Aspecto	Detalle
Responsabilidad	Que el servicio contratado exista físicamente y esté en el estado que el negocio dice
Módulos	<code>olt-nativo</code> , <code>mikrotik</code> , <code>openvpn</code> , <code>outbox-red</code> , <code>monitoreo</code> , <code>planta-externa</code> , <code>smartolt</code> , <code>tr069</code> , <code>reconciliador</code> , <code>sites</code>
Subdominios de <code>olt-nativo</code>	inventario · configuración declarativa (baselines/perfiles/VLANs) · pools de recursos · provisión FTTH · ZTP/TR-069 · saga de wizard · locking · salud/firmware · multi-proveedor
Entidades raíz	<code>olt_dispositivo</code> , <code>ftth_onu_registro</code> , <code>router</code> , <code>vpn_cliente</code> , <code>pe_nap</code>
Invariante fundacional	Nunca un <code>ont</code> en la OLT sin <code>ftth_onu_registro</code> , ni al revés. Sostenido por dos watchers: DELETE (<code>reintentarRollbacksFallidos</code> , estado <code>fallido_rollback</code>) y CREATE (<code>adoptarOnusHuerfanos</code>)
Regla de verdad	El hardware es la verdad; la BD es una creencia que se verifica (VIO)
Criticidad	Máxima — y máxima complejidad

5.3.4 COMUNICACIÓN

Aspecto	Detalle
Responsabilidad	Que el abonado y el operador se enteren de lo que ocurre
Módulos	<code>notificaciones</code> , <code>mensajería</code> , <code>crm-nativo</code> , <code>plantillas</code> , <code>webhooks</code>
Arquitectura	Event-driven: 15 eventos → listener → cola Bull con prioridad → estrategia de envío
Estrategias	WhatsApp nativo, mensajería masiva, SMTP. No hay SMS.
Garantía	Idempotencia por índice UNIQUE en <code>notificaciones_logs</code>
Criticidad	Alta (avisos de corte y alertas de red van por aquí)

5.3.5 CLIENTE FINAL

Aspecto	Detalle
Responsabilidad	Autoservicio del abonado
Módulos	<code>portal</code> , <code>tickets</code> , <code>xui</code>

Aspecto	Detalle
Arquitectura	Fachada sobre Comercial/Financiero/Red con autenticación completamente independiente (<code>PORTAL_JWT_SECRET</code> , cookies, <code>portal-tenant.service</code>)
Superficie	Pública, con vhost propio y API acotada por regex en Nginx
Criticidad	Alta — es superficie expuesta que alcanza hardware (WiFi por TR-069)

5.3.6 PLATAFORMA

Aspecto	Detalle
Módulos	<code>auth</code> , <code>usuarios</code> , <code>licencia</code> , <code>auditoria</code> , <code>sistema</code> , <code>backup</code> , <code>health</code> , <code>install</code> , <code>workers</code> , <code>sagas</code> , <code>mantenimiento</code> , <code>schema-guard</code> , <code>config</code>
Naturaleza	Genérico — no diferencia el negocio, pero su fallo lo detiene
Particularidad	<code>licencia</code> es el único guard que puede apagar el ERP entero , por diseño

5.4 Relaciones entre dominios — reglas de dirección

Relación	Dirección	Naturaleza	Estado
Comercial → Financiero	Unidireccional conceptual	El contrato genera facturas	Hay ciclo real: <code>pagos → contratos</code>
Comercial → Red	Unidireccional conceptual	El contrato se provisiona	Hay ciclo real: <code>mikrotik → contratos</code>
Financiero → Red	Vía outbox	La deuda corta el servicio	Correcta — pasa por el plano de intención
Red → Comercial	Reconciliación	El hardware corrige la creencia	Correcta (watchers)
Cliente final → todos	Lectura + acciones acotadas	Fachada	Correcta
Plataforma → todos	Transversal	Aspectos	Correcta

Las dos relaciones que se salieron de su dirección (`pagos→contratos` , `mikrotik→contratos`) son el origen de dos de los cuatro ciclos del sistema y se analizan en el Cap. 8.

CAPÍTULO 6 — Integraciones

6.1 Inventario con nivel de acoplamiento

Acoplamiento: **Bajo** = sustituible sin tocar dominio · **Medio** = hay abstracción parcial · **Alto** = el proveedor está incrustado en la lógica de negocio.

#	Integración	Propósito	Método	Acoplamiento	Degradable	Riesgo principal
1	GenieACS	Gestión remota de CPE (WiFi, PPPoE, reboot, factory-reset)	HTTP NBI	Medio — hay <code>genieacs.driver.ts</code> + registry + resolver, pero el driver es único y no hay segundo ACS	Sí	Credenciales de connreq duplicadas fuera del repo (provision <code>erp-connreq-creds</code> debe coincidir con el <code>.env</code> de cada VPS)
2	MikroTik RouterOS	Provisión PPPoE, colas, firewall, address-lists	API nativa + SSH	Alto — 3 caminos independientes (NestJS <code>node-routeros</code> , NestJS <code>ssh2</code> , Python <code>mikrotik_pool</code>)	Parcial	Tres pools distintos contra el mismo equipo; ninguna abstracción común
3	OLT (Huawei/V-SOL)	Provisión GPON, verificación, rollback	SSH CLI vía servicio Python	Bajo — <code>IOltProvider</code> + drivers Python	Sí	Serializado por 1 worker uvicorn
4	SmartOLT / AdminOLT	Proveedor OLT alternativo	HTTPS REST	Bajo — implementa <code>IOltProvider</code>	Sí (circuit breaker)	Camino legado en migración
5	OpenVPN	Canal hacia toda la planta MikroTik	Filesystem + callbacks HTTP	Alto — el ERP escribe CCD y certs directamente en el VPS	No	Punto único de falla del plano de red
6	Mercado Pago	Cobro en línea	REST + webhook	Alto — <code>mercadopago.service.ts</code> no usa el <code>adaptador-cobro.interface.ts</code> que ya existe	Sí	Es el único que cobra dinero real y no pasa por la abstracción
7	RENIEC	Validación de identidad en el alta	HTTPS REST	Bajo	Sí	Cacheado; sin proveedor alternativo
8	Google Workspace (Calendar, Contacts, Drive, Maps)	Sincronización y geocoding	OAuth2 + REST, vía cola	Bajo — todo pasa por <code>google-sync</code>	Sí	Tokens cifrados con clave propia
9	Evolution API	WhatsApp transaccional	HTTP + webhook	Medio	Sí	Base de datos propia en el mismo PostgreSQL
10	WhatsApp Web (<code>whatsapp-web.js</code>)	Bandeja CRM	Chromium headless	Alto — proceso PM2 dedicado	Sí	No es API oficial; el aislamiento del proceso es la mitigación
11	SMTP	Correo	SMTP	Bajo — es una <code>strategy</code>	Sí	—

#	Integración	Propósito	Método	Acoplamiento	Degradable	Riesgo principal
1 2	XUI.ONE	IPTV	HTTP REST	Bajo	Sí	Cron cada 30 s contra servicio externo
1 3	Servidor de licencias	Habilitación del ERP	HTTPS + webhook	Máximo — LicenciaGuard es el primer guard global	No, por diseño	Su caída bloquea el ERP completo
1 4	Let's Encrypt / Certbot	TLS	ACME	Bajo	—	—
1 5	OpenStreetMap	Tiles del mapa	Frontend	Bajo	—	—

6.2 Integraciones declaradas y no implementadas

Elemento	Estado	Consecuencia
SUNAT / Facturación electrónica	Página en el frontend, cero backend	Un usuario puede creer que existe
SMS	Sin proveedor en el gateway	Los avisos de corte dependen de WhatsApp
Niubiz / Culqi / Webpay / Stripe	Contrato definido, adaptadores deliberadamente ausentes	Correcto — hay puerta de estabilidad documentada
Telegram (telegraf)	Dependencia instalada, sin uso	Superficie de dependencia sin valor
Twilio	Dependencia instalada, sin uso	Ídem
net-snmp (Node)	Declarada; el SNMP real está en Python	Ídem

6.3 Riesgos transversales de integración

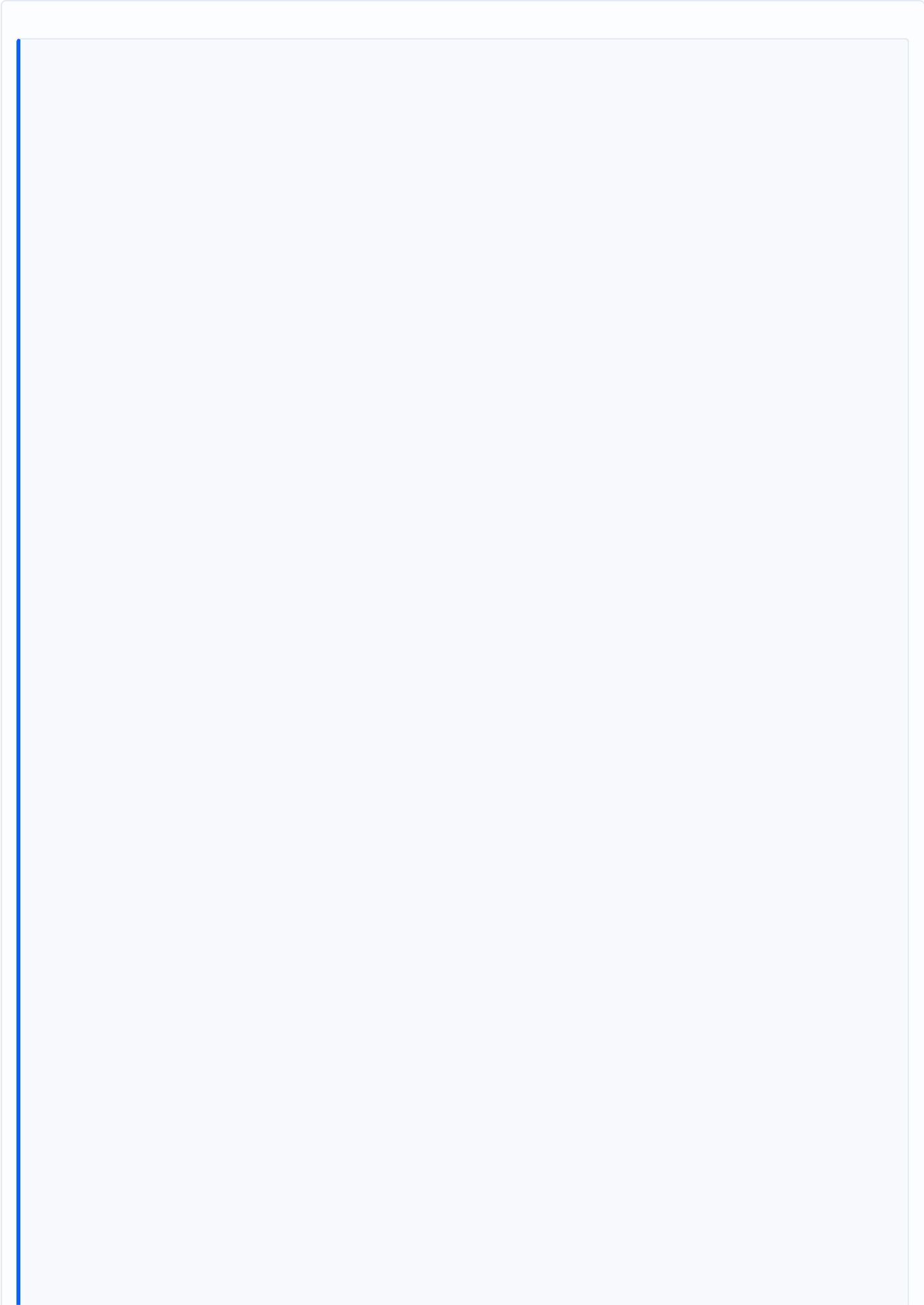
#	Riesgo	Evidencia
1	Acoplamiento del proveedor que cobra dinero	mercadopago.service.ts no implementa adaptador-cobro.interface.ts . La abstracción existe y el único proveedor real no la usa; no está probada contra la realidad
2	Tres caminos hacia MikroTik	Un cambio de credenciales o de política de reintento hay que hacerlo tres veces
3	Configuración externa al repositorio	Credenciales de GenieACS viven en la provision del ACS y en el .env . No hay verificación de coincidencia
4	Ningún contrato de integración versionado	No hay OpenAPI/JSON-Schema de las respuestas externas; un cambio del proveedor se descubre en producción
5	Dependencias instaladas sin uso	telegraf , twilio , net-snmp amplían superficie de seguridad y peso de build sin aportar
6	Licencia como punto único de falla total	Deliberado, pero sin modo degradado ni periodo de gracia documentado

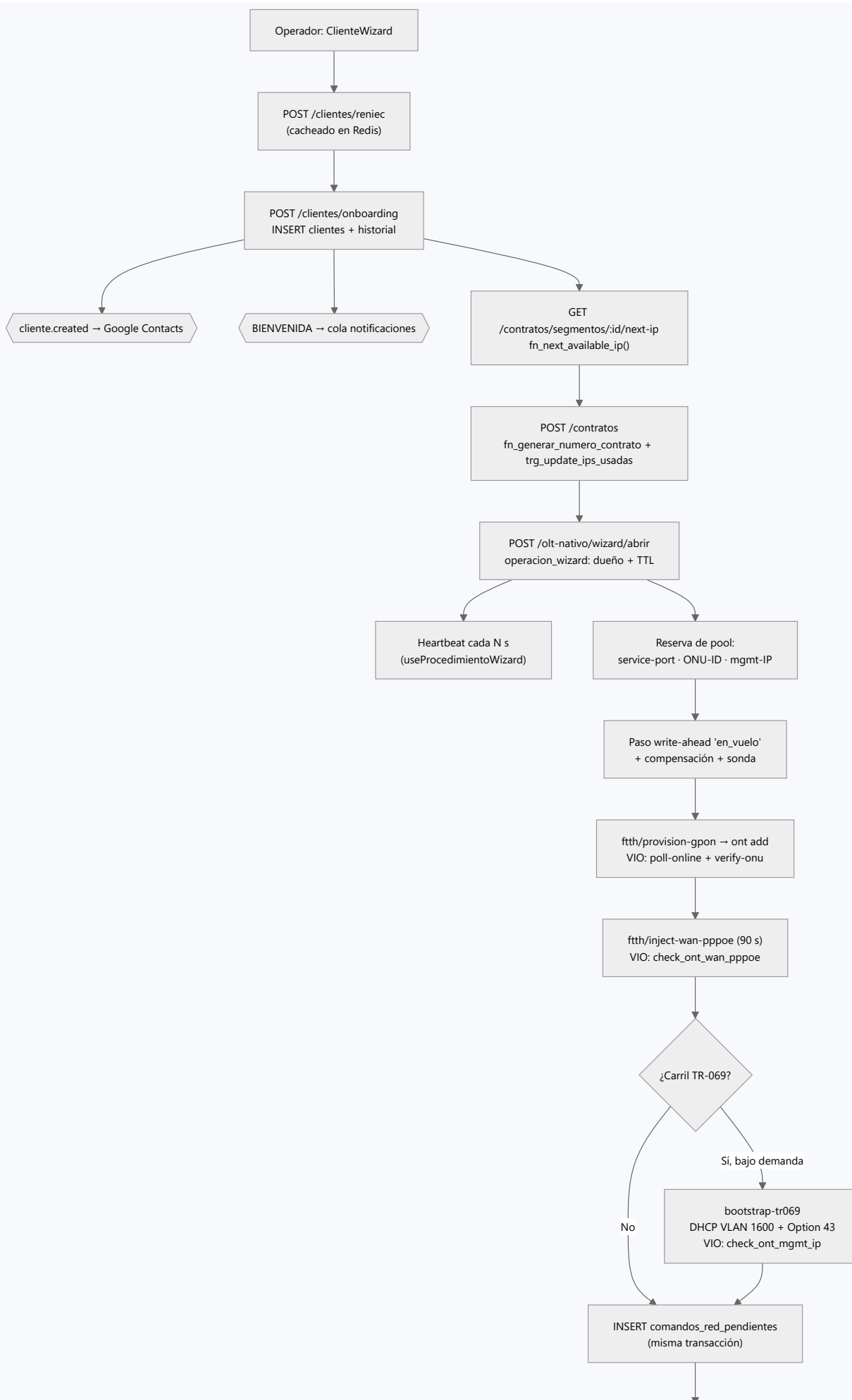
6.4 Fortaleza de integración destacable

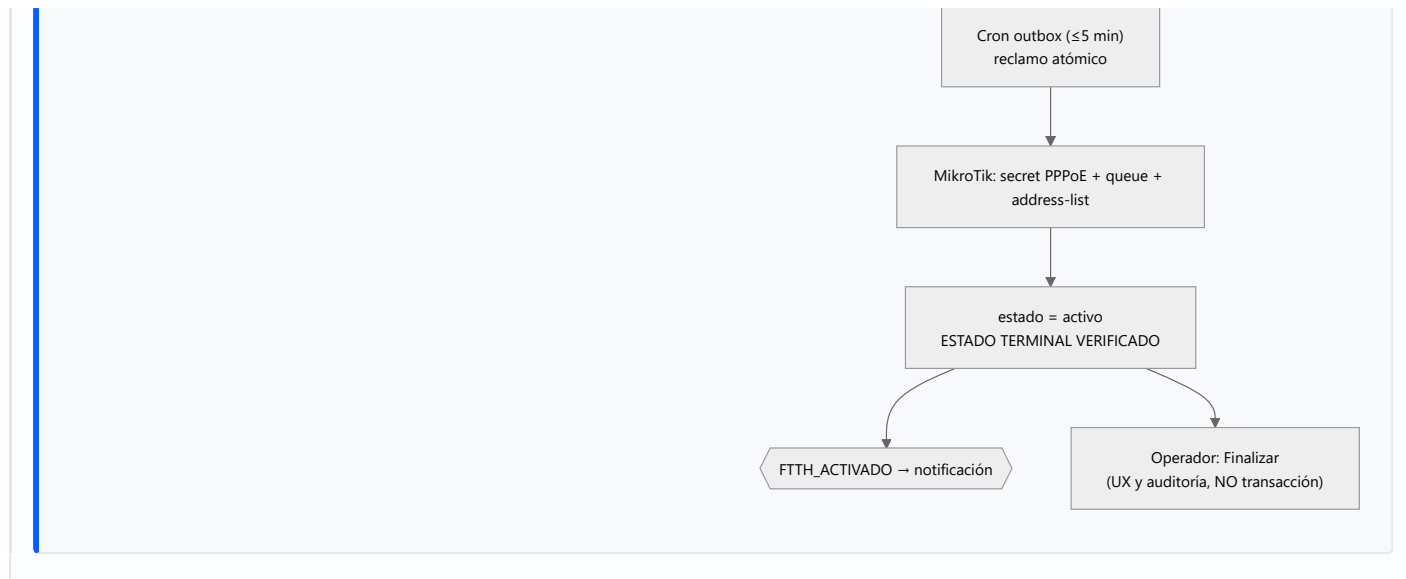
El patrón degradable está **implementado, no solo escrito**: `OnModuleInit` + probe ligero + `ModuleHealthService.registrar('<nombre>', 'degraded', '<razón>')` + `assertNotDegraded()`, con la regla explícita de **nunca relanzar la excepción del probe**. Y tiene su complemento correcto: la lista del **Core Indestructible** — módulos que *deben* crashear el backend si fallan al iniciar, para proteger al proceso anterior en PM2—. La distinción entre "puede degradarse" y "debe morir" está tomada conscientemente, módulo por módulo. Es una decisión de arquitectura de resiliencia poco habitual y bien ejecutada.

CAPÍTULO 7 — Flujos del Negocio

7.1 Alta de abonado con provisión FTTH





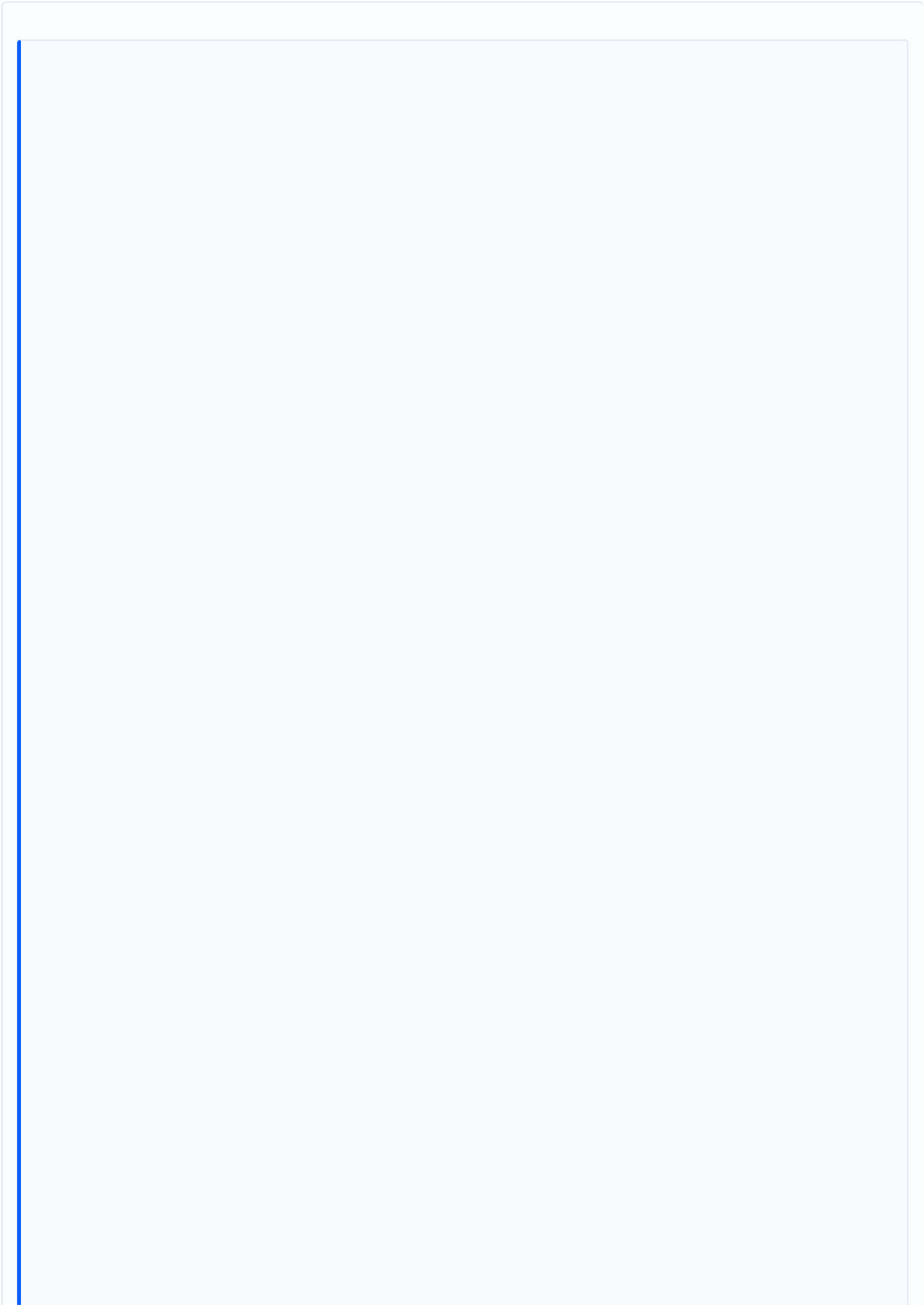


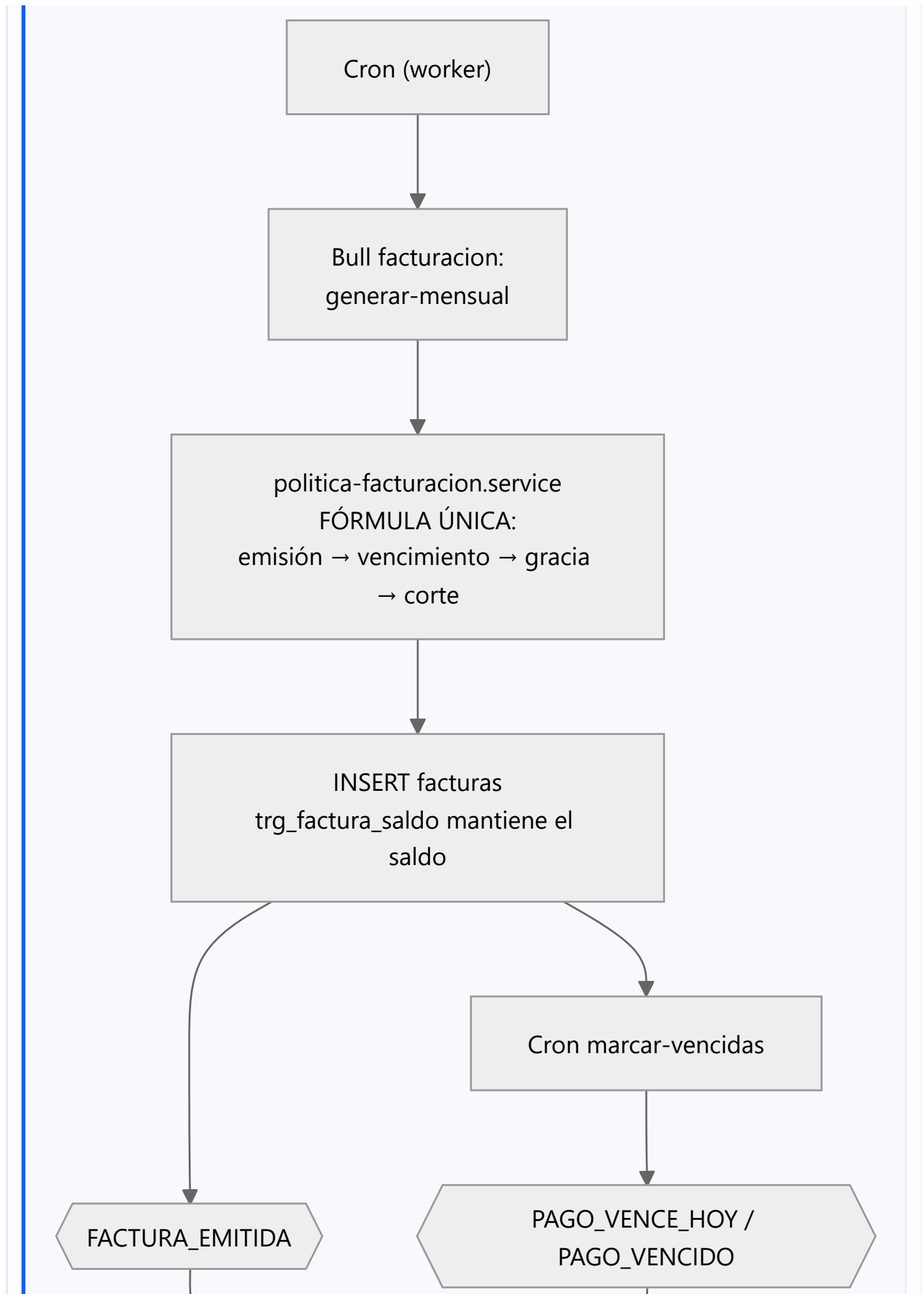
Puntos de diseño clave:

- La **frontera transaccional** es **estado = activo**, no el clic de "Finalizar".
- Si el wizard se cierra o el navegador muere antes, el cron **procesarAnulaciones** (cada 3 min) compensa en orden inverso leyendo **operacion_wizard_paso**.
- Un paso **en_vuelo** es **sospechoso de haberse ejecutado**: se resuelve con su sonda de verificación contra el hardware antes de decidir si hay algo que compensar.

Latencia real de extremo a extremo: la parte OLT es síncrona (segundos a minutos); la parte MikroTik espera al barrido del outbox — **hasta 5 minutos**.

7.2 Facturación mensual



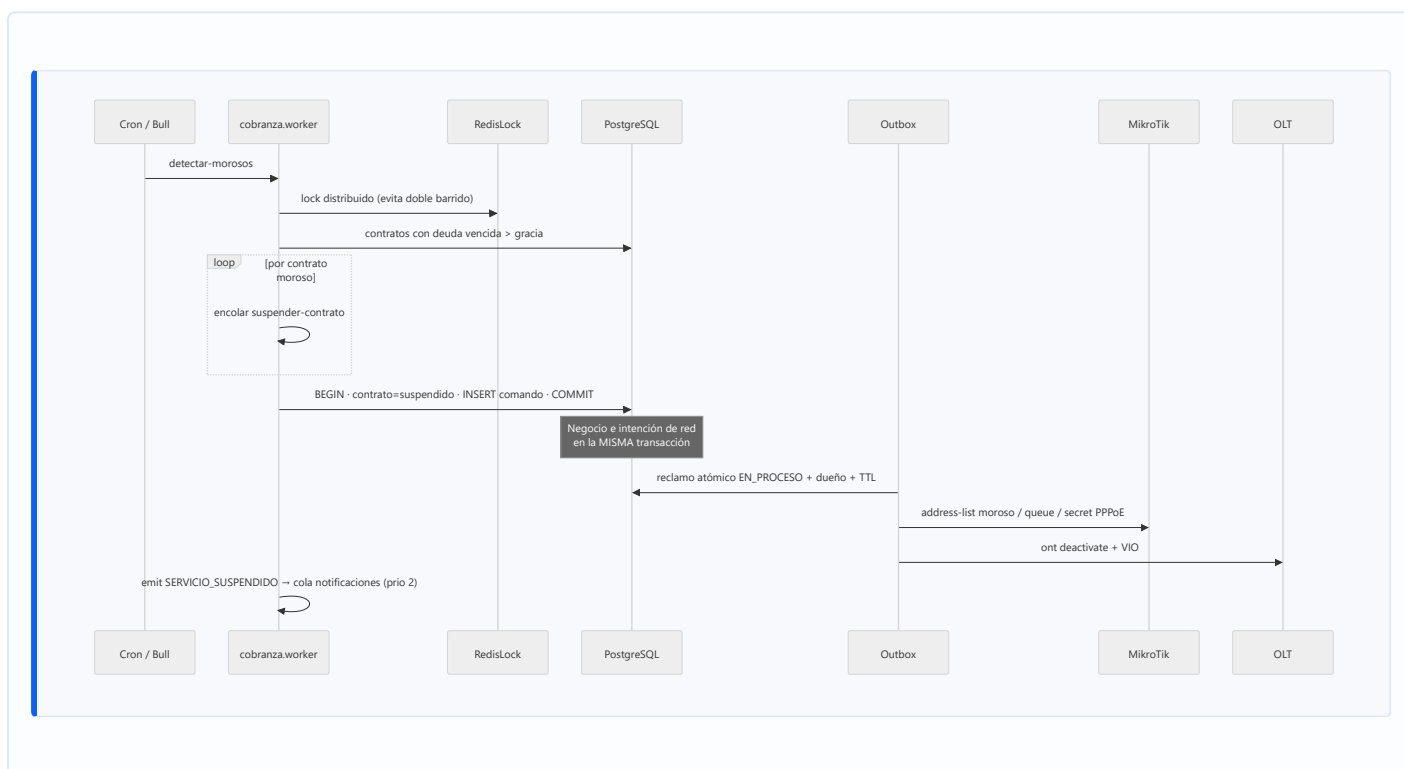


Bull notificaciones (prio 2)

Regla central: el ciclo de cobro tiene **una sola fórmula**. La gracia es la **distancia vencimiento→corte**; no se suma al vencimiento. Antes había tres fórmulas y el corte llegó a caer antes del vencimiento (incidente 05/08).

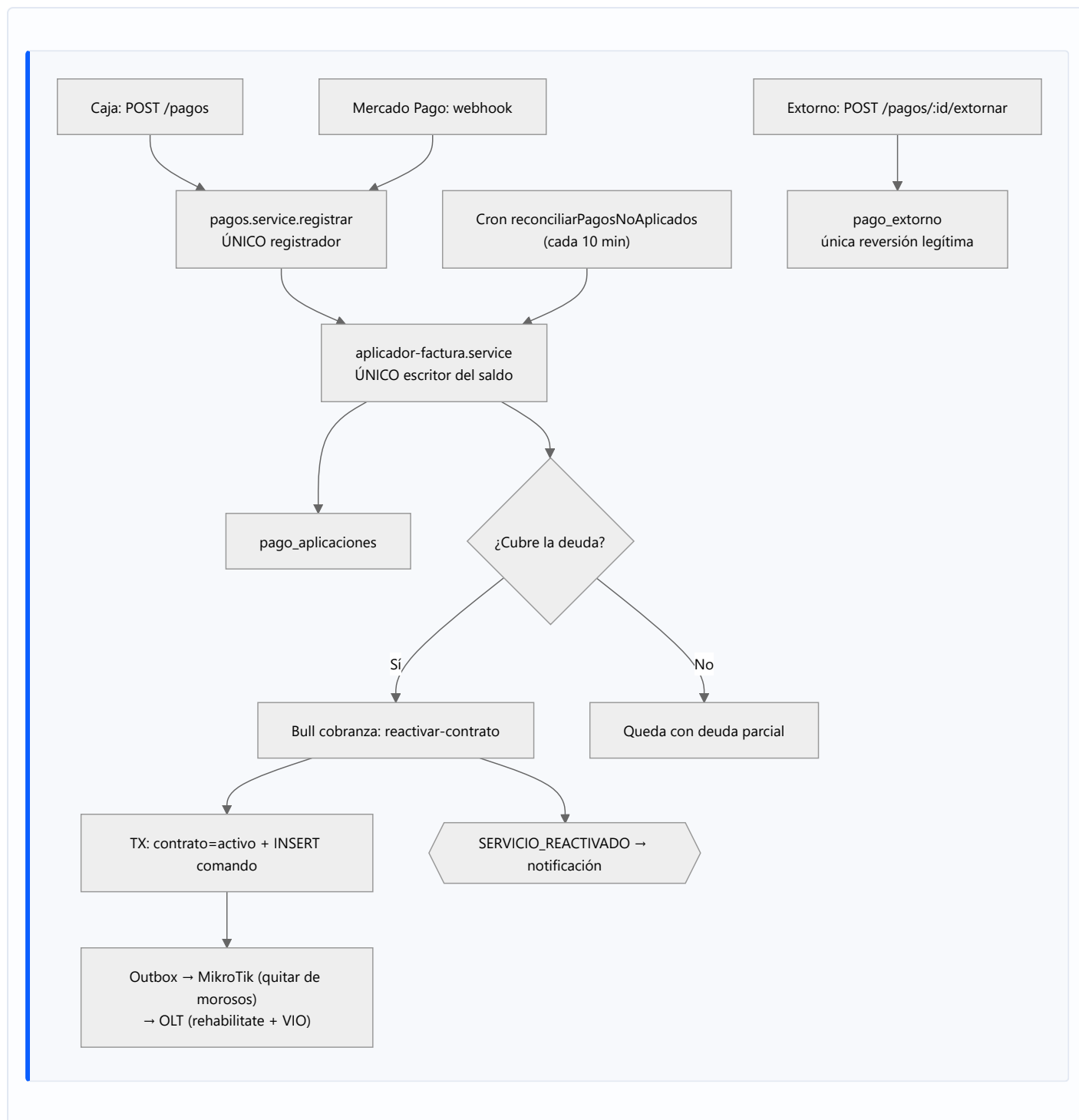
Política de reintento: perfil **MASIVO** — **1 intento, sin reintento**. Un fallo de generación masiva se registra y no se reintenta solo.

7.3 Cobranza y suspensión por deuda



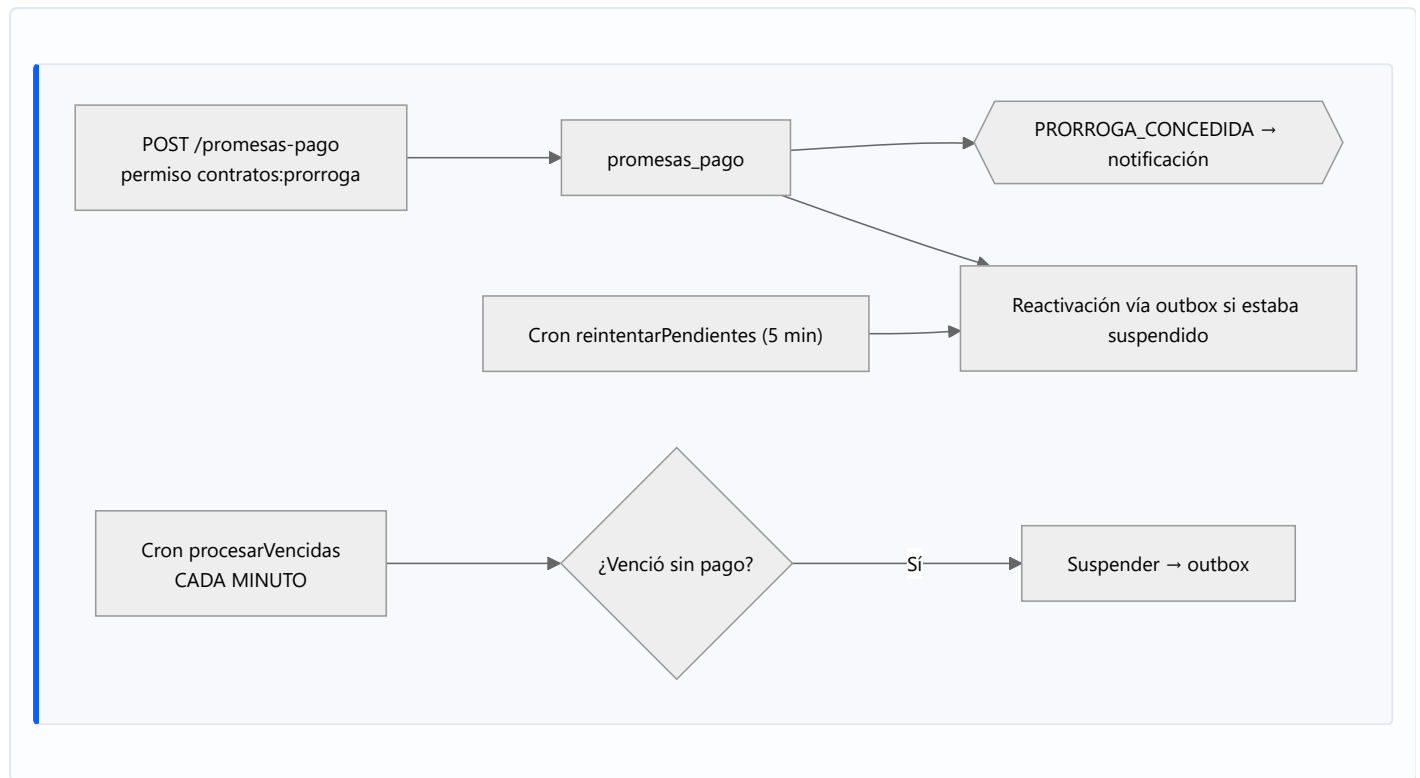
Clasificación del resultado (crítica): solo **400 y 404** son rechazos definitivos. 409/408/429/5xx son reintentables. Un **timeout** es **indeterminado** — se audita, **no se reintenta a ciegas**, porque la operación pudo aplicarse.

7.4 Registro de pago y reconexión



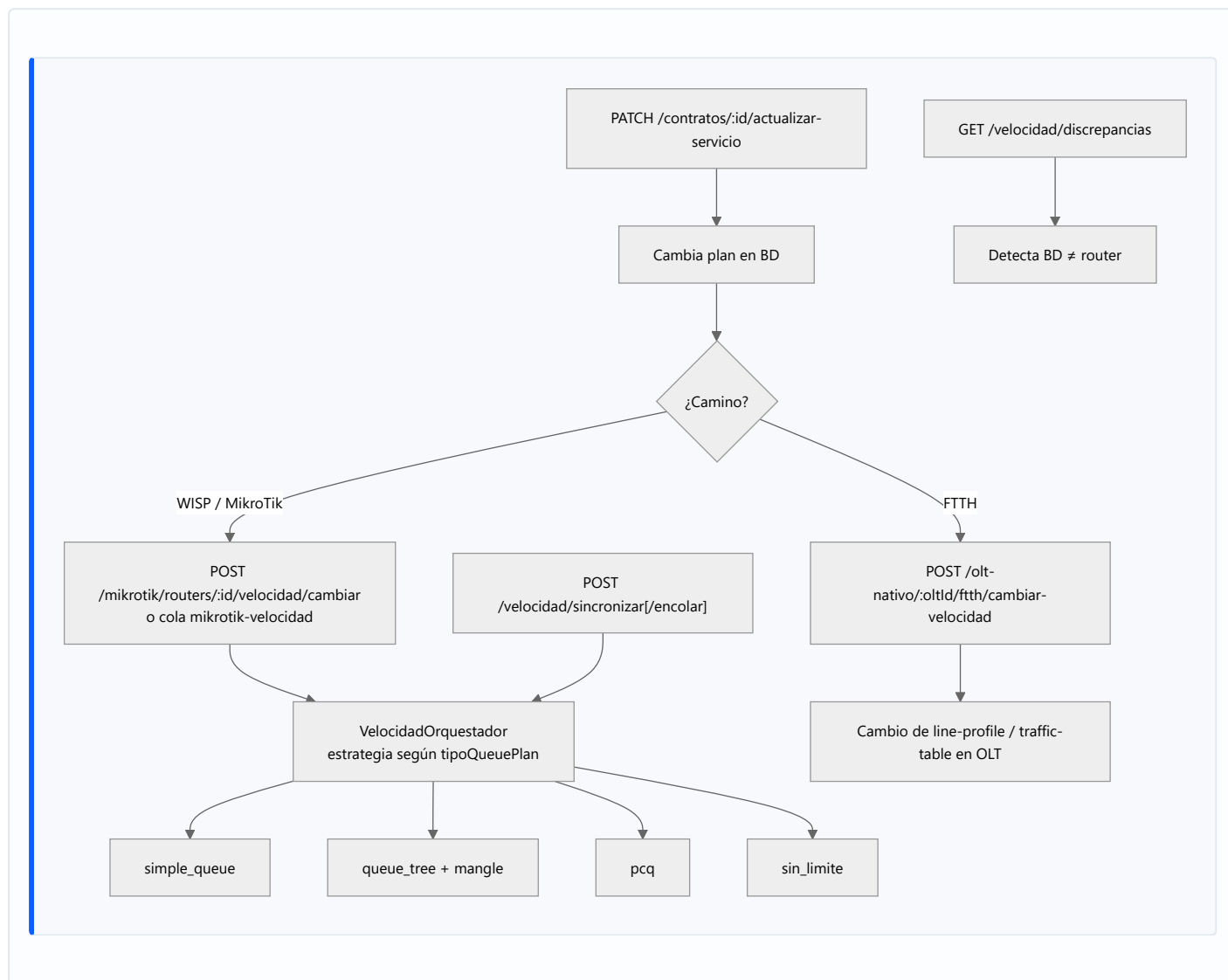
Latencia medida y corregida: REACTIVAR tardaba **287 s** por dos latencias encadenadas (outbox sin drenado inmediato + timeout de 30 s en **rehabilitate**). Tras corregir ambas: **8 s**. El segundo defecto solo se hizo visible al arreglar el primero.

7.5 Prórroga / promesa de pago



Es el cron más frecuente del plano financiero (cada minuto) porque materializa un corte real.

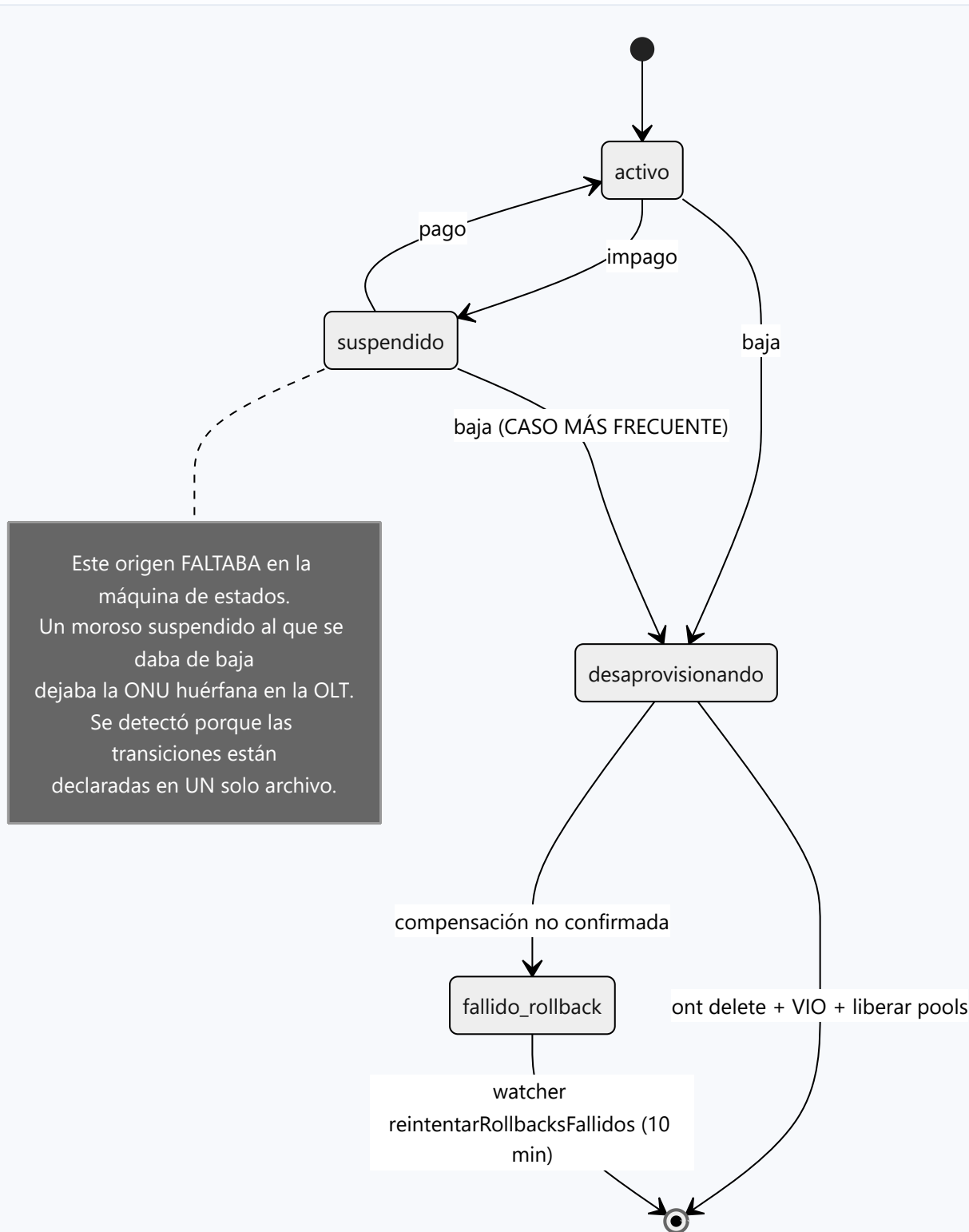
7.6 Cambio de plan / cambio de velocidad



Fortaleza: `VelocidadOrquestador` selecciona la estrategia de limitación según el plan (`simple_queue` / `queue_tree` / `pcq` / `sin_limite`) y el tipo de cliente (`residencial` / `empresarial` / `dedicado`), con soporte de burst. Es un buen ejemplo de patrón Strategy aplicado a un dominio de red.

Debilidad: el cambio de plan **no es una operación única**. Son dos operaciones independientes (BD y red) que pueden divergir; la divergencia se detecta después con `/velocidad/discrepancias`. No pasa por outbox.

7.7 Baja de servicio (desaprovisionamiento)



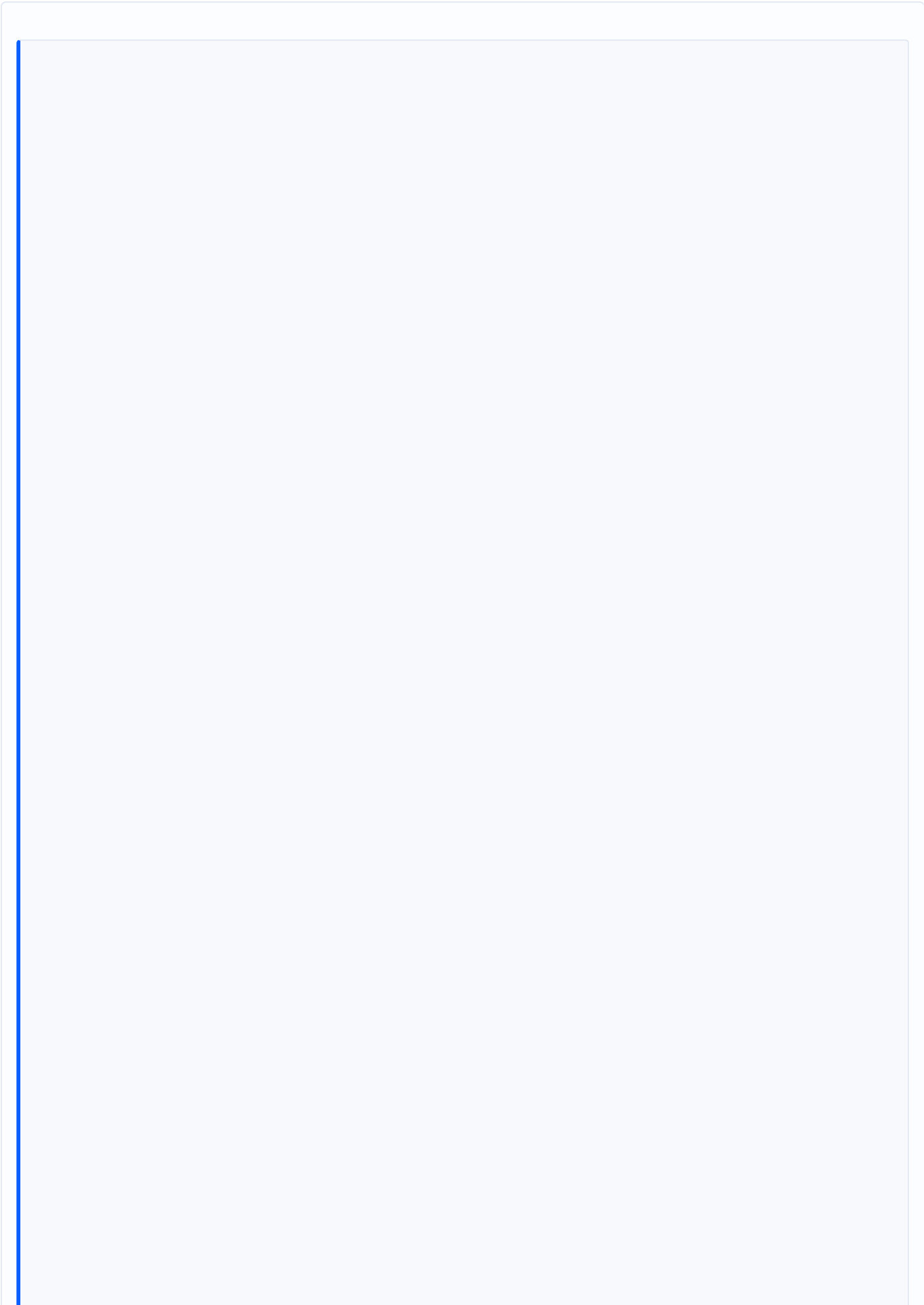
Endpoints: `POST /olt-nativo/ftth/desaprovisionar-contrato/:contratoId`, `POST /olt-nativo/:oltId/ftth/desaprovisionar`, `DELETE /contratos/:id`.

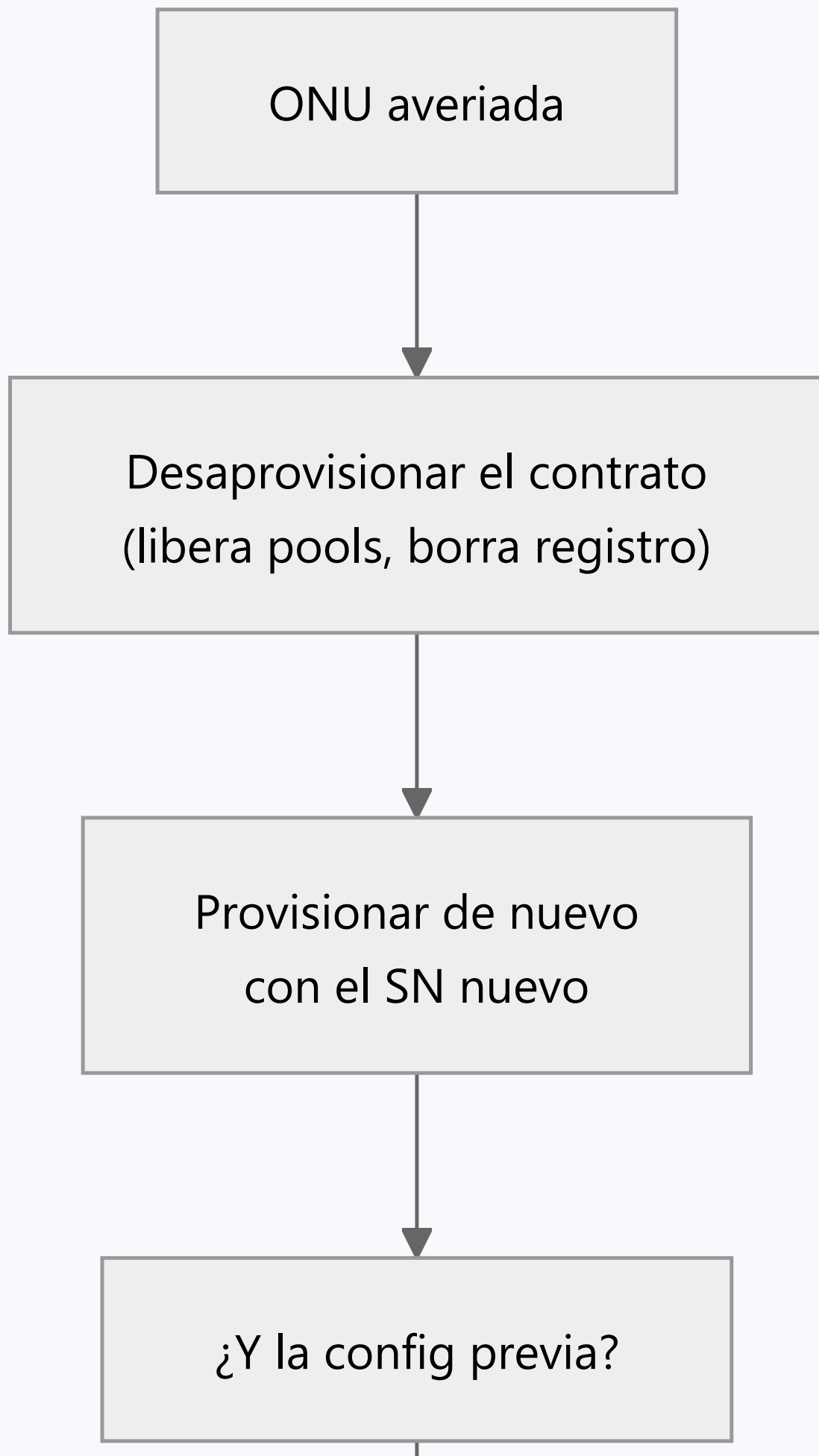
Invariante: nunca se borra `ftth_onu_registro` con la OLT sucia. Si la compensación no se confirma, el registro queda en `fallido_rollback` y lo hereda el watcher.

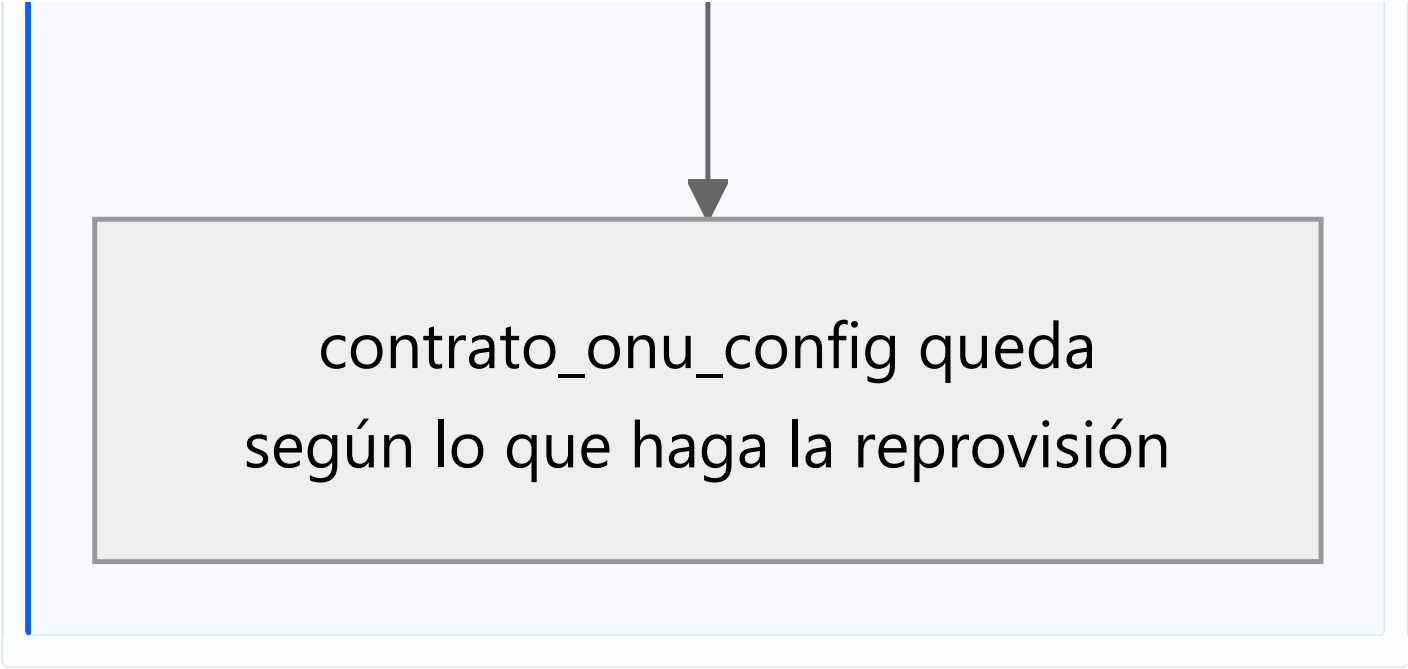
7.8 Cambio de ONU / cambio de equipo — FLUJO NO IMPLEMENTADO

Búsqueda exhaustiva en el backend: **no existe** ningún endpoint, servicio, transición de estado ni saga para sustituir la ONU de un contrato manteniendo el contrato.

Cómo se hace hoy (deducido de las piezas disponibles):







Riesgos concretos de este vacío:

#	Riesgo	Detalle
1	Corte de servicio innecesario	Baja + alta implica pasar por [*] , con toda la reserva de pools liberada y vuelta a pedir
2	Pérdida de la configuración del abonado	SSID y clave WiFi del cliente viven en contrato_onu_config ; una reprovisión los reescribe con el preset
3	Ventana de huérfano	Entre el ont delete y el ont add el contrato existe sin registro FTTH — el mismo estado que causó el incidente del 21/07
4	Sin trazabilidad de sustitución	Ningún registro dice "esta ONU sustituyó a aquella"; el histórico de equipos del abonado se pierde
5	uq_contratos_empresa_onu	El índice único fuerza a soltar la ONU vieja antes de asociar la nueva: no hay solape posible

Es la brecha funcional más significativa detectada, porque el reemplazo de ONU es una operación **rutinaria** en un ISP (avería, upgrade de modelo, cambio de tecnología), y el sistema no la modela: la improvisa componiendo dos operaciones destructivas.

7.9 Comparativa: qué flujos pasan por el plano de intención

Flujo	¿Pasa por outbox?	¿Máquina de estados?	¿Saga con compensación?	¿VIO?
Alta FTTH	Sí (parte MikroTik)	Sí	Sí	Sí
Suspensión por deuda	Sí	Sí	No	Sí
Reactivación por pago	Sí	Sí	No	Sí
Baja / desaprovisión	Parcial	Sí	Sí	Sí
Prórroga	Sí	No	No	Sí

Flujo	¿Pasa por outbox?	¿Máquina de estados?	¿Saga con compensación?	¿VIO?
Cambio de velocidad	No	No	No	Parcial (/discrepancias)
Alta WISP (solo MikroTik)	Parcial	No	No	No
Cambio de ONU	—	—	—	No existe el flujo
Operaciones del operador en /red/routers	No — síncronas	No	No	No

Lectura del cuadro: las garantías fuertes del sistema (outbox + máquina de estados + saga + VIO) están concentradas en **FTTH**. El plano WISP/MikroTik —que es donde vive buena parte del parque— opera con garantías notablemente menores, y las operaciones interactivas del operador no tienen ninguna.

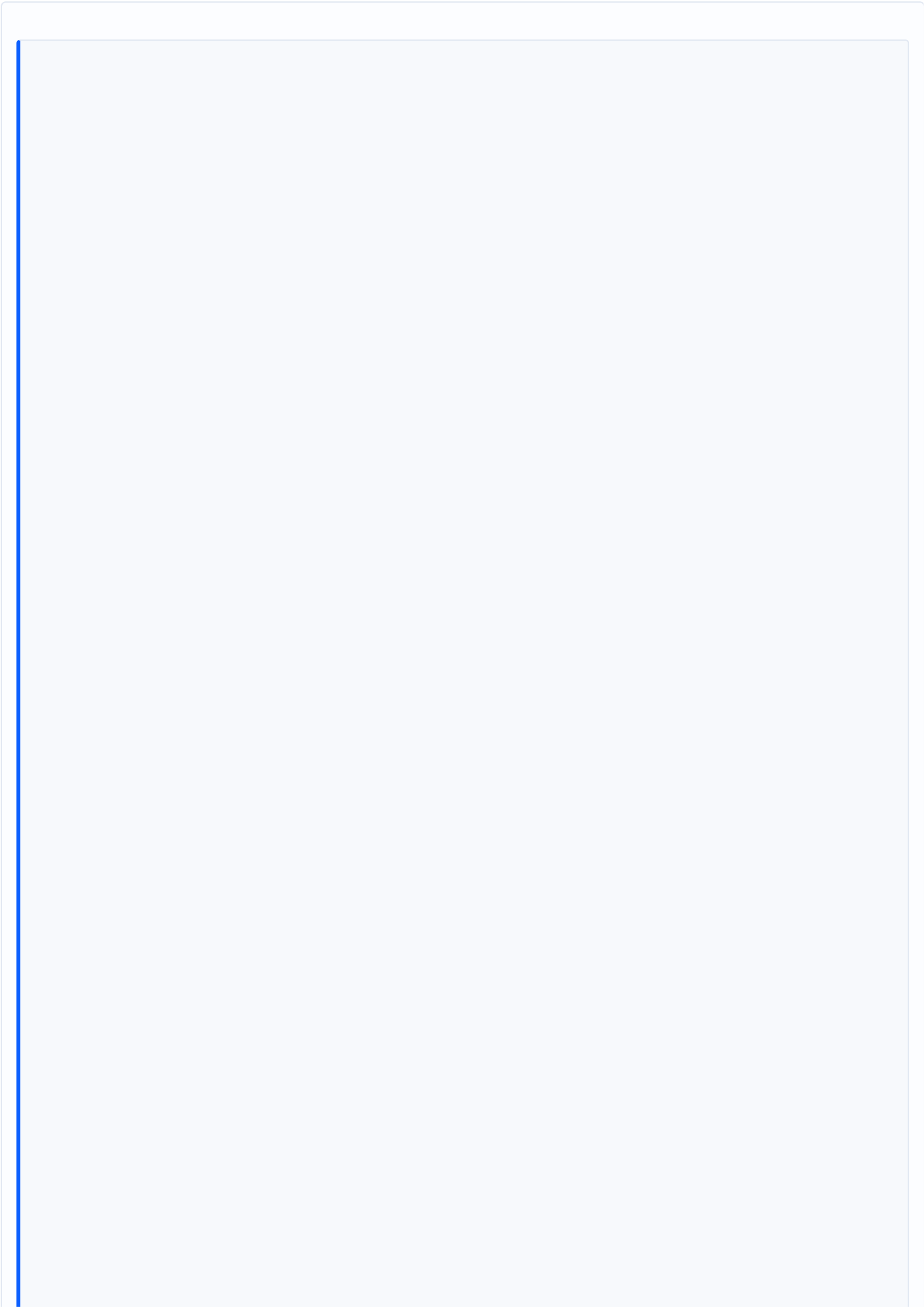
Capítulos 8–11 — Dependencias, Reutilización, Riesgos y Fortalezas

CAPÍTULO 8 — Dependencias

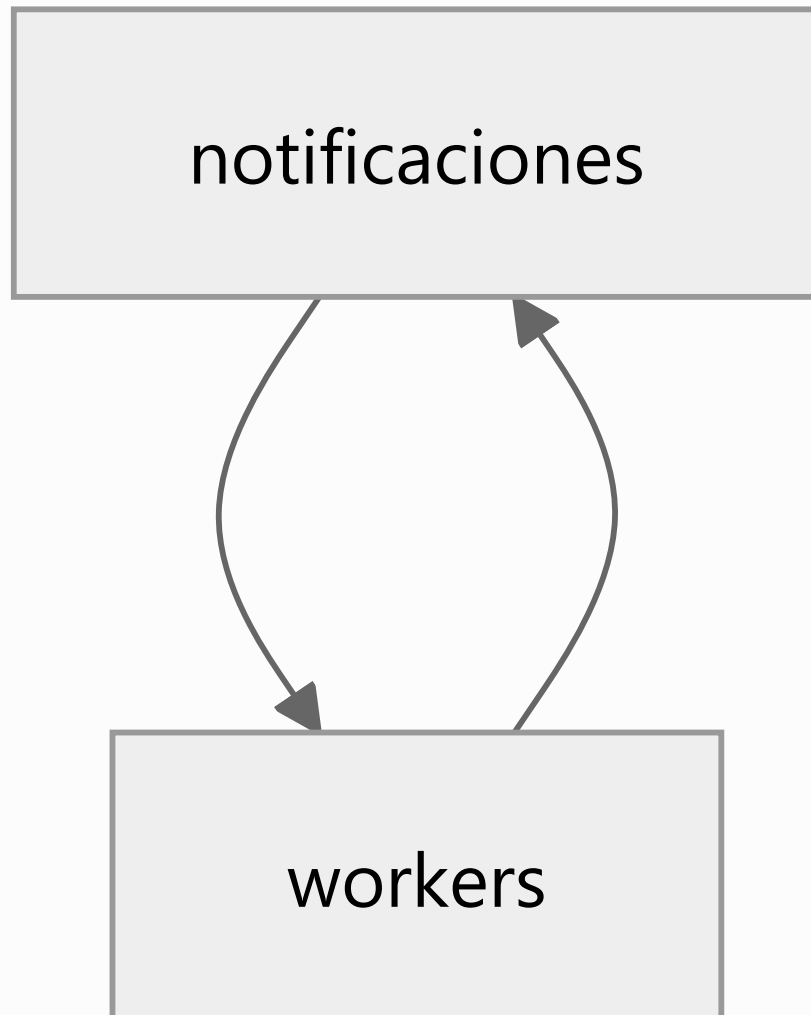
8.1 Mapa por grado

Módulo	Sale a	Entra de	Lectura
auth	1	11	Hub transversal legítimo
config	0	8	Hub transversal legítimo
mikrotik	5	10	Hub de dominio — el nodo más consumido del negocio
contratos	8	3	Nodo con más salidas — agregado raíz
workers	7	5	Hub por constantes, no por servicio
olt-nativo	5	3	Subsistema profundo
outbox-red	2	3	Frontera correcta
portal	5	0	Fachada — hoja del grafo
usuarios	0	2	Hoja

8.2 Los cuatro ciclos y su naturaleza

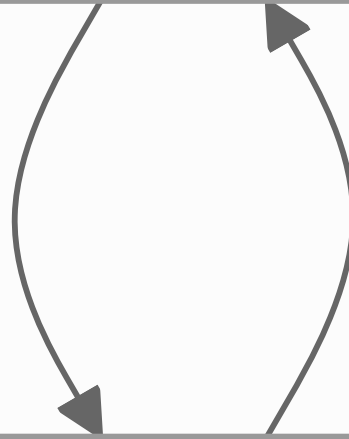


④ Notificación



③ Contrato/Red

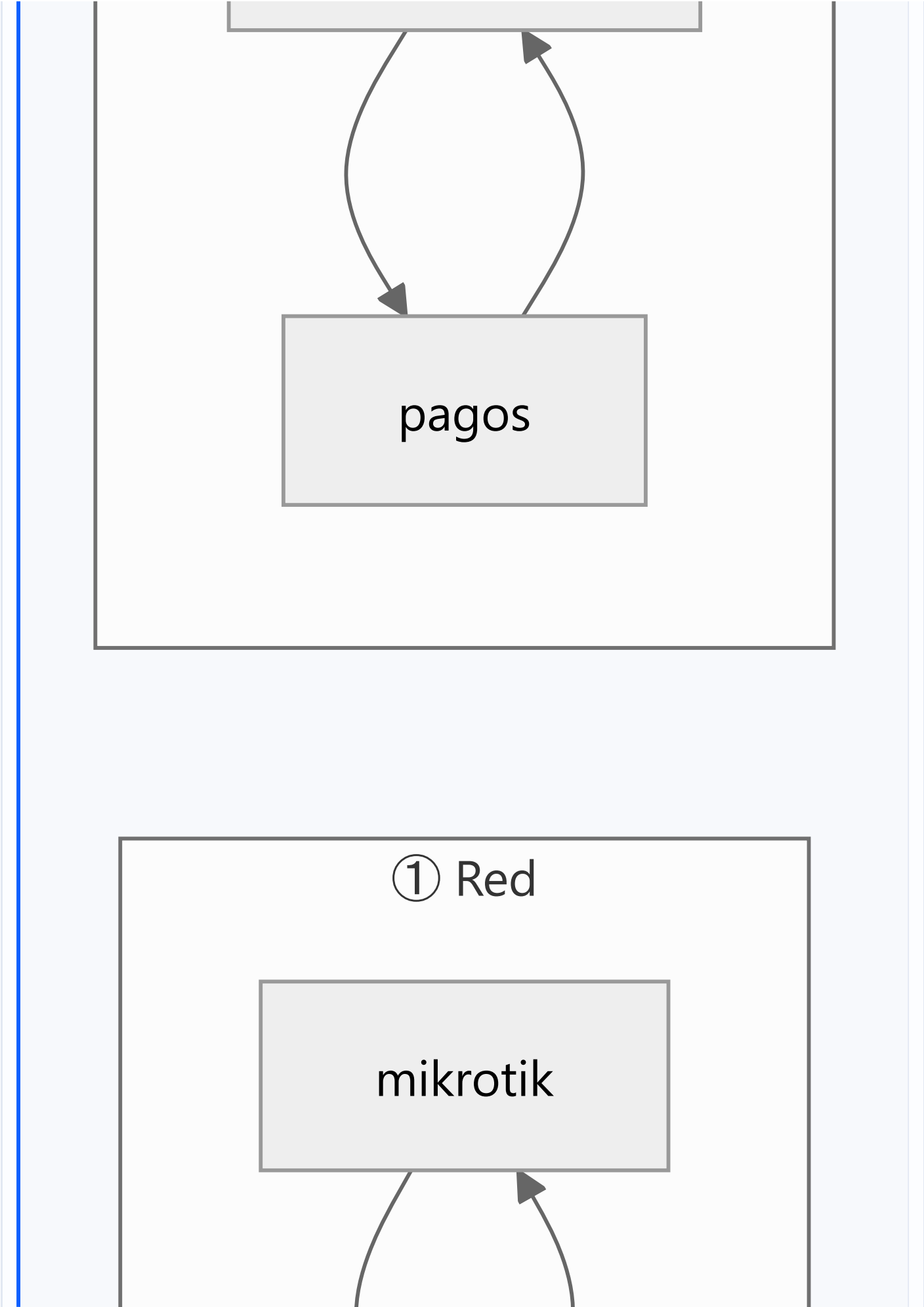
contratos

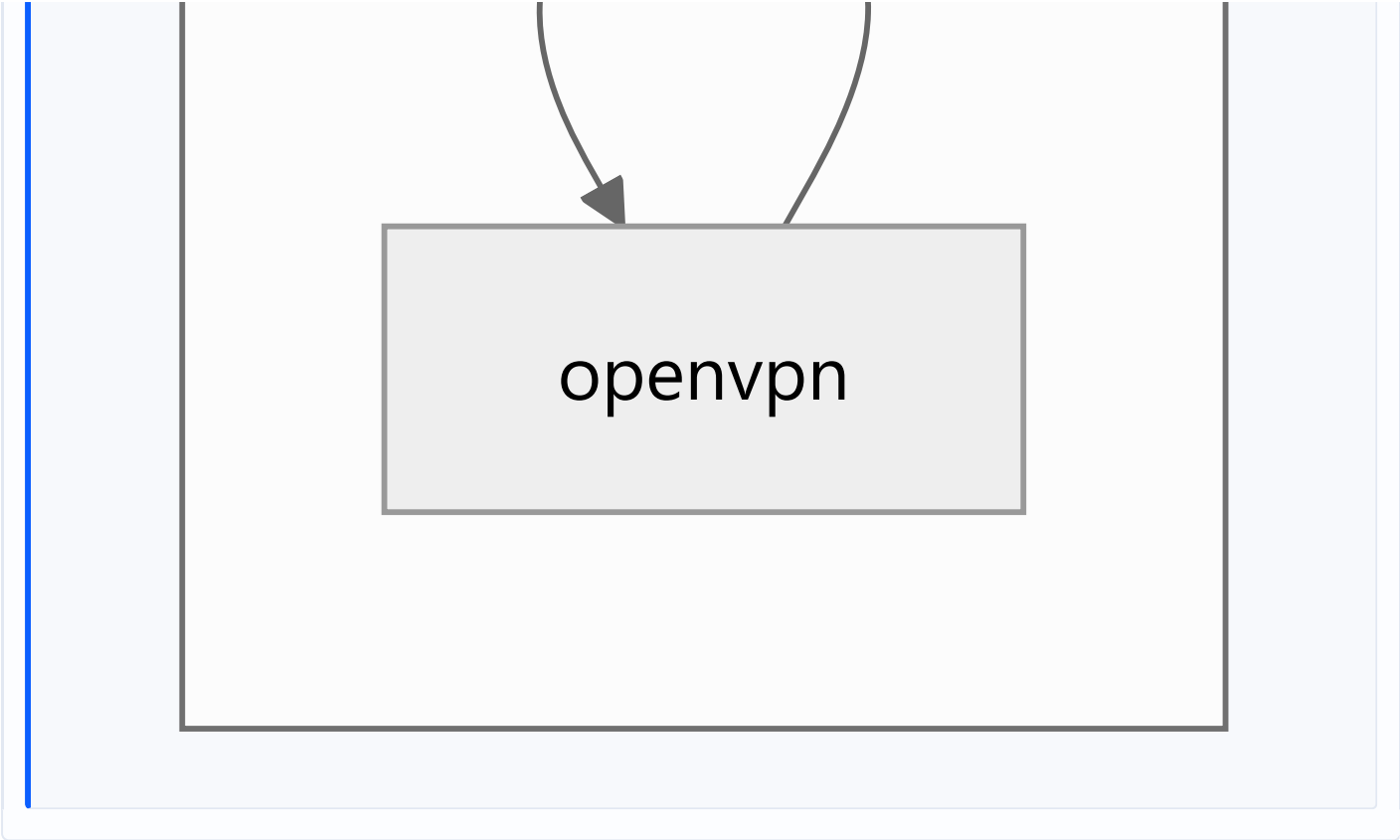


mikrotik

② Cobranza

facturacion





#	Ciclo	Por qué existe	¿Es esencial?	Diagnóstico
1	mikrotik ↔ openvpn	El router necesita túnel para ser alcanzable; el túnel se define contra un router registrado	Sí, conceptualmente	Real. Refleja que el router es simultáneamente objeto gestionado y extremo del canal de gestión
2	facturación ↔ pagos	La factura necesita lo pagado; el pago aplica contra facturas	Sí, conceptualmente	Real. Es el ciclo natural de una cuenta corriente
3	contratos ↔ mikrotik	El contrato provisiona; el router valida CIDR y consulta sesiones	No	Accidental. mikrotik importa contratos para validar check-cidr-en-router y resolver sesiones a contratos — una dependencia de lectura que no requiere el módulo entero
4	notificaciones ↔ workers	notificaciones necesita el nombre de la cola; workers notifica al cortar	No	Artificial. Se debe a que workers.constants.ts mezcla constantes con política de reintentos

Ciclos indirectos adicionales: contratos → outbox-red → mikrotik → contratos y contratos → mikrotik → openvpn → mikrotik .

Conclusión: de 4 ciclos, **2 son esenciales del dominio y 2 son accidentales**. Los esenciales no se eliminan: se gestionan. Los accidentales sí se pueden romper, y romperlos es barato (constantes en un paquete propio; interfaz de lectura para la validación de CIDR).

8.3 Componentes de riesgo por posición en el grafo

Componente	Riesgo	Motivo
<code>mikrotik</code>	Muy alto	10 consumidores + 3 caminos de acceso al hardware. Un cambio de firma o de política de conexión impacta a todo el plano de red
<code>contratos</code>	Alto	8 dependencias salientes; toda alta, baja o cambio pasa por aquí
<code>workers.constants.ts</code>	Alto	Un archivo de constantes acopla 6 módulos
<code>auth / config</code>	Medio	11 y 8 consumidores, pero interfaz estable y de solo lectura
<code>olt-nativo</code>	Medio	Solo 3 consumidores externos, pero es un subsistema de 25.659 LOC
<code>outbox-red</code>	Medio	Pocos consumidores, pero es la frontera por la que pasa todo lo que muta la red

8.4 Qué debería desacoplarse (diagnóstico, sin plan de ejecución)

Ordenado por relación beneficio/coste:

#	Acoplamiento	Coste de romperlo	Beneficio
1	<code>workers.constants.ts</code> como dependencia de 6 módulos	Bajo — mover constantes a <code>common/</code>	Elimina 1 ciclo y 5 dependencias falsas
2	<code>mikrotik</code> → <code>contratos</code>	Bajo-medio — extraer una interfaz de consulta	Elimina 1 ciclo real del núcleo
3	Los 3 caminos hacia MikroTik	Medio — unificar tras un puerto (ya existe el precedente <code>IOltProvider</code>)	Un solo punto de política de conexión, reintento y credenciales
4	<code>olt-nativo</code> como módulo único	Alto — pero las fronteras ya existen en el árbol (<code>capability/</code> , <code>compliance/</code> , <code>domain/</code> , <code>ztp/</code> , <code>providers/</code>)	Hace mantenible el módulo más grande
5	<code>olt-nativo.controller.ts</code> (150 endpoints)	Bajo — dividir por grupo funcional sin tocar rutas	Reduce conflictos de merge y hace legible la superficie

8.5 Dependencias hacia el exterior del repositorio

Acoplamientos que **no viven en el código** y por eso no aparecen en ningún grafo:

Dependencia	Dónde vive	Riesgo
Credenciales connreq de GenieACS	Provision <code>erp-connreq-creds</code> en el ACS + <code>.env</code> del VPS	Deben coincidir; nada lo verifica
CCD y certificados OpenVPN	Filesystem del VPS	El ERP los escribe; no están versionados
<code>mikrotik.conf</code> (rutas <code>10.0.0.0/8</code>)	VPS	Se confundieron con rutas huérfanas
Crontab del sistema	VPS, editable desde <code>/admin/sistema/crontab</code>	Fuera del control de versiones
Chromium de Puppeteer	<code>~/cache/puppeteer</code>	Instalación incompleta detectada en el entorno local

CAPÍTULO 9 — Reutilización

9.1 Servicios genuinamente reutilizados (activos compartidos)

Servicio	Módulo	Consumidores	Calidad
ModuleHealthService	common	Todos los degradables	Excelente — base del patrón de resiliencia
ResultadoOperacion	common/domain	Plano de red	Excelente — vocabulario de dominio con test
RedisLockService	common/redis	Solo cobranza.worker	Buena pieza, infrautilizada
CircuitBreakerRegistry	common	mikrotik, olt-nativo, monitoreo	Muy buena
WatcherHeartbeatService	common	Crons y watchers	Muy buena
QueuePauseService	common	mantenimiento, sistema	Buena
encryption.util	common/utils	Routers, OLTs, proveedores, XUI, Google	Buena
ip.util	common/utils	Contratos, mikrotik	Buena
pagination.util	common/utils	Repositorios	Buena
PoliticaFacturacionService	facturacion	Todo el ciclo de cobro	Excelente — fórmula única
AplicadorFacturaService	facturacion	pagos, adelantos	Excelente — único escritor del saldo
IOltProvider + Router	olt-nativo	3 proveedores	Excelente — puerto con reglas escritas
MikrotikService + pool	mikrotik	10 módulos	Buena, pero con 2 caminos paralelos
GatewayMensajeriaService	notificaciones	4 módulos	Buena
CTE PUNTOS_SERVICIO	planta-externa	Mapa, planta	Ejemplo modélico de consolidación tras incidente
capability.engine	olt-nativo/capability	ZTP y otros dominios	Muy buena — reglas puras, no muta la entrada

9.2 Lógica duplicada

#	Duplicación	Copias	Gravedad	Estado
1	Cálculo de deuda	4 (fn_calcular_deuda_contrato , deuda-por-contrato.service , pagos.service , cobranza.worker)	Crítica — decide cortes de servicio y saldos	Sin unificar

#	Duplicación	Copias	Gravedad	Estado
2	Predicado "contrato activo"	~6	Alta	Sin unificar; hay spec que sugiere fallo previo
3	Resúmenes agregados	3 superficies + 2 vistas	Media	Sin unificar
4	Acceso a MikroTik	3 caminos	Alta	Sin unificar
5	Aplicación de dinero a facturas	Eran 4 copias del mismo <code>UPDATE</code>	—	RESUELTA — la de <code>adelantos</code> había perdido el guard y aplicaba saldo a facturas ANULADAS. Hoy bloqueada por <code>frontera-dinero.spec.ts</code>
6	Ubicación del abonado	Eran 4 consultas (<code>clientes.latitud</code> vs <code>contratos.latitud_instalacion</code>)	—	RESUELTA — CTE <code>PUNTOS_SERVICIO</code>
7	Estado de ONU	2 caminos (SSH en vivo vs BD)	Baja	JUSTIFICADA por escrito — costes incompatibles. Falta test de no divergencia
8	Tipos backend↔frontend	2 definiciones de cada DTO	Media	Sin unificar, pese a existir Swagger
9	Validación de formularios	Backend (<code>class-validator</code>) y frontend (a mano)	Media	Sin unificar

Observación importante: las duplicaciones 5 y 6 **ya fueron resueltas por el equipo**, y en ambos casos la resolución fue "una sola definición, en el punto común". El sistema **sabe cómo** resolver este problema — lo ha hecho dos veces bien. Lo que falta es aplicarlo a las restantes.

9.3 Consultas y trabajo repetido

Repetición	Detalle
Reconciliación solapada	5 procesos (<code>reconciliador</code> 15/30 min, <code>ztp-reconcile</code> 03:30 + 2 min, <code>ftth-wan-watcher</code> 10 min, <code>address-list-reconciliador</code> 04:40, <code>olt-sync</code> 6 h) verifican estados solapados del mismo parque, sin coordinación entre sí
Lecturas de OLT	<code>syncPeriodico</code> , <code>olt-health-poller</code> , <code>adoptarHuerfanos</code> , <code>verificarWan</code> , <code>limpiarIdsHuerfanos</code> y cada <code>GET /:oltId/onus</code> del operador abren sesiones contra la misma OLT de sesiones VTY limitadas
Sin cache de servidor en frontend	Cada pantalla repite sus peticiones; no hay TanStack Query/SWR
Sin invalidación de cache	La consistencia se obtiene por TTL de 5 min, no por invalidación dirigida

9.4 Componentes compartidos del frontend

Categoría	LOC	% del frontend
<code>ui</code> + <code>shared</code> + <code>atoms</code> (reutilizable)	1.016	1,8 %

Categoría	LOC	% del frontend
Componentes de dominio	~56.400	98,2 %

Ver Cap. 3 para el análisis.

9.5 Procesos repetitivos susceptibles de patrón común

Sin proponer implementación, se observan **cuatro familias** de proceso que hoy se escriben desde cero cada vez:

1. **Watcher de reconciliación** — leer estado real, comparar con BD, actuar, registrar. 5 implementaciones distintas.
2. **Pool de recursos** — reservar, liberar, reconciliar, barrer huérfanos. 4 implementaciones (`service-port` , `mgmt-ip` , `onu-id` , `mgmt-port`) + puertos NAP + IPs.
3. **Operación mutante contra hardware** — lock, write-ahead, ejecutar, VIO, clasificar resultado, compensar. Bien resuelta en FTTH; ausente en MikroTik.
4. **Wizard con anulación** — abrir, heartbeat, pasos con compensación, confirmar/cerrar, barrido por TTL. Implementada dos veces (FTTH y VPN) con mecánicas distintas.

CAPÍTULO 10 — Riesgos Arquitectónicos

Escala: **Probabilidad** (Baja/Media/Alta) × **Impacto** (Bajo/Medio/Alto/Crítico) → **Criticidad** (Baja/Media/Alta/Crítica). Todas son **valoraciones** sustentadas en evidencia medida o en incidentes registrados.

R-01 · Reescritura masiva de configuración de ONUs en una migración

Campo	Valor
Causa	<code>ZtpReconcileCron</code> (03:30) reescribe SSID, clave WiFi y credenciales web de toda ONU con <code>provisioning_enabled=true</code> y <code>last_applied_revision IS NULL</code> . La protección actual es un efecto lateral de que <code>adoptarOnusHuerfanas</code> no crea <code>contrato_onu_config</code>
Impacto	Crítico — todo el parque migrado sin internet en sus dispositivos a la mañana siguiente, sin que nadie lo pidiera
Probabilidad	Alta — hay dos migraciones planificadas (SmartOLT 205 ONUs, MikroWISP)
Criticidad	CRÍTICA
Evidencia	<code>CLAUDE.md</code> \$Migraciones de ONUs; <code>ztp-reconcile.cron.ts</code>

R-02 · Punto único de falla: `worker-auxiliary`

Campo	Valor
Causa	Los 29 crons, las 6 colas, el outbox y todos los watchers corren en un solo proceso PM2
Impacto	Crítico — el ERP sigue respondiendo con normalidad mientras nadie se corta, nadie se reactiva, ningún comando de red se aplica y ningún watcher repara. Sin señal visible en la interfaz
Probabilidad	Media — límite de 800 MB, <code>max_restarts: 10</code>
Criticidad	CRÍTICA
Evidencia	<code>ecosystem.config.js</code> ; ausencia de alerta por watcher caído en la UI

R-03 · Aislamiento multi-tenant por convención

Campo	Valor
Causa	El filtrado por <code>empresa_id</code> depende de que cada una de las 445 consultas crudas lo incluya. No hay RLS ni guard central
Impacto	Crítico — fuga de datos entre empresas; una omisión no produce error , produce datos ajenos
Probabilidad	Media (Alta si crece el nº de tenants o de desarrolladores)
Criticidad	CRÍTICA
Evidencia	445 <code>.query()</code> ; índices UNIQUE por empresa existen, RLS no

R-04 · Cálculo de deuda en 4 lugares

Campo	Valor
Causa	<code>fn_calcular_deuda_contrato</code> , <code>deuda-por-contrato.service</code> , <code>pagos.service</code> , <code>cobranza.worker</code> calculan lo mismo con criterios potencialmente distintos (cargos pendientes, notas de crédito, adelantos, promesas)
Impacto	Crítico — el ERP puede cortar a quien no debe, o no cortar a quien sí; y responder distinto según por dónde se le pregunte
Probabilidad	Alta — 4 copias divergen en la primera modificación de una sola
Criticidad	CRÍTICA
Evidencia	Cap. 9.2; precedente idéntico ya ocurrido con la aplicación de dinero

R-05 · 39 tablas sin entidad TypeORM

Campo	Valor
Causa	Solo 81 de ~120 tablas tienen entidad. Las sin tipar incluyen <code>comandos_red_pendientes</code> , <code>ftth_operacion_lock</code> , <code>operacion_wizard(_paso)</code> , <code>pago_extorno</code> , <code>cierre_caja</code> , <code>cuentas_bancarias</code>
Impacto	Alto — un cambio de esquema en el outbox, el lock FTTH o la saga no rompe la compilación : rompe producción
Probabilidad	Media-Alta
Criticidad	ALTA

R-06 · Ausencia total de observabilidad

Campo	Valor
Causa	Sin APM, sin trazas, sin métricas, <code>logging: false</code> en TypeORM, sin <code>pg_stat_statements</code> declarado
Impacto	Alto — no se puede saber qué endpoint se usa, qué consulta duele ni si una corrección funcionó. Los incidentes se descubren por el cliente
Probabilidad	Alta — ya está ocurriendo
Criticidad	ALTA
Evidencia	Etapla I declaró 5 preguntas que no puede responder por esta causa

R-07 · Concentración de responsabilidades en `olt-nativo`

Campo	Valor
Causa	25.659 LOC, 41 servicios, 24 entidades, 11 crons, 1 controlador de 150 endpoints, 8 subdominios
Impacto	Alto — cada cambio requiere entender el módulo entero; los merges chocan
Probabilidad	Alta — es el módulo que más crece

Campo	Valor
Criticidad	ALTA
Atenuante	Las fronteras internas ya existen en el árbol

R-08 · Garantías desiguales entre FTTH y WISP/MikroTik

Campo	Valor
Causa	Outbox + máquina de estados + saga + VIO están en FTTH. El plano MikroTik tiene outbox parcial, sin máquina de estados, sin saga y con VIO solo como detección posterior (/discrepancias)
Impacto	Alto — los incidentes que FTTH ya no puede tener, MikroTik sí
Probabilidad	Media
Criticidad	ALTA
Evidencia	Cap. 7.9

R-09 · Flujo de cambio de ONU inexistente

Campo	Valor
Causa	No hay endpoint ni saga de sustitución. Se improvisa como baja + alta
Impacto	Alto — corte innecesario, pérdida de la configuración WiFi del cliente, ventana de huérfano, sin trazabilidad del equipo sustituido
Probabilidad	Alta — es una operación rutinaria en un ISP
Criticidad	ALTA

R-10 · `reconciliar()` sin cap ni lock

Campo	Valor
Causa	<code>reconciliador.service.ts</code> itera cada 15 min sin límite de trabajo ni lock; el carril automático multiplicó su carga potencial
Impacto	Alto — puede solaparse consigo mismo y saturar las sesiones VTY del MA5800
Probabilidad	Media-Alta
Criticidad	ALTA
Evidencia	<code>PENDIENTES.md</code> ; precedente de 1.788 reintentos contra el MA5800

R-11 · Serialización de todo el plano OLT en un worker unicorn

Campo	Valor
Causa	1 worker por el límite bajo de sesiones VTY del MA5800 (decisión correcta)
Impacto	Alto — es el cuello de botella estructural; toda operación OLT hace cola
Probabilidad	Alta al crecer el parque o el nº de OLTs
Criticidad	ALTA
Matiz	El riesgo no es la decisión , es que no hay encolado explícito ni medición de la cola

R-12 · Series temporales sin particionado ni retención

Campo	Valor
Causa	<code>metricas_monitoreo</code> escribe cada minuto × dispositivo; <code>consumo_datos</code> cada 15 min × contrato. Existe <code>fn_cleanup_old_data</code> sin política visible
Impacto	Medio-Alto — degradación progresiva de consultas y backups
Probabilidad	Alta — es acumulativo por definición
Criticidad	ALTA

R-13 · `TimeoutInterceptor(30 s)` global frente a hardware lento

Campo	Valor
Causa	Timeout global de 30 s; operaciones legítimas de 90–150 s
Impacto	Medio-Alto — toda operación síncrona nueva contra hardware nace rota y nada lo impide en compilación
Probabilidad	Media
Criticidad	MEDIA-ALTA

R-14 · Licencia como punto único de falla total

Campo	Valor
Causa	<code>LicenciaGuard</code> es el primer <code>APP_GUARD</code> ; su fallo bloquea el ERP incluido <code>auth</code>
Impacto	Crítico cuando ocurre
Probabilidad	Baja — hay validación diaria + recarga cada 6 h
Criticidad	MEDIA
Matiz	Es deliberado . El riesgo es que no hay modo degradado ni periodo de gracia documentado

R-15 · Cobertura de tests

Campo	Valor
Causa	~30 specs para ~96.000 LOC de backend; 2 tests para 57.435 LOC de frontend; la suite de facturación no compila; <code>sql:check</code> no está en CI
Impacto	Medio-Alto — las regresiones se descubren en producción
Probabilidad	Alta
Criticidad	MEDIA-ALTA
Matiz importante	Los tests que existen son de altísima calidad : cubren los invariantes que ya fallaron y nombran el incidente. El problema es cobertura, no criterio

R-16 · Concentración de crons en la madrugada

Campo	Valor
Causa	5 barridos pesados entre 03:00 y 04:40, sin coordinación ni presupuesto de tiempo
Impacto	Medio
Probabilidad	Media
Criticidad	MEDIA

R-17 · Tres caminos hacia MikroTik

Campo	Valor
Causa	<code>node-routeros</code> (NestJS), <code>ssh2</code> (NestJS), <code>mikrotik_pool.py</code> (Python)
Impacto	Medio — política de conexión, reintentos y credenciales triplicadas
Probabilidad	Media
Criticidad	MEDIA

R-18 · Deriva del frontend

Campo	Valor
Causa	3 convenciones de organización; 1,8 % de código reutilizable; componentes de 3.776 LOC; tipos duplicados; sin librería de formularios
Impacto	Medio — velocidad de desarrollo y consistencia de UX
Probabilidad	Alta
Criticidad	MEDIA

R-19 · Presupuesto de memoria sobrecomprometido

Campo	Valor
Causa	Límites PM2 suman 3,17 GB sobre ~1,9 GB de RAM
Impacto	Medio-Alto — ya hubo un episodio con 87 MB libres
Probabilidad	Media
Criticidad	MEDIA

R-20 · Configuración fuera del repositorio

Campo	Valor
Causa	Credenciales de GenieACS duplicadas en el ACS, CCD/certs en filesystem, crontab del VPS editable desde la UI
Impacto	Medio — una instalación nueva no es totalmente reproducible desde el repo
Probabilidad	Media
Criticidad	MEDIA

10.21 Matriz resumen

Criticidad	Riesgos
CRÍTICA	R-01 migración de ONUs · R-02 SPOF del worker · R-03 multi-tenant · R-04 deuda ×4
ALTA	R-05 tablas sin entidad · R-06 observabilidad · R-07 olt-nativo · R-08 FTTH vs WISP · R-09 cambio de ONU · R-10 reconciliador · R-11 serialización OLT · R-12 series temporales
MEDIA-ALTA	R-13 timeout global · R-15 tests
MEDIA	R-14 licencia · R-16 crons madrugada · R-17 caminos MikroTik · R-18 frontend · R-19 memoria · R-20 config externa

CAPÍTULO 11 — Fortalezas

Este capítulo existe porque **no todo debe modificarse**. Lo que sigue debe preservarse explícitamente en cualquier consolidación futura. Varias de estas fortalezas son superiores a lo que se encuentra en ERPs comerciales del sector.

F-01 · Verified Infrastructure Operations (VIO) — preservar sin excepción

`accepted ≠ materialized`. Toda mutación contra hardware se verifica con un comando de lectura **independiente**. Nació de un incidente real y bien diagnosticado: una EG8145V5 aceptó el comando OMCI sin error, la OLT lo mostraba configurado, y el firmware nunca activó el IP-host — confirmado con sniffer en cold-boot físico. El ERP reportó "aplicado" durante días con la gestión muerta.

Por qué es excepcional: la mayoría de sistemas del sector tratan "el CLI no dio error" como éxito. Este distingue explícitamente *aceptado* de *materializado* y lo dice en el mensaje al operador.

F-02 · Vocabulario de dominio en vez de transporte — preservar y extender

`ResultadoOperacion` con 6 clases: `aplicado` · `ya_en_destino` · `no_aplica` · `rechazado_definitivo` · `reintentable` · `indeterminado`. El transporte traduce en el borde.

Tres decisiones dentro de esta pieza que merecen conservarse literalmente:

1. **`indeterminado` es obligatorio ante un timeout de hardware.** Un timeout no significa "no pasó nada".
2. **La lista de rechazos definitivos es explícita y corta: solo 400 y 404.** Un criterio `status < 500` es incorrecto.
3. **Ante la duda: reintentable**, porque reintentar es recuperable y descartar no.

F-03 · Outbox transaccional con reclamo atómico — preservar

El negocio y la intención de red se escriben en la **misma transacción**. El reclamo es `UPDATE → EN_PROCESO + dueño + TTL` en **una sola sentencia**, no `SELECT FOR UPDATE SKIP LOCKED`, porque este último protegía la selección pero no la ejecución.

F-04 · Máquina de estados declarativa — preservar y replicar

Las transiciones legales viven en **un solo archivo** y la **idempotencia se deriva del estado destino** (`ya_en_destino` = ÉXITO): un método nuevo no puede olvidarse de ser idempotente porque no es él quien lo implementa. Gracias a la declaración única se detectó que faltaba `suspendido` como origen de `desaprovisionar` — el caso más frecuente del negocio.

Ya replicada voluntariamente en `planta-externa/domain/`.

F-05 · Saga con bitácora write-ahead — preservar

Seis reglas, todas correctas y poco habituales juntas:

1. La compensación se registra **antes** de ejecutar.
2. Cada paso guarda **cómo deshacerlo y cómo verificar si llegó a aplicarse**.
3. Las compensaciones son **idempotentes** ("does not exist" al deshacer = hecho, no error).
4. **VIO también al deshacer**.
5. El heartbeat **suprime** el barrido, nunca lo autoriza, y tiene **techo absoluto**.
6. **Anular es asíncrono** — se espera a que el paso en vuelo termine; nunca se corta el hardware a mitad.

F-06 · La frontera transaccional es el estado terminal verificado, no el clic

Razonamiento explícito y correcto: el clic es inalcanzable justo en los peores casos (crash, corte de sesión, corte de luz) y usarlo convertiría la regla en fábrica de cortes de servicio.

F-07 · Patrón degradable + Core Indestructible

Distinción consciente, módulo por módulo, entre "puede arrancar degradado" y "debe crashear el backend para proteger al proceso anterior en PM2".

F-08 · Invariante de atomicidad hardware↔ERP con dos watchers

Nunca un `ont` en la OLT sin `ftth_onu_registro`, ni al revés. Watcher DELETE (`reintentarRollbacksFallidos`, estado `fallido_rollback`) y watcher CREATE (`adoptarOnusHuerfanas`, que reconstruye desde inventario + pool + lectura viva con VIO). La descripción del `ont` lleva `DATAFAST_CNT-xxxx` para poder atribuirlo.

F-09 · Ports & Adapters con reglas escritas en el contrato

`IOltProvider` no solo declara métodos: prohíbe propagar excepciones, obliga a medir latencia y **prohíbe que un adaptador toque la BD**. Es arquitectura hexagonal correcta en el dominio más crítico.

F-10 · Un solo escritor del dinero

`PagosService.registrar` es el único registrador; `AplicadorFacturaService` el único que aplica. Protegido por `frontera-dinero.spec.ts` tras descubrir 4 copias del mismo `UPDATE`, una de ellas sin guard de estado aplicando saldo a favor contra facturas **anuladas**.

F-11 · Puerta de estabilidad documentada para las pasarelas de pago

El contrato de cobro se fijó **antes** que las implementaciones, deliberadamente: si se hubiera dejado para después, la primera integración lo habría definido de facto. Y las implementaciones están bloqueadas hasta cumplir criterios que **no dependen de escribir código** (30 días de contabilidad limpia, un extorno real revisado a mano, un cierre mensual cuadrado).

Es la decisión de gestión técnica más madura del repositorio: distingue entre "sé cómo hacerlo" y "es prudente hacerlo ahora".

F-12 · Tests que nombran el incidente

Regla explícita: un test llamado "no debería fallar" se borra en la primera limpieza; uno que dice "409 de lock es reintentable, no un veredicto (incidente 28/07)" sobrevive.

F-13 · "VIO hacia adentro" — el software también afirma sin verificar

Regla derivada de descubrir que un comentario garantizaba una propiedad de concurrencia que era **falsa**: todo comentario que garantice concurrencia, atomicidad o exclusión mutua lleva un test que lo ejercite, **o se borra**. Y: *un log describe lo que ocurrió, nunca lo que el código pretendía hacer*.

F-14 · Portabilidad multi-VPS implementada

Sin IPs ni dominios en el repositorio; lazy getters; `.env.example` como contrato; `ecosystem.config.js` sin secretos; vhosts por plantilla con caída elegante cuando falta un dominio; retrocompatibilidad al renombrar variables (`ERP_DOMAIN` → `APP_DOMAIN`).

F-15 · Aislamiento de procesos por causa real

Chromium en su propio proceso PM2 (tras dejar el VPS con 87 MB libres), migraciones en un solo proceso (tras la colisión del 21/07), frontend con entorno mínimo (tras arrastrar todos los secretos del backend). Cada aislamiento responde a un incidente concreto y está documentado en el propio archivo de configuración.

F-16 · Separación ERP ↔ Portal del abonado

Secreto JWT distinto, cookies propias, guard propio, servicio de tenant propio, vhost con API acotada por regex, y **tests de aislamiento** (`portal-auth.aislamiento.spec.ts`).

F-17 · Consolidación demostrada de duplicados

El CTE `PUNTOS_SERVICIO` y el escritor único del saldo prueban que el equipo **sabe resolver** la duplicación cuando la detecta: una sola definición en el punto común, no un parche por sitio.

F-18 · Cultura de causa raíz

Regla escrita y aplicada: reproducir y observar (no deducir), explicar cómo llegó el sistema a ese estado, preguntar dónde más ocurre lo mismo, corregir en el punto común, y dejar constancia **de la causa**, no del arreglo. Los tres fallos del mapa (2026-08-05) documentan explícitamente el parche superficial que se rechazó en cada caso.

11.19 Síntesis

El ERP Datafast tiene una arquitectura de resiliencia superior a su arquitectura de organización. El plano de red —lo más difícil— está resuelto con patrones que muchos sistemas comerciales no tienen. Lo que falta no es criterio arquitectónico: es **cobertura, uniformidad y obligatoriedad** de un criterio que ya existe y ya se demostró correcto.

Esa es exactamente la definición de un sistema listo para **consolidarse**, no para reescribirse.

Capítulo 12 — Inventario Arquitectónico

Inventario de activos arquitectónicos del ERP Datafast en el commit `f8d52b00` . Todos los conteos son extracción mecánica sobre el árbol de código.

12.1 Resumen cuantitativo

Activo	Cantidad
Módulos NestJS	44 (+1 vacío: <code>migracion</code>)
Controladores	46
Endpoints HTTP	~560
Servicios (<code>*.service.ts</code>)	~160
Repositorios	6 (1.614 LOC)
Puertos (interfaces de adaptador)	4
Adaptadores	5 en TS + 3 drivers Python
Máquinas de estados declarativas	2
Entidades TypeORM	81
Tablas en base de datos	~120 (39 sin entidad)
Migraciones	215
Índices	376
Triggers / Funciones / Vistas	27 / 13 / 4
Cron jobs	29
Colas Bull	6
Processors	6
Listeners de eventos	25
Eventos de dominio catalogados	23
WebSocket gateways	3
Guards globales	4
Interceptors globales	5
Integraciones externas activas	15
Páginas frontend	92
LOC backend (<code>modules/</code>)	~96.000
LOC frontend (<code>components/</code> + <code>app/</code>)	57.435

Activo	Cantidad
LOC Python (services/)	5.520
Tests backend	~30
Tests frontend	2

12.2 Módulos por dominio y criticidad

Dominio	Módulos	LOC	Criticidad
RED / OSS	olt-nativo, mikrotik, openvpn, outbox-red, monitoreo, planta-externa, smartolt, tr069, reconciliador, sites	47.199	Máxima
FINANCIERO	facturacion, pagos, promesas-pago, finanzas-opex, proyectos-inversion	13.425	Máxima
COMERCIAL	clientes, contratos, planes, zonas	7.005	Máxima
CLIENTE FINAL	portal, tickets, xui	6.687	Alta
COMUNICACIÓN	notificaciones, mensajería, crm-nativo, plantillas, webhooks	5.714	Alta
PLATAFORMA	auth, usuarios, licencia, auditoria, sistema, backup, health, install, workers, sagas, mantenimiento, schema-guard, config, dashboard, reportes, aprovisionamiento, google-integration	13.746	Máxima/Alta

12.3 Inventario de servicios por función arquitectónica

12.3.1 Servicios de dominio (deciden reglas de negocio)

Servicio	Módulo	Regla que encapsula
politica-facturacion.service.ts	facturacion	Fórmula única del ciclo de cobro
deuda-por-contrato.service.ts	facturacion	Cálculo de deuda (uno de 4 caminos)
aplicador-factura.service.ts	facturacion	Único escritor del saldo
ftth-maquina-estados.ts	olt-nativo/domain	Transiciones legales FTTH
planta-externa-maquina-estados.ts	planta-externa/domain	Transiciones de planta
presupuesto-optico.ts	planta-externa/domain	Cálculo de presupuesto óptico
resultado-operacion.ts	common/domain	Clasificación de resultados
olt-compliance-rules.ts	olt-nativo/compliance	Reglas de cumplimiento de OLT
capability.engine.ts	olt-nativo/capability	Filtrado de config por capacidad de dispositivo

12.3.2 Servicios de orquestación

provision-ftth.service.ts · operacion-wizard.service.ts · compensador-wizard.service.ts · olt-operation-router.service.ts · velocidad-orquestador.service.ts · outbox-red.service.ts · gateway-mensajeria.service.ts · ztp.service.ts · olt-baseline-plan.service.ts · arqueo-caja.service.ts

12.3.3 Servicios de infraestructura / acceso

connection-pool.service.ts (MikroTik) · olt-conn.service.ts · olt-automation.client.ts · genieacs.driver.ts · mercadopago.service.ts · reniec.service.ts · smtp.strategy.ts · whatsapp.service.ts · pdf.service.ts · backup.service.ts

12.3.4 Servicios de coordinación y resiliencia

Servicio	Función
ftth-operacion-lock.service.ts	Lock por contrato/operación, TTL corto, 409
olt-atomic-lock.service.ts	Lock atómico de OLT
olt-idempotency.service.ts	Idempotencia de operaciones
circuit-breaker.service.ts + circuit-breaker.registry.ts	Breakers por equipo/proveedor
redis-lock.service.ts	Lock distribuido
module-health.service.ts	Estado degradado por módulo
watcher-heartbeat.service.ts	Latido de procesos de fondo
queue-pause.service.ts	Pausa coordinada de colas
operacion-wizard-paso.service.ts	Bitácora write-ahead de compensación

12.3.5 Servicios de pools de recursos

olt-service-port-pool.service.ts · olt-mgmt-ip-pool.service.ts · olt-onu-id-pool.service.ts · (mgmt-port-pool vía controlador) · pools IPv4 en contratos/repositories · puertos NAP en planta-externa

12.4 Inventario de puertos y adaptadores

Puerto	Ubicación	Adaptadores	Estado
IoltProvider	olt-nativo/interfaces/olt-provider.interface.ts	nativo-ssh · smartolt · adminolt	Activo, 3 implementaciones
Adaptador de cobro	pagos/adaptadores/adaptador-cobro.interface.ts	Ninguno — deliberado	Contrato fijado, implementación bloqueada por puerta de estabilidad
Provisionamiento	aprovisionamiento/interfaces/provisionamiento-provider.interface.ts	mock-provisionamiento	Esqueleto
Driver ACS	olt-nativo/ztp/	genieacs.driver.ts + registry + resolver	Activo, driver único

Puerto	Ubicación	Adaptadores	Estado
Drivers OLT (Python)	olt-automation-service/app/drivers/	huawei · vsol (sobre base)	Activo
Estrategias de mensajería	notificaciones/services/*.strategy.ts	datafast-native · datafast-mensajeria-masiva · smtp	Activo
Estrategias de queue	mikrotik/services/velocidad/	simple_queue · queue_tree · pcq · sin_limite	Activo

12.5 Inventario de entidades por dominio

Dominio	Entidades
Comercial	cliente · cliente-historial-estado · contrato · red (segmentos_ipv4) · plan · zona · site
Financiero	factura · cargo-pendiente · comprobante-config · configuracion-facturacion · pago · pago-aplicacion · canal-pago · promesa-pago · egreso-ingreso · proyecto-inversion
Red — OLT/FTTH (24)	olt-dispositivo · olt-board · olt-vlan · olt-line-profile · olt-service-profile · olt-traffic-table · olt-baseline · olt-alerta · olt-health-snapshot · olt-sync-job · olt-operacion-log · olt-proveedor-config · olt-onu-inventario · olt-onu-preset · olt-service-port-pool · olt-mgmt-ip-pool · olt-onu-id-pool · ftth-onu-registro · ftth-rollback-log · contrato-onu-config · cpe-provisioning-attempt · cpe-web-credential · metricas-onu-optical · historial-firmware
Red — MikroTik/VPN	router · openvpn-config · vpn-cliente · vpn-alerta
Red — Planta externa (9)	pe-mufa · pe-fusion · pe-splitter · pe-splitter-salida · pe-nap · pe-nap-puerto · pe-fibra-segmento · pe-fibra-hilo · pe-acometida
Monitoreo	dispositivo-monitoreo · metricas-monitoreo · alerta-sistema · umbral-alerta
Comunicación	notificacion-log · plantilla-mensaje · plantilla-abonado · crm-chat · crm-mensaje
Cliente final	portal-config · portal-banner · ticket · xui-servidor · xui-line
Plataforma	usuario · rol · permiso · auditoria-log · entity-version · empresa · licencia-estado · backup · saga-log · google-account · google-sync-log · tr069-device
SmartOLT (legado)	onu (→ tabla olts)

Tablas críticas SIN entidad (39 en total; las de mayor riesgo)

comandos_red_pendientes · ftth_operacion_lock · operacion_wizard · operacion_wizard_paso · pago_extorno · cierre_caja · cuentas_bancarias · bancos_isp · formas_pago_isp · consumo_datos · consumo_snapshot · contratos_historial · ips_asignadas · drift_detectado · reconciliation_log · nodos · nodos_mediciones · ordenes_trabajo · tickets_comentarios · portal_solicitud_plan · google_client_contacts · eventos_sistema · notificaciones · usuarios_rols

12.6 Inventario de APIs

Superficie	Controladores	Endpoints	Autenticación
API del ERP (<code>/api/v1/*</code>)	43	~520	JWT operador + licencia + rol
API del Portal (<code>/api/v1/portal/*</code>)	2	28	JWT propio del portal (cookies)
Webhooks entrantes	3	4	Firma / token del proveedor
Callbacks de OpenVPN	1	5	Token / CN
Health	1	5	Público
API interna Python (<code>:8001</code>)	—	~40	API key, solo <code>127.0.0.1</code>

Top de superficie por controlador

Controlador	Endpoints
<code>olt-nativo.controller.ts</code>	~150
<code>pagos.controller.ts</code>	32
<code>openvpn</code> (2 controladores)	30
<code>facturacion</code> + <code>comprobantes-config</code>	30
<code>mikrotik.controller.ts</code>	28
<code>contratos.controller.ts</code>	25
<code>portal.controller.ts</code>	20

12.7 Inventario de eventos

Eventos de notificación (15)

`FACTURA_EMITIDA` · `PAGO_RECIBIDO` · `SERVICIO_SUSPENDIDO` · `SERVICIO_REACTIVADO` · `BIENVENIDA` · `PAGO_VENCE_HOY` · `PAGO_VENCIDO` · `PRORROGA_CONCEDIDA` · `ALERTA_EGRESO` · `EMISOR_CAIDO` · `EMISOR_CONECTADO` · `ROUTER_CAIDO` · `ROUTER_CONECTADO` · `FTTH_ACTIVADO` · `IPTV_LINE_CREADA` · `OUTBOX_RED_AGOTADO`

Eventos de dominio (5)

`cliente.created` · `instalacion.completed` · `pago.registered` · `visita.scheduled` · `contrato.suspended` · `ftth.inventario.reobserver` · `GATEWAY_EVENTS.PROVIDER_ACTIVATED`

Eventos de WebSocket (3)

`OLT_SYNC_PROGRESS` · `OLT_SYNC_COMPLETED` · `OLT_SYNC_ERROR`

Nota arquitectónica: el bus es **in-process**; los listeners encolan en Bull para cruzar procesos. No hay catálogo versionado ni contrato de payload.

12.8 Inventario de procesos automáticos

Cron jobs (29) por frecuencia

Frecuencia	Tareas
30 s	<code>xui-monitor.tick</code>
1 min	<code>promesas.procesarVencidas</code> · <code>monitoreo.runCycle</code>
2–5 min	<code>ztp.watchPendingReinjection</code> · <code>ftth.procesarAnulaciones</code> · <code>ftth.liberarBloqueados</code> · <code>outbox.barridoProgramado</code> · <code>outbox.barrerClaimsExpirados</code> · <code>promesas.reintentarPendientes</code>
10–15 min	<code>ftth.reintentarRollbacks</code> · <code>ftth.verificarWan</code> · <code>notif-orphan-cleanup</code> · <code>pagos.reconciliarPagosNoAplicados</code> · <code>portal.recolectar</code> · <code>notif-reconciler</code> · <code>reconciliador.reconciliar</code>
20–30 min	<code>tr069.verificarDrift</code> · <code>tr069.verificarStaleness</code> · <code>ftth.limpiarIdsHuerfanos</code> · <code>reconciliador.reconciliarFtthOnu</code> · <code>ftth.adoptarHuerfanos</code> · <code>openvpn.limpiarWizardsAbandonados</code>
6 h	<code>licencia.recargaPeriodica</code> · <code>olt.syncPeriodico</code>
Diaria (madrugada)	03:00 <code>licencia.validacionDiaria</code> + <code>auditoria.purgar</code> · 03:30 <code>ztp.reconciliarDiario</code> · 03:40 <code>tr069.desendurecerAuthResidual</code> · 04:20 <code>tr069.barrerTtl</code> · 04:40 <code>mikrotik.reconciliarDiario</code>

Colas Bull (6) y sus jobs

Cola	Jobs	Concurrencia
<code>cobranza</code>	<code>detectar-morosos</code> · <code>suspender-contrato</code> · <code>reactivar-contrato</code> · <code>evaluar-prorroga</code> · <code>vencer-prorroga</code> · <code>procesar-pago</code>	por defecto
<code>facturacion</code>	<code>marcar-vencidas</code> · <code>generar-mensual</code> · <code>generar-facturas-empresa</code> · <code>generar-factura-contrato</code>	por defecto
<code>notificaciones</code>	<code>notif-envio</code> (+ tipos de aviso)	5
<code>campanas</code>	<code>campana-masiva</code>	1 (goteo 12 s + jitter)
<code>google-sync</code>	<code>sync-contact</code> · <code>contacts-bulk</code> · <code>calendar-event</code> · <code>drive-backup</code> · <code>geocode-address</code>	por defecto
<code>mikrotik-velocidad</code>	<code>sincronizar-router</code> · <code>cambiar-velocidad</code>	por defecto
<code>mikrotik-jobs</code>	<code>mk-suspender</code> · <code>mk-reactivar</code> · <code>mk-sync-velocidades</code>	declarada, no usada (el trabajo va por outbox)

Watchers de invariante (los que reparan el sistema)

Watcher	Invariante que sostiene
<code>reintentarRollbacksFallidos</code>	Nunca borrar <code>ftth_onu_registro</code> con la OLT sucia
<code>adoptarOnusHuerfanos</code>	Nunca un <code>ont</code> en la OLT sin registro en el ERP
<code>procesarAnulaciones</code>	Un wizard no confirmado se anula por completo
<code>liberarBloqueados</code>	Ningún lock FTTH queda colgado

Watcher	Invariante que sostiene
<code>barrerClaimsExpirados</code>	Ningún comando de red queda en <code>EN_PROCESO</code> huérfano
<code>limpiarWizardsAbandonados</code>	Ningún cert VPN queda reservando IP
<code>limpiarIdsHuerfanos</code>	El pool de ONU-ID refleja la OLT
<code>reconciliarPagosNoAplicados</code>	Ningún pago queda sin aplicar
<code>limpiarHuerfanosEnProceso</code>	Ninguna notificación queda en <code>EN_PROCESO</code>

12.9 Inventario de integraciones

Integración	Acoplamiento	Degradable	Puerto/abstracción
GenieACS	Medio	Sí	<code>ztp/genieacs.driver.ts</code>
MikroTik RouterOS	Alto (3 caminos)	Parcial	Ninguna común
OLT Huawei/V-SOL	Bajo	Sí	<code>IOltProvider</code> + drivers Python
SmartOLT / AdminOLT	Bajo	Sí	<code>IOltProvider</code>
OpenVPN	Alto	No	Ninguna
Mercado Pago	Alto	Sí	Existe puerto, no lo usa
RENIEC	Bajo	Sí	Servicio dedicado + cache
Google Workspace ×4	Bajo	Sí	Cola <code>google-sync</code>
Evolution API	Medio	Sí	Strategy
WhatsApp Web	Alto	Sí	Proceso PM2 aislado
SMTP	Bajo	Sí	Strategy
XUI.ONE	Bajo	Sí	Servicio degradable
Servidor de licencias	Máximo	No, por diseño	Guard global
Certbot / Let's Encrypt	Bajo	—	—
OpenStreetMap	Bajo	—	Frontend

12.10 Inventario de componentes reutilizables

Backend — `common/`

Categoría	Componentes
Dominio	<code>resultado-operacion.ts</code>
Resiliencia	<code>module-health.service</code> · <code>circuit-breaker.registry</code> · <code>watcher-heartbeat.service</code> · <code>redis-lock.service</code> · <code>queue-pause.service</code> · <code>degradable.interface</code>

Categoría	Componentes
Aspectos	2 guards + 4 interceptors + 1 filtro + 3 decoradores
Utilidades	<code>encryption</code> · <code>ip</code> · <code>pagination</code> · <code>pg-result</code> · <code>telefono</code>
Observabilidad	<code>errores-proceso.ts</code>
Base	<code>base.entity</code> · <code>response.dto</code> · <code>service-types</code>

Backend — reutilizables de dominio no ubicados en `common/`

`capability.engine.ts` (motor genérico, explícitamente diseñado para otros dominios) · `PoliticaFacturacionService` · `AplicadorFacturaService` · CTE `PUNTOS_SERVICIO` · `FtthOperacionLockService` · `OltProviderRegistry` + `OltOperationRouter`

Frontend

Categoría	LOC
<code>ui/</code> (8 archivos)	544
<code>shared/</code> (5)	201
<code>atoms/</code> (4)	271
<code>organisms/</code> (1)	311
<code>molecules/</code> (0)	0
Total reutilizable	1.327 (2,3 %)
Hooks	5
Stores	4
Clientes API	27

12.11 Inventario de tests (protección de invariantes)

Test	Invariante protegido
<code>frontera-dinero.spec.ts</code>	Un solo escritor del saldo
<code>extorno.spec.ts</code>	El extorno es la única reversión
<code>politica-facturacion.service.spec.ts</code>	Fórmula única del ciclo de cobro
<code>deuda-por-contrato.service.spec.ts</code>	Cálculo de deuda
<code>pagos.reconciliacion.spec.ts</code>	Pagos no aplicados
<code>estados-sql-validos.spec.ts</code>	Los estados usados en SQL existen
<code>outbox-red.claim.spec.ts</code>	Dos instancias PM2 no toman el mismo comando
<code>resultado-operacion.spec.ts</code>	Clasificación de resultado y traducción HTTP

Test	Invariante protegido
<code>contrato-adaptador.spec.ts</code>	No inferir reintentabilidad de un código HTTP
<code>ftth-maquina-estados.spec.ts</code>	Transiciones legales e idempotencia derivada
<code>planta-externa-maquina-estados.spec.ts</code>	Ídem en planta externa
<code>olt-ownership-guard.spec.ts</code>	Propiedad de recursos OLT
<code>olt-perfiles-idempotencia.spec.ts</code>	Perfiles idempotentes
<code>provision-ftth.autoconfig.spec.ts</code> · <code>.barrido-carril.spec.ts</code>	Auto-config y barrido del carril
<code>tr069-staleness.service.spec.ts</code>	Sesiones ACS rancias
<code>router-zombi.spec.ts</code>	La baja de router limpia el cliente VPN
<code>subnet-route.remove.spec.ts</code>	Retiro de ruta
<code>portal-auth.aislamiento.spec.ts</code>	Un token de portal no accede a otro tenant
<code>portal-cookies.spec.ts</code>	Manejo de cookies del portal
<code>descripcion-consolidada.spec.ts</code>	Descripción del <code>ont</code> en la OLT
<code>presupuesto-optico.spec.ts</code>	Cálculo óptico
<code>olt-capability-catalog.spec.ts</code> · <code>olt-model-catalog.spec.ts</code>	Catálogos de capacidad
<code>olt-compliance-rules.spec.ts</code>	Reglas de compliance
<code>capability.engine.spec.ts</code> (×2)	Motor de capacidades
<code>genieacs.driver.spec.ts</code> · <code>registry.spec.ts</code> · <code>resolver.spec.ts</code>	Subsistema ZTP
<code>cwmp-auth.service.spec.ts</code>	Autenticación CWMP
<code>ztp.service.reconcile.spec.ts</code>	Reconcile ZTP
<code>contrato-onu-config.service.spec.ts</code>	Config de ONU por contrato

Lectura: la cobertura es baja en volumen pero **quirúrgica en criterio** — cada test protege un invariante que ya falló en producción. Es el inventario de las cicatrices del sistema.

12.12 Inventario de deuda declarada (trabajo abierto conocido)

Elemento	Estado
Cobranza Etapa II (pasarelas)	Pendiente a propósito — puerta de estabilidad de 30 días
Planta Externa Fases 2-3	En pausa por decisión de diseño
SUNAT / Facturación electrónica	No implementado (solo página)
Inventario / almacén	No implementado como módulo
Proveedor de SMS	No existe
ACS URL por OMCI	Objetivo abierto; hoy solo converge Option 43
Señal FTTH en el listado	<code>rx_power_dbm</code> y <code>metricas_onu_optical</code> vacías

Elemento	Estado
Tab ONU/Router del cliente	Pendiente de eliminar tras validar el modal
Suite de facturación	No compila
sql:check en CI	No integrado
Revisión del reconciliador nocturno	Pendiente registrado
Campos de config de cliente sin cablear	mora , reconexión , esquemaImpuesto , impuesto1 , avisoPantalla
migracion/	Directorio vacío
Migración MikroWISP	No iniciada; requiere diseño previo
Dependencias sin uso	telegraf , twilio , net-snmp
Cola mikrotik-jobs	Declarada, no usada
molecules/	Directorio vacío
mock-data/	En el árbol de producción

Capítulo 13 — Recomendaciones Estratégicas para la Consolidación Arquitectónica de Datafast ERP

13.0 Tesis del capítulo

El diagnóstico completo de la Etapa II se resume en una frase:

Datafast ERP no carece de arquitectura. Carece de cobertura uniforme de una arquitectura que ya inventó, ya probó en producción y ya demostró correcta.

Cada patrón valioso del sistema —VIO, outbox transaccional, máquina de estados declarativa, saga con bitácora write-ahead, **ResultadoOperacion**, puertos y adaptadores, patrón degradable— nació **reactivamente**, resolviendo un incidente concreto, y se aplicó **solo donde dolió**. Ninguno se declaró obligatorio. Por eso un módulo nuevo no los hereda: los hereda quien se acuerda.

En consecuencia, **la mayoría de las recomendaciones de este capítulo no introducen nada nuevo**. Extienden al resto del sistema lo que el propio equipo ya construyó bien en uno o dos sitios. Eso las hace baratas, de bajo riesgo y culturalmente aceptables: no piden al equipo aprender una arquitectura ajena, piden aplicar la propia con consistencia.

Orden de prioridad

Prioridad	Criterio	Recomendaciones
CRÍTICA	Puede causar daño irreversible a clientes o datos	R1 · R2 · R3 · R4
ALTA	Compromete la evolución del sistema a 12 meses	R5 · R6 · R7 · R8 · R9
MEDIA	Mejora sostenida de mantenibilidad	R10 · R11 · R12 · R13
BAJA	Higiene y consistencia	R14 · R15
TRANSVERSAL	Gobierno arquitectónico permanente	R16

PRIORIDAD CRÍTICA

R1 — Convertir la protección contra reescritura masiva de ONUs en una regla explícita

1. Problema identificado `ZtpReconcileCron.reconciliarDiario` (03:30 `America/Lima`) reescribe SSID, clave WiFi y credenciales de acceso web de toda ONU con `contrato_onu_config.provisioning_enabled = true` y `last_applied_revision IS NULL`. Hoy no explota **solo porque** `adoptarOnusHuerfanas` **no crea** `contrato_onu_config` — un efecto lateral de cómo está escrita esa función, no una regla. (Riesgo R-01; `CLAUDE.md` \$Migraciones de ONUs.)

2. Análisis técnico La garantía actual es **estructural pero implícita**, que es la peor combinación: funciona, y nadie sabe que depende de ella. Cualquier desarrollador que "complete" el registro creando la config para ONUs adoptadas —una acción que parece correcta y ordenada— detona la mina sin señal previa. El daño es masivo (todo el parque migrado), simultáneo (un solo cron), nocturno (nadie lo ve ocurrir) e irreversible en la práctica: nadie tiene la clave WiFi que el cliente puso hace tres años. Hay **dos migraciones planificadas** (SmartOLT 205 ONUs, MikroWISP), es decir, la probabilidad no es teórica: es una fecha.

3. Recomendación arquitectónica Elevar la garantía de efecto lateral a **invariante codificado y verificado**:

- Un origen explícito por cada fila de `contrato_onu_config` (`origen: 'erp' | 'adoptada' | 'migrada'`), y que el reconcile **solo actúe sobre** `erp` .
- Un test que ejercite el invariante y **nombre el incidente potencial**, según la regla F-12 del propio proyecto.
- Un *pre-flight* obligatorio en cualquier migración que ejecute la consulta de conteo de drift y **falle en seco** si el resultado supera un umbral.

4. Justificación técnica Es la aplicación directa de una regla que el ERP **ya tiene escrita** y ya aplicó a otro caso: "*una afirmación sobre el propio sistema es una afirmación sin verificar hasta que un test la demuestra*" (VIO hacia adentro, F-13). El precedente es exacto: el comentario del outbox garantizaba una propiedad de concurrencia que era falsa y nadie lo verificó — lo verificó producción. Aquí la propiedad es "el reconcile no toca ONUs preexistentes", y hoy tampoco la verifica nadie.

5. Beneficios esperados

- *Corto plazo*: desbloquea las migraciones pendientes con seguridad demostrable en vez de confiada.
- *Mediano*: el reconcile pasa de ser un proceso temido a uno auditable.
- *Largo*: el ERP puede absorber parques ajenos —que es el modelo de crecimiento del negocio— sin que cada migración sea un acto de fe.

6. Prioridad: CRÍTICA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Bajo — <code>ztp.service.ts</code> , <code>contrato-onu-config.service.ts</code> , <code>adoptarOnusHuerfanas</code>
Base de datos	Bajo — una columna + migración
Frontend	Nulo
Infraestructura	Nulo
Operación	Alto y positivo — desbloquea migraciones

8. Lineamientos para el equipo

- Una ONU que el ERP **no aprovisionó** se **observa y se respeta**; jamás se reconfigura.
- Ningún proceso automático modifica configuración de un CPE en producción sin que un campo explícito lo autorice.
- Toda migración ejecuta el pre-flight de drift **antes** y adjunta su resultado al registro de la migración.

R2 — Hacer visible y redundante el plano de intención (`worker-auxiliary`)

1. Problema identificado Los 29 crons, las 6 colas, el outbox de red y los 9 watchers de invariante corren en **un solo proceso PM2**. Si muere, el ERP sigue respondiendo con total normalidad mientras nadie se corta, nadie se reactiva, ningún comando de red se aplica y ningún watcher repara nada — **sin ninguna señal en la interfaz**. (Riesgo R-02.)

2. Análisis técnico Es el fallo más peligroso posible en un sistema de este tipo: **silencioso y con apariencia de normalidad**. El operador ve el ERP funcionando y concluye que todo está bien; el cliente que pagó sigue cortado; el moroso sigue navegando; la ONU huérfana sigue huérfana. Y cuanto más tiempo pasa, mayor es el volumen de trabajo acumulado que se aplicará de golpe al reiniciar.

Existen ya las piezas para detectarlo — `WatcherHeartbeatService` , `GET /admin/sistema/watchers` , `GET /outbox-red/status` — pero **son consultables, no vigilantes**: hay que ir a mirarlas. Nadie mira lo que parece que funciona.

3. Recomendación arquitectónica Tres medidas, en orden de coste creciente:

- **Alarma de latido**: si `api-core` no ve latido reciente de los watchers, lo muestra como alerta persistente en la interfaz. El proceso que **sí** responde es quien debe denunciar al que no.
- **Presupuesto y cap por cron**: cada tarea declara su límite de trabajo por ejecución y su tiempo máximo; el que lo excede lo registra en vez de correr indefinidamente (resuelve además R-10).
- **Segregación por criticidad**: separar el plano de dinero/red (outbox, cobranza, watchers de invariante) del plano accesorio (Google sync, campañas, XUI) en dos procesos, de modo que una tarea accesorio no pueda tumbar el drenado de la red.

4. Justificación técnica La segmentación por rol **ya es el patrón del sistema** y ya se aplicó tres veces por causa real (Chromium, migraciones, secretos del frontend). Esta recomendación no propone un mecanismo nuevo: propone aplicar el mismo criterio una cuarta vez, con la misma justificación —"si se descontrola muere solo, sin arrastrar lo crítico"— que ya está escrita en `ecosystem.config.js` .

5. Beneficios esperados

- *Corto plazo*: deja de ser posible que el ERP esté roto sin que nadie lo sepa.
- *Mediano*: los crons dejan de competir entre sí en la ventana de madrugada.
- *Largo*: base para escalar el plano automático horizontalmente sin tocar el plano de negocio.

6. Prioridad: CRÍTICA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Medio — cada <code>@Cron</code> declara presupuesto
Frontend	Bajo — un indicador de salud persistente
Base de datos	Nulo
Infraestructura	Medio — un proceso PM2 más (atención a R-19: memoria)

Capa	Impacto
Operación	Alto y positivo

8. Lineamientos para el equipo

- Todo proceso de fondo **late**. Un watcher sin latido no existe para el sistema.
- Todo cron declara **cap de trabajo y tiempo máximo**. Ninguno itera sin límite.
- Un fallo silencioso es peor que uno ruidoso: ante la duda, **hacer ruido**.

R3 — Garantizar el aislamiento multi-tenant por mecanismo, no por convención

1. Problema identificado El filtrado por `empresa_id` depende de que **cada una de las 445 consultas crudas** lo incluya. Los índices UNIQUE por empresa impiden que los datos **colisionen**, pero nada impide que se **lean entre sí**. No hay Row-Level Security ni guard central. (Riesgo R-03.)

2. Análisis técnico Es un fallo de la peor categoría: **una omisión no produce error, produce datos de otra empresa**. No hay excepción, no hay log, no hay síntoma. El sistema responde con normalidad y con información ajena. La probabilidad crece linealmente con el número de consultas, de tenants y de desarrolladores — y las tres magnitudes crecen.

Además, el ERP ya demostró conocer este modo de fallo en otro dominio: la directriz del `iroute` dice literalmente que *"dos routers reclamando la misma red no falla ruidosamente, da la respuesta equivocada con naturalidad"*. Es exactamente el mismo patrón de error, aplicado a datos en lugar de a rutas.

3. Recomendación arquitectónica Defensa en dos capas, ninguna de las cuales requiere reescribir consultas:

- **Row-Level Security en PostgreSQL** sobre las tablas con `empresa_id`, con la empresa activa fijada por sesión de conexión. Convierte la omisión de "fuga silenciosa" en "cero filas".
- **Un test de barrido** que detecte consultas crudas sobre tablas con `empresa_id` que no filtren por él, integrado en CI junto con el `sql:check` que ya existe y aún no está en CI.

4. Justificación técnica RLS mueve la garantía del lugar donde se puede olvidar (445 consultas) al lugar donde solo se declara una vez (la tabla). Es el mismo razonamiento que llevó al equipo a poner tres invariantes en triggers: *el invariante no debe depender de la disciplina del llamador*. La consistencia con esa decisión previa es total.

5. Beneficios esperados

- *Corto plazo*: elimina la clase entera de fallo, incluidas las consultas que hoy ya la tengan sin que nadie lo sepa.
- *Mediano*: permite crecer en número de tenants sin auditoría manual.
- *Largo*: condición necesaria para operar como plataforma multi-empresa con garantías contractuales de aislamiento.

6. Prioridad: CRÍTICA

7. Impacto sobre el sistema

Capa	Impacto
Base de datos	Alto — políticas RLS + migración
Backend	Medio — fijar la empresa activa por sesión de conexión
Frontend	Nulo
Infraestructura	Bajo

Capa	Impacto
Operación	Requiere validación cuidadosa: RLS mal configurado devuelve cero filas

8. Lineamientos para el equipo

- Toda tabla nueva con datos de negocio nace con `empresa_id` y con su política de aislamiento.
- Ninguna consulta asume el filtrado; el filtrado es del motor.
- El aislamiento se **prueba**, no se supone.

R4 — Una sola definición del cálculo de deuda

1. Problema identificado La deuda de un contrato se calcula en **cuatro lugares**: `fn_calcular_deuda_contrato` (PostgreSQL), `deuda-por-contrato.service.ts`, `pagos.service.ts` (`/cliente-deuda`) y `cobranza.worker.ts` (`detectar-morosos`). Divergen potencialmente en el tratamiento de cargos pendientes, notas de crédito, adelantos y promesas de pago vigentes. Ningún test verifica que coincidan. (Riesgo R-04.)

2. Análisis técnico Es el mismo defecto que **ya causó un incidente real en este sistema**: había cuatro copias del `UPDATE` que aplicaba dinero, y la de `adelantos` había perdido el guard de estado y aplicaba saldo a favor contra facturas **anuladas**. La corrección fue un único aplicador protegido por `frontera-dinero.spec.ts`.

La deuda es más peligrosa que la aplicación, porque **decide cortes de servicio**. Cuatro definiciones significan que el ERP puede cortar a quien no debe y no cortar a quien sí, y responder distinto según por dónde se le pregunte —la ficha del cliente, el reporte de cobranza y el worker que corta pueden discrepar el mismo día sobre el mismo abonado.

3. Recomendación arquitectónica Una sola definición de "cuánto debe un contrato", en el punto común, con los cuatro consumidores pasando por ella. La elección entre función de BD y servicio de dominio debe decidirse una vez y documentarse, con un criterio claro: **si algún escritor no pasa por la aplicación, la definición va en la base; si todos pasan, va en el dominio y se testea**.

4. Justificación técnica Es la regla del propio proyecto —"corregir en el punto común, no en cada sitio donde se manifiesta", de la que salió el CTE `PUNTOS_SERVICIO` — aplicada al dominio donde el coste del error es dinero y servicio. El equipo ya ejecutó esta consolidación dos veces con éxito (ubicación del abonado, aplicación de dinero); esta es la tercera y la más valiosa.

5. Beneficios esperados

- *Corto plazo*: el ERP responde una sola cosa cuando se le pregunta cuánto debe alguien.
- *Mediano*: cambiar la política de deuda (incluir/excluir promesas, adelantos, notas de crédito) pasa a ser un cambio de una línea con un test que lo respalda.
- *Largo*: condición previa para cualquier integración de cobranza automática — un adaptador de pasarela no puede apoyarse en cuatro verdades.

6. Prioridad: CRÍTICA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Alto — toca facturación, pagos y cobranza
Base de datos	Medio — decidir el destino de <code>fn_calcular_deuda_contrato</code>

Capa	Impacto
Frontend	Bajo
Operación	Alto y positivo

8. Lineamientos para el equipo

- Antes de escribir una consulta, **buscar si ya existe**. Un `grep` por el concepto de negocio cuesta un minuto.
 - Si existe y sirve, usarlo; si no encaja, **extenderlo**, nunca clonarlo.
 - Si la duplicación es inevitable, **escribir en el código por qué**, y qué hay que cambiar en los dos sitios si cambia la regla.
-

PRIORIDAD ALTA

R5 — Extender las garantías del plano FTTH al plano MikroTik/WISP

1. Problema identificado Outbox + máquina de estados + saga + VIO están implementados en FTTH. El plano MikroTik tiene outbox **parcial** (solo lo disparado por el negocio), **sin** máquina de estados, **sin** saga y con verificación solo como detección posterior (`/velocidad/discrepancias`). Las operaciones interactivas del operador en `/red/routers` son **síncronas y sin ninguna garantía**. (Riesgos R-08, R-17; Cap. 7.9.)

2. Análisis técnico Los incidentes que FTTH **ya no puede tener** —huérfanos, dobles ejecuciones, timeouts leídos como fallo, transiciones ilegales— MikroTik **sí puede tenerlos**, y el parque WISP no es marginal. Además hay **tres caminos independientes** hacia el mismo hardware (`node-routeros` , `ssh2` , `mikrotik_pool.py`), cada uno con su política de conexión, reintentos y credenciales.

El caso del cambio de plan lo ilustra: son **dos operaciones independientes** (BD y router) que pueden divergir, y la divergencia se descubre después con un endpoint de discrepancias. En FTTH esa misma operación pasa por lock, write-ahead, VIO y clasificación de resultado.

3. Recomendación arquitectónica

- Un **puerto único hacia MikroTik** (`IRouterProvider`), copiando literalmente las tres reglas de `IOltProvider` : no propagar excepciones, medir latencia, **no tocar la BD desde el adaptador**.
- Una **máquina de estados declarativa** para el servicio WISP, hermana de la de FTTH.
- Que **toda** mutación de MikroTik —incluidas las del operador— pase por el outbox, con la operación interactiva devolviendo "encolado" en vez de bloquear 30 s.

4. Justificación técnica No hay que diseñar nada: el patrón existe, está probado en el dominio más hostil (SSH a un MA5800 con sesiones VTY limitadas) y tiene sus reglas escritas en el propio contrato. Replicarlo a un dominio **más benigno** (API RouterOS sobre VPN) es de riesgo bajo. Y ya hay precedente de replicación voluntaria: la máquina de estados de `planta-externa` .

5. Beneficios esperados

- Corto plazo*: desaparece la clase de fallo de "la BD dice una velocidad y el router otra".
- Mediano*: un solo punto de política de conexión y reintento hacia MikroTik.
- Largo*: el plano de red completo con garantías homogéneas, prerequisite para automatizar más operaciones sin multiplicar el riesgo.

6. Prioridad: ALTA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Alto — <code>mikrotik</code> (10 consumidores)
Frontend	Medio — operaciones que devuelven "encolado" en vez de resultado inmediato
Base de datos	Bajo — una máquina de estados y su columna
Operación	Medio — cambia la percepción de inmediatez del operador

8. Lineamientos para el equipo

- Toda mutación contra hardware externo pasa por el outbox. **Sin excepciones por comodidad de UI.**
- Todo adaptador de protocolo cumple las tres reglas de `IoltProvider`.
- Ninguna operación de hardware se ejecuta dentro del ciclo de vida de un request HTTP.

R6 — Modelar el cambio de ONU como operación de primera clase

1. Problema identificado No existe endpoint, servicio, transición ni saga para sustituir la ONU de un contrato. Hoy se improvisa como **baja + alta**. (Riesgo R-09; Cap. 7.8.)

2. Análisis técnico El reemplazo de ONU es una operación **rutinaria** en un ISP: avería, upgrade de modelo, cambio de tecnología. Componerla con dos operaciones destructivas produce cinco daños concretos: corte de servicio innecesario; **pérdida de la configuración del abonado** (su SSID y su clave viven en `contrato_onu_config` y la reprovisión los reescribe con el preset); una ventana en la que el contrato existe sin registro FTTH —exactamente el estado que causó el incidente del 21/07—; ausencia total de trazabilidad de qué equipo sustituyó a cuál; y la imposibilidad de solape por `uq_contratos_empresa_onu`.

La ausencia es especialmente llamativa porque el sistema **tiene todas las piezas**: máquina de estados, saga con compensación, pools con reserva, VIO. Falta la transición, no la capacidad.

3. Recomendación arquitectónica Declarar `sustituir_onu` como transición de primera clase en `ftth-maquina-estados.ts`, con su saga propia: reservar recursos de la ONU nueva **antes** de liberar los de la vieja, migrar `contrato_onu_config` conservando SSID y clave del abonado, y registrar la sustitución en el histórico del contrato.

4. Justificación técnica La máquina de estados declarativa existe precisamente para que **falte un caso sea visible**. Ya cumplió esa función una vez: reveló que `desaprovisionar` no aceptaba `suspendido`, el caso más frecuente del negocio. Esta recomendación es la misma lección aplicada a una transición que directamente no está declarada.

5. Beneficios esperados

- *Corto plazo*: el técnico cambia una ONU averiada sin perder la configuración del cliente ni provocar un corte largo.
- *Mediano*: trazabilidad del equipamiento por abonado.
- *Largo*: base para renovación planificada de parque (upgrade masivo de modelo).

6. Prioridad: ALTA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Medio — <code>olt-nativo</code> (transición + saga)
Frontend	Medio — flujo nuevo en el modal de ONU
Base de datos	Bajo — histórico de sustitución
Operación	Alto y positivo

8. Lineamientos para el equipo

- Toda operación **rutinaria** del negocio se modela explícitamente. Componerla con operaciones destructivas no es modelarla.
- En una sustitución, se **reserva antes de liberar**. Nunca al revés.
- La configuración que puso el abonado es **suya**: se migra, no se regenera.

R7 — Completar la capa de repositorio y tipar las tablas críticas

1. Problema identificado La capa de repositorio existe en **6 de 44 módulos** (14 %). Hay **39 tablas sin entidad TypeORM**, entre ellas `comandos_red_pendientes`, `ftth_operacion_lock`, `operacion_wizard`, `operacion_wizard_paso`, `pago_extorno`, `cierre_caja` y `cuentas_bancarias`. Un `ALTER` sobre ellas **no rompe la compilación**. (Riesgos R-05, y ver Corrección 0.1.)

2. Análisis técnico La ironía estructural del sistema: **las tablas que sostienen las garantías más fuertes son las que menos protección de tipos tienen**. El outbox garantiza que ningún comando de red se pierda, y su esquema no está tipado. La saga garantiza que ningún wizard deje basura, y su bitácora tampoco. Un renombrado de columna en cualquiera de ellas compila perfectamente y rompe en producción, en el plano automático, de madrugada — combinándose con R-02 para producir un fallo silencioso.

3. Recomendación arquitectónica Dos frentes, empezando por el más barato:

- **Tipar primero las tablas de coordinación y de dinero** — no las 39, sino las que sostienen invariantes. Es un subconjunto pequeño y de altísimo retorno.
- **Extender el patrón de repositorio** siguiendo la forma ya decidida en `ContratoRepository`, empezando por los módulos que ya lo tienen a medias (conviven repositorio y acceso directo en el mismo módulo).

4. Justificación técnica No es adopción de un patrón nuevo: es **completar** uno que ya está en producción, en los módulos correctos y con una forma probada. El coste de aprendizaje es cero y el criterio de diseño ya está tomado.

5. Beneficios esperados

- *Corto plazo*: el compilador empieza a detectar cambios de esquema en las piezas críticas.
- *Mediano*: las consultas se pueden refactorizar con herramientas en lugar de a mano.
- *Largo*: condición previa para cualquier evolución del modelo de datos con seguridad.

6. Prioridad: ALTA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Alto en extensión, bajo en riesgo — es incremental por tabla
Base de datos	Nulo (no cambia el esquema)
Frontend	Nulo
Operación	Nulo durante el cambio, positivo después

8. Lineamientos para el equipo

- **Toda tabla nueva nace con entidad TypeORM**. Sin excepciones.
- Columnas `string | null` llevan `type:` explícito — sin él, SWC crashea el backend en frío.
- Un módulo con repositorio **no consulta por fuera del repositorio**.

R8 — Instrumentar el sistema (observabilidad mínima viable)

1. Problema identificado No hay APM, ni trazas, ni métricas. `logging: false` en TypeORM. La propia Etapa I tuvo que declarar **cinco preguntas que no puede responder**: frecuencia real de endpoints, latencias p50/p95/p99, volúmenes por tabla, índices realmente usados (de 376 creados) y comportamiento bajo carga. (Riesgo R-06.)

2. Análisis técnico Sin instrumentación, toda decisión de rendimiento es una hipótesis y toda corrección es un acto de fe. Los incidentes documentados lo demuestran: los 287 s de REACTIVAR se descubrieron por observación manual, y el segundo defecto **solo se hizo visible al corregir el primero** — con métricas habrían sido dos líneas en un gráfico.

Es además el **multiplicador de todos los demás riesgos**: sin medición no se sabe si el reconciliador satura el MA5800, si los 376 índices sirven, si la cola del worker crece, ni si una recomendación de este documento funcionó.

- 3. Recomendación arquitectónica** Mínimo viable, en orden de retorno decreciente:
- **Métricas del plano de intención**: profundidad de las 6 colas, edad del comando más antiguo del outbox, latido de cada watcher, duración de cada cron. Es el dato que evita el fallo silencioso de R-02.
 - `pg_stat_statements` activado, para saber qué consulta duele de verdad en vez de deducirlo.
 - **Latencia y contador por endpoint**, para que la próxima auditoría pueda responder lo que esta no pudo.
- 4. Justificación técnica** Alineado con la regla más profunda del proyecto —"si el diagnóstico no se apoya en una medición, es una hipótesis, y debe decirse así"—. Hoy esa regla se cumple heroicamente, a base de `tcpdump`, sniffers y scripts `_monitor*.mjs` puntuales. Instrumentar es convertir ese heroísmo en infraestructura.

- 5. Beneficios esperados**
- *Corto plazo*: se ve la cola del worker y la edad del outbox; el fallo silencioso deja de serlo.
 - *Mediano*: las decisiones de índices y consultas se toman con datos.
 - *Largo*: capacidad de planificación de capacidad al crecer el parque.

6. Prioridad: ALTA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Medio — instrumentación transversal
Infraestructura	Alto — un recolector más en un VPS con RAM justa (ver R-19)
Base de datos	Bajo
Operación	Alto y positivo

- 8. Lineamientos para el equipo**
- Todo proceso de fondo **expone su progreso**, no solo su resultado.
 - Una afirmación de rendimiento sin medición se enuncia como hipótesis.
 - Si una corrección no se puede verificar con un número, no está verificada.

R9 — Descomponer olt-nativo por las fronteras que ya tiene

1. Problema identificado 25.659 LOC, 41 servicios, 24 entidades, 11 crons y un controlador de **1.845 líneas con ~150 endpoints**, cubriendo 8 subdominios. (Riesgo R-07.)

2. Análisis técnico Parte del tamaño es **irreductible**: el dominio FTTH es genuinamente el más complejo del negocio y no se simplifica dividiéndolo por gusto. Pero el módulo **ya se está dividiendo solo**: el árbol contiene `capability/` , `compliance/` , `domain/` , `ztp/` , `providers/` , `interfaces/` , `types/` y `cron/` — fronteras reales que el equipo trazó por necesidad. Lo que falta es formalizarlas.

El controlador es el caso más claro y más barato: 150 endpoints en un archivo garantizan conflictos de merge y hacen imposible leer la superficie del módulo.

3. Recomendación arquitectónica Formalizar las fronteras existentes en submódulos NestJS —**sin cambiar rutas ni contratos**— empezando por lo de menor riesgo:

1. Dividir el controlador por grupo funcional (mismas rutas, varios archivos).

2. Promover `ztp/` , `capability/` y `compliance/` a submódulos con su propio `@Module` .

3. Separar el subdominio de **pools de recursos**, que es genérico y reutilizable (ver R-12).

4. Justificación técnica Es refactor estructural sin cambio de comportamiento: bajo riesgo, verificable con los tests existentes, y respeta el principio de no reconstruir. Las fronteras no las inventa este documento: las descubrió el equipo escribiendo el código.

5. Beneficios esperados

- *Corto plazo*: menos conflictos de merge; la superficie del módulo se puede leer.
- *Mediano*: los subdominios se pueden testear y evolucionar por separado.
- *Largo*: el módulo más crítico deja de ser el más intimidante para un desarrollador nuevo.

6. Prioridad: ALTA

7. Impacto sobre el sistema

Capa	Impacto
Backend	Alto en volumen, bajo en riesgo — sin cambio de comportamiento
Frontend	Nulo — las rutas no cambian
Base de datos	Nulo
Operación	Nulo

8. Lineamientos para el equipo

- Un controlador no supera un **umbral acordado de endpoints**; al superarlo, se divide por grupo funcional.
- Un subdirectorio con `domain/` , `providers/` o `interfaces/` propios **es un submódulo**; formalizarlo, no dejarlo implícito.

PRIORIDAD MEDIA

R10 — Unificar el patrón de reconciliación

1. Problema identificado Cinco procesos (`reconciliador` 15/30 min, `ztp-reconcile` 03:30 y cada 2 min, `ftth-wan-watcher` 10 min, `address-list-reconciliador` 04:40, `olt-sync` 6 h) verifican estados solapados del mismo parque sin coordinación. `reconciliar()` itera **sin cap ni lock**. (Riesgos R-10, R-16; Cap. 9.3.)

2. Análisis técnico Cinco implementaciones de la misma idea —leer estado real, comparar, actuar, registrar— con cinco criterios de cap, lock, reintento y registro. Todas compiten por el mismo recurso escaso: las **sesiones VTY del MA5800**, que ya provocaron 1.788 reintentos en 4 días cuando un solo proceso se descontroló.

3. Recomendación arquitectónica Un **primitivo común de reconciliación** que aporte lo que hoy cada uno resuelve a su manera: cap de trabajo, lock, presupuesto de tiempo, registro uniforme y respeto de una cuota global de sesiones contra cada equipo.

4. Justificación técnica El sistema ya extrajo un primitivo genérico de un caso concreto y lo hizo bien: `capability.engine.ts` generaliza el patrón de `ztp/capability.engine.ts` con una nota explícita de que el original **no se toca** porque es un contrato congelado. Ese es exactamente el modelo a seguir aquí.

5. Beneficios esperados

- *Corto*: ningún reconciliador puede saturar un equipo.
- *Mediano*: un reconciliador nuevo hereda cap, lock y registro sin escribirlos.
- *Largo*: la ventana de madrugada se planifica en lugar de acumularse.

6. Prioridad: MEDIA

7. Impacto: Backend medio · BD bajo · Frontend nulo · Operación positivo.

8. Lineamientos

- Ningún proceso de fondo itera sin cap.
- Todo acceso a un equipo con recurso escaso respeta una **cuota global**, no una por proceso.

R11 — Migrar Mercado Pago al contrato de cobro que ya existe

1. Problema identificado `pagos/adaptadores/adaptador-cobro.interface.ts` define el contrato de cobro, y `mercadopago.service.ts` —**el único proveedor que cobra dinero real**— no lo implementa.

2. Análisis técnico Una abstracción que ningún caso real ejercita **no está validada**. Si la primera integración nueva descubre que el contrato no absorbe a Mercado Pago, la corrección llegará con tres proveedores encima. El propio README lo dice: *"si la abstracción no lo absorbe, la abstracción está mal y se corrige con un proveedor, no con tres"*.

3. Recomendación arquitectónica Migrar Mercado Pago al contrato **antes** que ningún proveedor nuevo, respetando la puerta de estabilidad de 30 días que ya está documentada.

4. Justificación técnica Es literalmente el paso 3 del plan que el equipo ya escribió. Esta recomendación no propone nada: **confirma que el plan existente es correcto** y recomienda no saltárselo.

5. Beneficios esperados

- *Corto*: el contrato queda validado contra la realidad.
- *Mediano*: añadir Niubiz/Culqi/Izipay pasa a ser un adaptador, no un servicio.
- *Largo*: la cobranza automática crece sin multiplicar caminos hacia el dinero.

6. Prioridad: MEDIA (sube a **ALTA** en el momento en que se decida integrar cualquier pasarela)

7. Impacto: Backend medio · BD bajo · Operación **crítico durante la migración** (toca dinero real).

8. Lineamientos

- **No se integra una segunda pasarela antes de que la primera pase por el contrato.**
- Un timeout cobrando es **indeterminado** : ni se reintenta a ciegas (cobra dos veces) ni se reporta fallo (deja dinero sin registro).

R12 — Extraer el primitivo de pool de recursos

1. Problema identificado Existen al menos **seis pools** con la misma mecánica y seis implementaciones: **service-port** , **mgmt-ip** , **onu-id** , **mgmt-port** , puertos NAP e IPs de segmento. (Cap. 9.5.)

2. Análisis técnico Todos hacen lo mismo: reservar, confirmar, liberar, reconciliar contra la realidad y barrer reservas huérfanas por TTL. Cada uno lo resuelve a su manera, y la reserva huérfana es una clase de fallo ya vivida (cert VPN reservando una IP tras un **UPDATE** directo, 28/07).

3. Recomendación arquitectónica Un primitivo **PoolDeRecursos<T>** con reserva con TTL, confirmación, liberación, reconciliación contra la fuente real y barrido — dejando que cada pool aporte solo su fuente de verdad.

4. Justificación técnica Mismo modelo que R10 y que **capability.engine** : extraer el primitivo sin tocar las implementaciones congeladas, y migrar deliberadamente, nunca por arrastre.

5. Beneficios esperados

- *Corto*: comportamiento uniforme ante reservas huérfanas.
- *Mediano*: un pool nuevo (VLAN, puerto de switch, IP pública) se declara en lugar de programarse.
- *Largo*: menos superficie donde puede aparecer una fuga de recursos.

6. Prioridad: MEDIA

7. Impacto: Backend medio · BD bajo · Frontend nulo.

8. Lineamientos

- Toda reserva de recurso tiene **TTL y barrido**. Una reserva sin caducidad es una fuga con retraso.
- Todo pool se **reconcilia contra la realidad**, no contra su propio registro.

R13 — Consolidar el frontend

1. Problema identificado Tres convenciones de organización simultáneas (**molecules/** vacío, **organisms/** con 1 archivo); **1,8 % de código reutilizable** sobre 57.435 LOC; componentes de hasta 3.776 LOC; tipos duplicados respecto al backend pese a existir Swagger; sin librería de formularios; **2 tests**. (Riesgo R-18.)

2. Análisis técnico El frontend es el único plano del sistema **sin patrones arquitectónicos declarados**. Mientras el backend tiene máquinas de estados, puertos y vocabulario de dominio, el frontend tiene `ClienteDetalle.tsx` con 3.776 líneas. La duplicación de tipos y validaciones crea una clase de divergencia estructural: **el frontend puede aceptar lo que el backend rechaza, y al revés**.

3. Recomendación arquitectónica

- **Una** convención de organización (la de dominio, que es la viva) y eliminar los directorios muertos.
- **Tipos generados desde el Swagger** que el backend ya expone, en lugar de escritos a mano dos veces.
- Umbral acordado de tamaño de componente y descomposición de los que lo superan, empezando por las fichas maestras y wizards.
- Capa de estado de servidor con cache, y librería única de formularios con esquema compartido.

4. Justificación técnica El backend demostró que declarar el patrón funciona. El frontend tiene el mismo problema (crecimiento por acumulación) y aún no aplicó la misma medicina.

5. Beneficios esperados

- *Corto*: un desarrollador nuevo sabe dónde poner un componente.
- *Mediano*: menos peticiones repetidas y menos divergencia de validación.
- *Largo*: la interfaz crece sin que cada pantalla nueva sea un archivo de mil líneas.

6. Prioridad: MEDIA

7. Impacto: Frontend **alto** · Backend bajo (exponer Swagger en build) · Operación nulo.

8. Lineamientos

- **Una** convención de organización.
 - Los tipos de la API **se generan**, no se escriben.
 - Un componente que supera el umbral se descompone antes de añadirle nada.
-

PRIORIDAD BAJA

R14 — Higiene de dependencias y artefactos muertos

Problema: `telegraf`, `twilio` y `net-snmp` instalados sin uso; cola `mikrotik-jobs` declarada y no usada; `migracion/` y `moleculas/` vacíos; `mock-data/` en el árbol de producción; 25 scripts `.mjs` en la raíz, varios ad-hoc con prefijo `_`; ruta `/installl` con typo.

Análisis: cada dependencia sin uso amplía superficie de seguridad y peso de build; cada artefacto muerto hace dudar al siguiente lector sobre si está vivo.

Recomendación: retirar lo no usado; mover los scripts operativos a `scripts/`; documentar los que se conservan.

Justificación: coherente con la regla del propio proyecto: *"una garantía que nadie sostiene es peor que ninguna, porque el siguiente lector construye encima"* — aplicable igualmente al código muerto.

Beneficios: build más ligero, menos superficie, menos ambigüedad. **Prioridad:** BAJA. **Impacto:** transversal bajo.

Lineamiento: una dependencia sin consumidor **se retira**; si se conserva para el futuro, se documenta por qué y hasta cuándo.

R15 — Llevar la configuración externa al repositorio

Problema: credenciales de connreq de GenieACS duplicadas en la provision del ACS y en el `.env`; CCD y certificados OpenVPN solo en el filesystem; crontab del VPS editable desde la UI y fuera de control de versiones. (Riesgo R-20.)

Análisis: una instalación nueva **no es totalmente reproducible desde el repositorio**, lo que contradice el excelente trabajo de portabilidad multi-VPS ya hecho (F-14). Y una divergencia entre el ACS y el `.env` no la detecta nada: se manifiesta como TR-069 que no responde.

Recomendación: verificación de coincidencia al arrancar (probe del patrón degradable), e inventario versionado de lo que vive fuera del repositorio, con su procedimiento de restauración.

Beneficios: instalación reproducible; una divergencia se detecta al arrancar y no en campo. **Prioridad:** BAJA (sube a ALTA antes de instalar un VPS nuevo). **Lineamiento:** todo lo que deba coincidir entre dos sistemas **se verifica automáticamente**; si no se puede verificar, se documenta como riesgo aceptado.

TRANSVERSAL

R16 — Establecer el gobierno arquitectónico permanente

1. Problema identificado Todos los patrones valiosos del sistema **nacieron reactivamente** y **ninguno es obligatorio**. `CLAUDE.md` los documenta con calidad excepcional, pero es un documento que se lee, no un mecanismo que se aplica. Por eso la cobertura es desigual: FTTH tiene todas las garantías y MikroTik casi ninguna; 6 módulos tienen repositorio y 38 no; 4 módulos declaran permisos finos y 40 no.

2. Análisis técnico Este es el **problema raíz del que se derivan casi todos los demás de este documento**. Las recomendaciones R1–R15 corrigen síntomas de una misma causa: **no hay ningún punto del proceso de desarrollo donde se compruebe que una decisión arquitectónica se respetó**. La calidad depende hoy de que quien escribe el código conozca las reglas y se acuerde de aplicarlas — lo cual funciona con un equipo pequeño y con memoria, y deja de funcionar exactamente cuando el sistema crece, que es el objetivo declarado.

Sin esto, en dos años habrá un segundo documento de consolidación describiendo los mismos problemas en módulos distintos.

- 3. Recomendación arquitectónica** Tres mecanismos, ninguno burocrático:
- **Decisiones arquitectónicas registradas (ADR).** Cada decisión estructural se registra con contexto, alternativas y consecuencias. El proyecto **ya escribe esto**, disperso en comentarios excelentes (`ecosystem.config.js` , `IOltProvider` , `pagos/adaptadores/README.md`); falta darle un lugar y un formato.
 - **Checklist de módulo nuevo**, verificado en revisión: ¿degradable o Core Indestructible? ¿repositorio? ¿entidad para toda tabla? ¿ `ResultadoOperacion` si lo invoca un orquestador? ¿máquina de estados si tiene ciclo de vida? ¿outbox si muta hardware? ¿cap y lock si es un cron? ¿ `@RequirePermission` ? ¿filtra por `empresa_id` ?
 - **Verificaciones automáticas en CI:** `sql:check` (existe y no está en CI), typecheck, la suite de tests (hoy no compila la de facturación), y detección de consultas sin `empresa_id` .
- 4. Justificación técnica** El propio proyecto ya demostró que **el mecanismo vence a la disciplina**: el índice UNIQUE parcial resolvió los routers duplicados que la revisión manual no evitaba; el test `frontera-dinero.spec.ts` impide la cuarta copia del `UPDATE` que la convención no impidió; el trigger mantiene el saldo aunque el escritor sea un script manual. La regla escrita en `CLAUDE.md` —“el invariante que solo vive en la doc no es invariante”— aplica al propio `CLAUDE.md` .

- 5. Beneficios esperados**
- *Corto plazo:* un módulo nuevo nace con las garantías puestas en lugar de adquirirlas tras su primer incidente.
 - *Mediano:* la cobertura desigual deja de crecer; la brecha entre FTTH y el resto se cierra por construcción.
 - *Largo:* **el ERP puede crecer sin aumentar su complejidad**, que es literalmente el objetivo declarado de esta etapa.

6. Prioridad: TRANSVERSAL — condiciona el éxito de todas las demás

7. Impacto sobre el sistema

Capa	Impacto
Backend / Frontend / BD	Nulo inmediato, decisivo acumulado
Infraestructura	Bajo — CI
Operación	Alto y positivo

8. Lineamientos para el equipo — el decálogo permanente

- ### 13.17 Secuencia recomendada

```

graph TD
    R16[R16 - Gobierno arquitectónico  
TRANSVERSAL — habilita todo lo demás]
    R9[R9 - Observabilidad  
ALTA — multiplicador]
    R2[R2 - Visibilidad del worker  
CRÍTICA]
    R1[R1 - Protección de migración de  
ONU  
CRÍTICA — bloquea las  
migraciones]
    R3[R3 - Aislamiento multi-tenant  
CRÍTICA]
    R4[R4 - Deuda única  
CRÍTICA]
    R5[R5 - Descomponer olt nativo  
ALTA]
    R13[R13 - Frontend]
    R14[R14 - Higiene]
    R15[R15 - Config externa]
    R7[R7 - Repositorios y entidades  
ALTA]
    R5[R5 - Garantías a MicroTx  
ALTA]
    R12[R12 - Pools unificados]
    R10[R10 - Reconciliación unificada]
    R11[R11 - MercadoPago al contrato]
    R6[R6 - Cambio de ONU  
ALTA]

    R16 --> R9
    R16 --> R2
    R16 --> R1
    R16 --> R3
    R16 --> R4
    R16 --> R5
    R16 --> R13
    R16 --> R14
    R16 --> R15
    R16 --> R7
    R16 --> R5
    R16 --> R12
    R9 --> R10
    R4 --> R11
    R5 --> R6
    R7 --> R5
    R5 --> R12
  
```

1. **R16 va primero** aunque no arregle nada por sí solo: sin él, cada recomendación aplicada se degrada con el tiempo igual que se degradaron las anteriores.
2. **R8 (observabilidad) es un multiplicador**, no una mejora aislada: sin medición no se puede verificar que R2, R5 o R10 funcionaron.
3. **R1 bloquea las migraciones**: mientras no esté, cada migración de parque es una apuesta con el servicio de clientes reales.

13.18 Cierre

Datafast ERP llega a esta etapa con algo que la mayoría de los sistemas de su categoría no tienen: **una arquitectura de resiliencia diseñada a partir de incidentes reales, correctamente diagnosticados y documentados con honestidad**. VIO, el outbox con reclamo atómico, la saga con bitácora write-ahead, el vocabulario de dominio y la puerta de estabilidad de las pasarelas de pago no son prácticas de manual: son conocimiento ganado en producción, y varias de ellas están mejor resueltas aquí que en productos comerciales del sector.

El problema no es lo que falta por inventar. Es que **lo ya inventado no es obligatorio**, y por eso convive un plano FTTH con garantías de nivel industrial junto a un plano WISP casi sin ellas, seis módulos con repositorio junto a treinta y ocho sin él, y una cultura de tests quirúrgicos junto a una suite que no compila.

Consolidar esta arquitectura no significa reconstruirla. Significa **tomar las decisiones que ya se demostraron correctas y convertirlas en la forma por defecto de construir**. Ese es el trabajo de la Etapa III, y este documento es su punto de partida.